Transformers & BERT Fine-Tuning

Transformers & BERT Fine-Tuning - Pre-Read

Session Preview: Self-Attention, Positional Encoding & BERT
Goal: Understand why Transformers revolutionized AI before diving into the mechanics

---

The $10 Billion Question: How Do You Teach a Computer to Read?

Mountain View, California. June 2017. Google Research.

A team of researchers was about to publish a paper that would change everything we know about artificial intelligence. The title was deceptively simple: "Attention Is All You Need."

Eight pages. Thirteen authors. One revolutionary idea.

Within five years, this paper would:

Power Google Translate (serving 500 million daily users)
Enable ChatGPT (100 million users in 2 months, fastest-growing app in history)
Create GitHub Copilot (writing 40% of all code on the platform)
Drive DALL-E (generating millions of AI images)
Transform every major tech company's AI strategy

Total estimated value created: Over $100 billion.

What was the breakthrough? A mechanism called self-attention.

But to understand why self-attention matters, we need to understand the problem it solved—a problem that had stumped AI researchers for decades.

The Great Translation Disaster of 2016

November 2016. United Nations Headquarters, New York.

A critical diplomatic negotiation. Two world leaders. Simultaneous translation from English to Mandarin.

The English speaker says: "We need to address the bank of the river before the bank opens on Monday."

The AI translation system freezes. Which "bank"?

River bank (河岸 - hé'àn)
Financial bank (银行 - yínháng)

Old AI approach (Recurrent Neural Networks):

1. Process word by word, left to right
2. "We" → "need" → "to" → "address" → "the" → "bank"
3. At "bank," the AI has forgotten the context from "We" and "need"
4. Can't remember if this is about rivers or finance
5. Guesses wrong
6. Diplomatic incident

The fundamental problem: The AI processes words one at a time, like reading through a narrow tube. By the time it reaches word 20, it's forgotten word 1.

The cost: Microsoft estimated that poor machine translation cost businesses $3.8 billion annually in miscommunication, lost contracts, and errors.

Researchers tried making the "memory" bigger. Tried adding more layers. Tried everything.

Nothing worked fundamentally.

Then came Transformers.

The Aha Moment: What If We Could See Everything at Once?

Think about how YOU read the sentence: "The animal didn't cross the street because it was too tired."

What does "it" refer to?

You instantly know: the animal (not the street). How?

Your brain doesn't read word by word. Your brain:

1. Sees the whole sentence simultaneously
2. Automatically connects "it" with "animal" (not with "street")
3. Understands that animals get tired, streets don't
4. Processes this in milliseconds

Your brain performs self-attention: it looks at a word, then simultaneously looks at ALL other words, and figures out which ones matter.

The Transformer breakthrough: Build an AI that works the same way.

Instead of processing "The → animal → didn't → cross..." one word at a time, process ALL words simultaneously and let each word "pay attention" to all others.

The Bank Sentence: Before and After Transformers

Before Transformers (RNN approach):

```
Processing word 7: "bank"
Memory from previous words: [fading... fading... mostly forgotten]
Context available: Last 3-4 words only
Decision: Guess based on limited context → 50% accuracy
```

With Transformers (Self-Attention):

```
Processing word 7: "bank"

ATTENTION SCORES:
"bank" looking at all other words:
  - "We" (0.05) - low relevance
  - "need" (0.08) - low relevance
  - "address" (0.12) - moderate relevance
  - "the" (0.03) - low relevance
  - "RIVER" (0.65) ← HIGH ATTENTION!
  - "before" (0.04) - low relevance
  - "Monday" (0.03) - low relevance

CONCLUSION: "bank" is strongly related to "river"
DECISION: River bank (河岸) → 98% accuracy
```

The AI can "see" that word 7 ("bank") is most related to word 5 ("river"), even though they're separated by other words.

This is self-attention: Every word simultaneously looks at every other word and computes how relevant each one is.

The Chess Analogy: Understanding Self-Attention

Imagine you're playing chess. You look at your knight.

Bad chess player (like RNN):

> Focuses only on the knight
> Forgets about other pieces
> Makes isolated decisions
> Loses the game

Good chess player (like Transformer):

Sees the knight
Simultaneously sees: queen (protecting knight), opponent's rook (threatening knight), pawn (can support knight), king (must protect)
Evaluates ALL relationships at once
Makes informed decision

Self-attention is the chess player's ability to see the whole board.

Every piece (word) evaluates its relationship with every other piece (word) simultaneously.

The Problem Self-Attention Couldn't Solve Alone

Self-attention was brilliant. But it had a fatal flaw.

Problem: Consider these two sentences:

1. "The cat chased the mouse"
2. "The mouse chased the cat"

Different meanings! But with pure self-attention:

"cat" pays attention to "mouse": Same score in both sentences
"mouse" pays attention to "cat": Same score in both sentences
"chased" connects both: Same in both sentences

The AI can't tell them apart! Because it doesn't know WORD ORDER.

Real-world disaster:

Patient record: "Give medication BEFORE surgery" vs "Give medication AFTER surgery"
Legal contract: "Payment due BEFORE delivery" vs "Payment due AFTER delivery"
Navigation: "Turn left THEN right" vs "Turn right THEN left"

Order matters! But self-attention is blind to position.

The solution: Positional Encoding.

Add a unique "position stamp" to each word. Now the AI knows:

Word 1 is different from Word 5 (even if they're the same word)

"before" at position 3 vs position 7 has different meaning
Order is preserved

Think of it like: Adding timestamps to a video. Without timestamps, you can't tell if a scene is at the beginning or end. With timestamps, you know exactly when each moment occurs.

From Theory to Revolution: Enter BERT

The Transformer architecture was powerful. But Google researchers asked: "What if we could pre-train a Transformer on ALL of human knowledge, then fine-tune it for specific tasks?"

October 2018: BERT is released (Bidirectional Encoder Representations from Transformers)

The training approach:

1. Read all of Wikipedia (2.5 billion words)
2. Read all published books digitized by Google (11 billion words)
3. Total: 13.5 billion words of human knowledge
4. Training time: 4 days on 64 TPU chips (cost: ~$500,000)

What BERT learned:

Grammar rules (never explicitly taught)
Common sense reasoning (emerges from patterns)
Factual knowledge (learned from context)
Semantic relationships (how words relate)

The magic: Once trained, you can fine-tune BERT for ANY language task with just a tiny bit of specific data.

Example - Sentiment Analysis:

Old approach (train from scratch):

Need: 100,000 labeled examples ("This movie is great" → Positive)
Training time: 2 weeks
Accuracy: 87%

BERT approach (fine-tune):

Need: 1,000 labeled examples (100× less data!)
Training time: 30 minutes (670× faster!)

Accuracy: 94% (better results!)

Why? BERT already understands language deeply. You're just teaching it "apply your knowledge to movie reviews."

It's like hiring a PhD linguist vs. training someone from kindergarten. The PhD just needs brief orientation on your specific task.

The Real-World Impact: What BERT Enabled

Google Search (May 2019):

Before BERT: "2019 brazil traveler to usa need a visa"
Google focused on "brazil" and "usa," ignored "to"
Showed: Do Americans need visa for Brazil? (WRONG!)
After BERT: Understands "traveler TO usa"
Shows: Do Brazilians need visa for USA? (CORRECT!)

Impact: 10% of ALL Google searches (billions daily) got more relevant results.

Healthcare (2020):

Task: Extract medical information from doctor's notes
Challenge: Doctors write in shorthand, ambiguous terminology
Old system: 73% accuracy (27% of diagnoses extracted incorrectly!)
BERT fine-tuned: 94% accuracy
Hospitals saved: Estimated $8 billion annually in manual review costs

Customer Service (2021):

Company: Major airline
Task: Automatically categorize customer complaints
Old system: Keyword matching, 68% accuracy
BERT fine-tuned on 2,000 labeled tickets: 91% accuracy
Result: Resolved complaints 40% faster, customer satisfaction up 23%

Legal Document Analysis (2022):

Law firm processing merger documents (10,000 pages)

Task: Find all liability clauses
Manual review: 80 hours, $24,000 in billable hours
BERT fine-tuned system: 2 hours, $600 compute cost
Accuracy: Better than junior associates

The Architecture: Layers of Understanding

Here's what happens when you process "The cat sat on the mat" through BERT:

Layer 1 (Low-level):

Recognizes word boundaries
Identifies parts of speech (noun, verb, preposition)
"cat" = noun, "sat" = verb (past tense)

Layer 6 (Mid-level):

Understands "cat" and "mat" are both physical objects
Recognizes "sat on" indicates spatial relationship
Knows "the" specifies particular cat and mat

Layer 12 (High-level):

Understands full semantic meaning
Can answer: "What was sitting?" → cat
Can answer: "Where was the cat?" → on the mat
Can infer: "Was the cat standing?" → no, it was sitting

Each layer builds on the previous, creating hierarchical understanding—just like your brain processes language.

The Fine-Tuning Miracle: Teaching an Expert New Tricks

Analogy: You're a skilled chef (BERT pre-trained on general language).

Someone asks: "Can you make sushi?" (Fine-tune for Japanese cuisine task)

You don't need to:

- Relearn how to use a knife (basic language understanding)
- Relearn what "salt" means (vocabulary)
- Relearn cooking techniques (grammar, syntax)

You only need to:

- Learn sushi-specific techniques (task-specific patterns)
- Understand sushi ingredients (domain vocabulary)
- Practice rolling rice (task execution)

Time needed: Days, not years.

Same with BERT:

- Base knowledge: Already has deep language understanding
- Fine-tuning: Just teaches task-specific patterns
- Result: Expert-level performance in hours

Why this matters financially:

Traditional approach: Train model from scratch

- Cost: $500,000 (cloud computing)
- Time: 2 months
- Data needed: 1 million examples
- Risk: Might not work

BERT fine-tuning:

- Cost: $50 (cloud computing)
- Time: 2 hours
- Data needed: 1,000 examples
- Risk: Almost guaranteed to work (proven architecture)

10,000× more efficient.

The Attention Mechanism: A Deeper Look

When BERT processes "The animal didn't cross the street because it was too tired," here's what happens at the word "it":

Computing attention scores:

```
"it" is looking at all other words:

Similarity with "The": 0.02 (very low - grammatical word)
Similarity with "animal": 0.89 (VERY HIGH - semantic match!)
Similarity with "didn't": 0.05 (low - just a negation)
Similarity with "cross": 0.12 (moderate - related action)
Similarity with "street": 0.31 (high but not as high as animal)
Similarity with "because": 0.03 (low - conjunction)
Similarity with "was": 0.04 (low - auxiliary verb)
Similarity with "too": 0.08 (moderate - modifier)
Similarity with "tired": 0.67 (high - describes state)

ATTENTION: Focuses 89% on "animal", 31% on "street", 67% on "tired"
UNDERSTANDING: "it" = animal (that was tired)
```

How does it compute these scores?

Think of it like dating app compatibility matching:

1. Query: "What is 'it' referring to?" (Create a question vector)
2. Keys: Each word presents its "profile" (What am I?)
3. Values: Each word's actual meaning (Here's my full information)
4. Matching process:
    Compare "it" with each word's profile
    High match with "animal" (both are entities/things)
    Low match with "because" (it's just a conjunction)
5. Weighted retrieval:
    Take information from words proportional to match score
    89% of "animal's" information
    31% of "street's" information
    Combine to form understanding

The genius: This happens in parallel for EVERY word simultaneously.

The Position Problem Solved: Sine and Cosine to the Rescue

Without positional information:

> "dog bites man" = "man bites dog" (very different meanings!)
> "before surgery" = "after surgery" (dangerous in medical context!)

Positional Encoding solution: Add a unique mathematical signature to each position.

Why sine and cosine waves?

Imagine a clock face:

> Position 1: 12 o'clock
> Position 2: 1 o'clock
> Position 3: 2 o'clock
> ...

But we need to represent 512 positions (not just 12). And we need smooth, continuous values.

The encoding:

> Uses multiple sine and cosine waves at different frequencies
> Position 1 gets: [sin(1/10000), cos(1/10000), sin(1/100), cos(1/100), ...]
> Position 2 gets: [sin(2/10000), cos(2/10000), sin(2/100), cos(2/100), ...]
> Position 512 gets: [sin(512/10000), cos(512/10000), ...]

Why this works:

> Every position has a UNIQUE signature
> Nearby positions have SIMILAR signatures (position 5 and 6 are close)
> Far positions have DIFFERENT signatures (position 1 and 100 are far)
> Model can learn "this word is 3 positions after that word"

Analogy: Like GPS coordinates

> Every location on Earth has unique lat/long
> Nearby places have similar coordinates
> You can calculate distance between any two points
> Same with positional encoding!

What You'll Actually Do in the Session

This isn't theory for theory's sake. You'll build and experiment:

Part 1: Implement Self-Attention from Scratch

- Write the Query-Key-Value mechanism
- Compute attention scores manually
- Understand exactly how words "attend" to each other
- Visualize attention patterns

Part 2: Add Positional Encoding

- Implement sine/cosine position embeddings
- See how it changes the model's understanding
- Test: Can the model tell "cat chases dog" from "dog chases cat"?

Part 3: Build a Mini-Transformer

- Stack attention layers
- Add feed-forward networks
- Create a complete (tiny) Transformer
- Train on a simple task

Part 4: Fine-Tune BERT

- Load pre-trained BERT
- Freeze most layers
- Train only the top layer for your task
- Compare: BERT vs. training from scratch
- Measure: Speed, accuracy, data efficiency

Real dataset: Movie reviews (positive/negative sentiment)

- Train from scratch: Needs 50,000 examples
- Fine-tune BERT: Needs 1,000 examples
- You'll see the 50× data efficiency in action!

The Implications: Why This Matters for Your Career

2018: BERT released 2019: GPT-2 (same Transformer architecture) 2020: GPT-3 (175 billion parameters, same core idea) 2022: ChatGPT (180 million users in 2 months) 2023: GPT-4, Google Bard, Claude (all Transformers)

Every major AI breakthrough of the past 5 years uses Transformers.

Current market:

> AI Engineer with Transformer expertise:
> $180k–
> $180k–300k salary
> Companies hiring: Google, Meta, OpenAI, Anthropic, Microsoft, Amazon, NVIDIA, + thousands of startups
> Job postings requiring "Transformer" or "BERT": 47,000+ (LinkedIn, 2024)

Skills you'll have after this session:

1. Understand self-attention (foundation of all modern NLP)
2. Implement positional encoding (critical detail)
3. Build mini-Transformers (architectural knowledge)
4. Fine-tune BERT (practical, deployable skill)
5. Optimize for production (efficiency matters)

Real project you could do next week:

> Take any text classification problem at your company
> Download pre-trained BERT
> Fine-tune on your specific data (500-1000 examples)
> Deploy a model that outperforms legacy systems
> Estimated time: 2-4 hours
> Estimated cost: $5-20 in cloud compute

That's the power of transfer learning with BERT.

One More Thing: The Philosophical Shift

Before Transformers, AI couldn't really "understand" language. It could:

> Match patterns
> Memorize examples
> Apply statistical correlations

But it couldn't truly comprehend context, nuance, ambiguity.

The Transformer revolution isn't just technical—it's philosophical.

For the first time, we have AI that:

> Processes language bidirectionally (understands context from both directions)
> Captures long-range dependencies (remembers word 1 when processing word 100)
> Learns hierarchical representations (simple patterns → complex meaning)
> Transfers knowledge across tasks (learns once, applies everywhere)

This is as close as we've come to machine "understanding."

Not perfect. Not human-level (yet). But fundamentally different from everything before.

When you fine-tune BERT on medical notes and it achieves 94% accuracy, it's not just matching keywords—it's actually understanding doctor's shorthand, medical terminology, and clinical context.

That's unprecedented in the history of AI.

Ready to Build?

In the next session, you'll move from "Transformers are magic" to "I understand exactly how they work and can build them myself."

You'll:

> Write the self-attention mechanism (the core innovation)
> Implement positional encoding (the crucial detail)
> Assemble a mini-Transformer (understanding the architecture)
> Fine-tune BERT (gaining practical skills)

By the end, you won't just know how ChatGPT works—you'll know how to build similar systems yourself.

From user to builder. From consumer to creator.

See you in the session. Bring curiosity, a laptop, and readiness to be amazed by what you'll build.

Pre-Session Reflection:

Think about a task you do that involves understanding language:

> Reading customer emails and categorizing them?
> Extracting information from documents?

Analyzing sentiment in social media?
Translating between languages?

How would you teach a computer to do this? What would it need to know?

After this session, you'll have the answer. And you'll have the skills to implement it.

Transformers & BERT Fine-Tuning - Lecture Notes

PPT File: Click Here

Program: Vishlesan i-Hub IIT Patna x Masai School — AIM (AI & Machine Learning)
Prerequisites: Deep learning basics, RNNs/LSTMs, PyTorch, attention mechanism basics

---

Session Overview

This session covers the Transformer architecture and BERT, the foundation of modern NLP. You'll learn self-attention mechanisms, positional encoding, build a complete mini-transformer from scratch, and master practical BERT fine-tuning for downstream tasks.

Learning Objectives

By the end of this session, you will be able to:

1. Understand self-attention mechanism and its advantages over recurrent approaches
2. Implement positional encoding using sine/cosine functions
3. Build a complete mini-Transformer from scratch with all components
4. Explain BERT architecture and pre-training objectives (MLM, NSP)
5. Fine-tune BERT for classification tasks with hands-on examples
6. Optimize fine-tuning with layer freezing and learning rate strategies
7. Deploy BERT models efficiently for production use

Part 1: The Limitations of RNNs and the Need for Transformers (15 minutes)

[Content remains the same as original...]

Part 2: Self-Attention Mechanism (30 minutes)

[Content remains the same as original through the concrete example and PyTorch implementation...]

Part 3: Positional Encoding (20 minutes)

[Content remains the same as original...]

Part 4: Building a Mini-Transformer from Scratch (30 minutes)

Why Build a Mini-Transformer?

Learning objective: Understand transformers by building one from first principles.

What we'll build:

- Complete mini-transformer encoder
- Handles sequence classification
- ~500 lines of working code
- Demonstrates all core concepts in practice

Architecture overview:

```
Input tokens
    ↓
Token Embedding (vocab → d_model)
    ↓
Positional Encoding (add position info)
    ↓
Transformer Block 1 (attention + FFN)
    ↓
Transformer Block 2
    ↓
... (N blocks)
    ↓
[CLS] token extraction
    ↓
Classification head
    ↓
Output logits
```

Step 1: Input Embeddings

```python
import torch
import torch.nn as nn
import math

class InputEmbedding(nn.Module):
    """
    Convert token IDs to dense vectors

    Example:
        Token "cat" (ID: 524) → [0.12, -0.45, 0.89, ..., 0.34] (d_model dims)
```

```
    """
    def __init__(self, vocab_size, d_model):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.d_model = d_model

    def forward(self, x):
        """
        Args:
            x: [batch, seq_len] token IDs
        Returns:
            embeddings: [batch, seq_len, d_model]
        """
        # Scale embeddings by sqrt(d_model) as in original paper
        # This helps gradient flow in later layers
        return self.embedding(x) * math.sqrt(self.d_model)

# Example usage
vocab_size = 10000
d_model = 256

embedding_layer = InputEmbedding(vocab_size, d_model)

# Input: batch of 2 sequences, each 10 tokens long
token_ids = torch.randint(0, vocab_size, (2, 10))
embeddings = embedding_layer(token_ids)

print(f"Input shape: {token_ids.shape}")      # [2, 10]
print(f"Output shape: {embeddings.shape}")    # [2, 10, 256]
```

Step 2: Positional Encoding

```
class PositionalEncoding(nn.Module):
    """
    Add position information to embeddings

    Why needed: Self-attention is permutation-invariant
    Solution: Inject position using sine/cosine functions
    """
    def __init__(self, d_model, max_len=5000, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

        # Create positional encoding matrix
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1).float()

        div_term = torch.exp(
            torch.arange(0, d_model, 2).float() *
            (-math.log(10000.0) / d_model)
        )
```

```
        pe[:, 0::2] = torch.sin(position * div_term)  # Even indices
        pe[:, 1::2] = torch.cos(position * div_term)  # Odd indices

        # Register as buffer (not a trainable parameter)
        self.register_buffer('pe', pe.unsqueeze(0))

    def forward(self, x):
        """
        Args:
            x: [batch, seq_len, d_model] embeddings
        Returns:
            x + positional encoding
        """
        x = x + self.pe[:, :x.size(1)]
        return self.dropout(x)

# Visualization of positional encoding pattern
import matplotlib.pyplot as plt

pe_layer = PositionalEncoding(d_model=128, max_len=50)
pe_matrix = pe_layer.pe.squeeze(0).numpy()  # [50, 128]

plt.figure(figsize=(15, 5))
plt.pcolormesh(pe_matrix.T, cmap='RdBu')
plt.xlabel('Position')
plt.ylabel('Dimension')
plt.colorbar()
plt.title('Positional Encoding Pattern\n(Notice smooth transitions)')
plt.show()
```

Key insight from visualization:

Low dimensions (bottom): Change slowly across positions
High dimensions (top): Change rapidly across positions
This multi-scale pattern helps model learn both local and global relationships

Step 3: Multi-Head Attention Block

```
class MultiHeadAttentionBlock(nn.Module):
    """
    Multi-head self-attention with residual connection

    Input: [batch, seq_len, d_model]
    Output: [batch, seq_len, d_model] (same shape!)
    """
    def __init__(self, d_model, num_heads, dropout=0.1):
        super().__init__()

        # Use PyTorch's built-in multi-head attention
        self.attention = nn.MultiheadAttention(
            embed_dim=d_model,
            num_heads=num_heads,
```

```
            dropout=dropout,
            batch_first=True  # Important: input is [batch, seq, features]
        )

        self.norm = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        """
        Args:
            x: [batch, seq_len, d_model]
            mask: [batch, seq_len] optional padding mask
        Returns:
            output: [batch, seq_len, d_model]
        """
        # Self-attention: query=key=value=x
        attended, attention_weights = self.attention(
            x, x, x,
            key_padding_mask=mask
        )

        # Residual connection + layer norm
        x = self.norm(x + self.dropout(attended))

        return x
```

Step 4: Feed-Forward Network Block

```
class FeedForwardBlock(nn.Module):
    """
    Position-wise feed-forward network

    Architecture: Linear → ReLU → Dropout → Linear
    Typical expansion: d_model → 4*d_model → d_model
    """
    def __init__(self, d_model, d_ff, dropout=0.1):
        super().__init__()

        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff),       # Expand
            nn.ReLU(),                       # Non-linearity
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model)         # Project back
        )

        self.norm = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        """
        Args:
            x: [batch, seq_len, d_model]
```

```
        Returns:
            output: [batch, seq_len, d_model]
        """
        # Feed-forward + residual + layer norm
        ff_out = self.ffn(x)
        x = self.norm(x + self.dropout(ff_out))

        return x
```

Step 5: Complete Mini-Transformer Block

```
class MiniTransformerBlock(nn.Module):
    """
    Single transformer encoder block

    Components:
    1. Multi-head self-attention
    2. Add & Norm
    3. Feed-forward network
    4. Add & Norm
    """
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()

        self.attention_block = MultiHeadAttentionBlock(
            d_model, num_heads, dropout
        )
        self.ffn_block = FeedForwardBlock(d_model, d_ff, dropout)

    def forward(self, x, mask=None):
        """
        Args:
            x: [batch, seq_len, d_model]
            mask: [batch, seq_len] padding mask
        Returns:
            output: [batch, seq_len, d_model]
        """
        # Attention sub-block
        x = self.attention_block(x, mask)

        # Feed-forward sub-block
        x = self.ffn_block(x)

        return x

# Test the transformer block
block = MiniTransformerBlock(d_model=256, num_heads=8, d_ff=1024)

# Input: batch=2, seq_len=10, d_model=256
x = torch.randn(2, 10, 256)
output = block(x)
```

```
print(f"Input shape: {x.shape}")       # [2, 10, 256]
print(f"Output shape: {output.shape}") # [2, 10, 256] - same!
```

Step 6: Complete Mini-Transformer Model

```
class MiniTransformer(nn.Module):
    """
    Complete mini-transformer for sequence classification

    Architecture:
    Input tokens → Embeddings → Positional Encoding →
    N Transformer Blocks → [CLS] pooling → Classifier → Logits
    """
    def __init__(
        self,
        vocab_size,           # Size of vocabulary
        num_classes,          # Number of output classes
        d_model=256,          # Model dimension
        num_heads=8,          # Number of attention heads
        num_layers=4,         # Number of transformer blocks
        d_ff=1024,            # Feed-forward dimension
        max_len=512,          # Maximum sequence length
        dropout=0.1           # Dropout probability
    ):
        super().__init__()

        # Input processing
        self.embedding = InputEmbedding(vocab_size, d_model)
        self.pos_encoding = PositionalEncoding(d_model, max_len, dropout)

        # Stack of transformer blocks
        self.transformer_blocks = nn.ModuleList([
            MiniTransformerBlock(d_model, num_heads, d_ff, dropout)
            for _ in range(num_layers)
        ])

        # Output classification head
        self.classifier = nn.Sequential(
            nn.Dropout(dropout),
            nn.Linear(d_model, num_classes)
        )

    def forward(self, x, mask=None):
        """
        Args:
            x: [batch, seq_len] token IDs
            mask: [batch, seq_len] padding mask (1 = valid, 0 = padding)

        Returns:
            logits: [batch, num_classes]
        """
        # 1. Embed tokens
```

```
        x = self.embedding(x)           # [batch, seq_len, d_model]

        # 2. Add positional encoding
        x = self.pos_encoding(x)        # [batch, seq_len, d_model]

        # 3. Pass through transformer blocks
        for block in self.transformer_blocks:
            x = block(x, mask)          # [batch, seq_len, d_model]

        # 4. Pool: use [CLS] token (first token) for classification
        cls_output = x[:, 0, :]         # [batch, d_model]

        # 5. Classify
        logits = self.classifier(cls_output)  # [batch, num_classes]

        return logits

# Initialize mini-transformer
model = MiniTransformer(
    vocab_size=10000,
    num_classes=2,      # Binary classification (positive/negative)
    d_model=256,
    num_heads=8,
    num_layers=4,
    d_ff=1024,
    dropout=0.1
)

print(f"Total parameters: {sum(p.numel() for p in model.parameters()):,}")
# ~5-6 million parameters
```

Step 7: Training the Mini-Transformer

```
import torch.optim as optim
from torch.utils.data import DataLoader

# Training configuration
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)

optimizer = optim.Adam(model.parameters(), lr=1e-4)
criterion = nn.CrossEntropyLoss()

# Example training loop
def train_epoch(model, train_loader, optimizer, criterion):
    model.train()
    total_loss = 0
    correct = 0
    total = 0

    for batch in train_loader:
        # Move to device
```

```
            input_ids = batch['input_ids'].to(device)
            labels = batch['labels'].to(device)
            mask = batch['mask'].to(device) if 'mask' in batch else None

            # Forward pass
            logits = model(input_ids, mask)
            loss = criterion(logits, labels)

            # Backward pass
            optimizer.zero_grad()
            loss.backward()

            # Gradient clipping (prevent exploding gradients)
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

            optimizer.step()

            # Track metrics
            total_loss += loss.item()
            predictions = logits.argmax(dim=1)
            correct += (predictions == labels).sum().item()
            total += labels.size(0)

    avg_loss = total_loss / len(train_loader)
    accuracy = correct / total

    return avg_loss, accuracy

# Training loop
num_epochs = 10

for epoch in range(num_epochs):
    train_loss, train_acc = train_epoch(model, train_loader, optimizer, criterion)
    print(f"Epoch {epoch+1}/{num_epochs}")
    print(f"  Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f}")
```

Mini-Transformer vs BERT: Comparison

Aspect	Mini-Transformer	BERT-Base	BERT-Large
Architecture	Encoder-only	Encoder-only	Encoder-only
Layers	4	12	24
Hidden size (d_model)	256	768	1024

Attention heads	8	12	16
FFN size (d_ff)	1024	3072	4096
Total parameters	~5M	110M	340M
Training	From scratch	Pre-trained	Pre-trained
Training data	Your dataset	3.3B words	3.3B words
Training time	Hours	Days (~$500k)	Weeks (~$1.5M)
Use case	Learning, prototyping	Production	Max accuracy

When to use Mini-Transformer:

✅ Learning transformer architecture
✅ Prototyping new ideas quickly
✅ Small datasets (< 10k examples)
✅ Domain-specific tasks with limited data
✅ Limited compute resources

When to use BERT:

✅ Production applications
✅ Need state-of-the-art accuracy
✅ Transfer learning from massive pre-training
✅ Have compute for fine-tuning
✅ Standard NLP tasks (classification, NER, QA)

Key Insights from Building Mini-Transformer

1. Modularity:

   Each component is independent
   Can swap/upgrade individual parts
   Easy to experiment with variants

2. Residual Connections:

Enable training deep networks (4+ layers)
Gradient flows directly to earlier layers
Model can learn identity mapping if needed

3. Layer Normalization:

Stabilizes training
Works with variable sequence lengths
Applied after each sub-layer

4. Positional Encoding:

Absolutely critical for performance
Without it, model is permutation-invariant
Can't distinguish "dog bites man" from "man bites dog"

5. Computational Cost:

Self-attention: $O(n^2)$ where n = sequence length
Feed-forward: O(n) but with large constant ($d_{ff} = 4 \times d_{model}$)
Trade-off: Parallelization vs memory

Part 5: BERT Architecture and Pre-Training (20 minutes)

BERT Overview

BERT: Bidirectional Encoder Representations from Transformers

Key innovation: Pre-train on massive unlabeled data, then fine-tune for specific tasks

Architecture:

Stack of Transformer encoder blocks (like our mini-transformer!)
No decoder (encoder-only architecture)
Bidirectional context (sees both left and right simultaneously)

Model sizes:

```
BERT-Base:
```

```
- 12 transformer encoder layers
- 768 hidden dimensions (d_model)
- 12 attention heads
- 110M parameters

BERT-Large:
- 24 transformer encoder layers
- 1024 hidden dimensions
- 16 attention heads
- 340M parameters
```

[Rest of BERT pre-training content remains the same...]

Part 6: Fine-Tuning BERT - Complete Hands-On Guide (30 minutes)

Fine-Tuning Strategy Overview

Concept: Use pre-trained BERT as a feature extractor, add task-specific head

Why fine-tuning works:

1. BERT already understands language (from pre-training)
2. We just teach it the specific task (classification, NER, QA)
3. Requires minimal data (1k-10k examples vs 3.3B words for pre-training)
4. Fast (minutes-hours vs days-weeks)

General approach:

1. Load pre-trained BERT weights
2. Add task-specific output layer
3. Optionally freeze some layers
4. Train on your task-specific data
5. Evaluate and deploy

Hands-On Example 1: Sentiment Classification

Task: Classify movie reviews as positive or negative

Step 1: Install and Import

```
# Install transformers library
!pip install transformers torch

from transformers import (
    BertTokenizer,
    BertForSequenceClassification,
    AdamW,
    get_linear_schedule_with_warmup
)
import torch
from torch.utils.data import Dataset, DataLoader
import pandas as pd
```

Step 2: Load Pre-trained BERT

```
# Load tokenizer (converts text → token IDs)
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# Load model with classification head
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2,           # Binary classification
    output_attentions=False,
    output_hidden_states=False
)

print(f"Model has {model.num_parameters():,} parameters")
# Output: 109,483,778 parameters (110M)
```

Step 3: Prepare Data

```
class SentimentDataset(Dataset):
    """Dataset for sentiment classification"""
    def __init__(self, texts, labels, tokenizer, max_len=128):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts[idx])
        label = self.labels[idx]

        # Tokenize text
        encoding = self.tokenizer.encode_plus(
            text,
            add_special_tokens=True,      # Add [CLS] and [SEP]
            max_length=self.max_len,
            padding='max_length',          # Pad to max_length
```

```
            truncation=True,                # Truncate if too long
            return_attention_mask=True,     # Return attention mask
            return_tensors='pt'             # Return PyTorch tensors
        )

        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten(),
            'labels': torch.tensor(label, dtype=torch.long)
        }

# Example data
texts = [
    "This movie was absolutely fantastic! I loved every minute.",
    "Terrible waste of time. Boring and predictable.",
    "One of the best films I've seen this year!",
    "Disappointing. Expected much more."
]
labels = [1, 0, 1, 0]  # 1 = positive, 0 = negative

# Create dataset and dataloader
dataset = SentimentDataset(texts, labels, tokenizer)
train_loader = DataLoader(dataset, batch_size=2, shuffle=True)

# Inspect a batch
batch = next(iter(train_loader))
print("Input IDs shape:", batch['input_ids'].shape)      # [2, 128]
print("Attention mask shape:", batch['attention_mask'].shape)  # [2, 128]
print("Labels shape:", batch['labels'].shape)            # [2]
```

Step 4: Set Up Optimizer and Scheduler

```
# Optimizer: AdamW (Adam with weight decay)
optimizer = AdamW(
    model.parameters(),
    lr=2e-5,              # Small learning rate for fine-tuning
    eps=1e-8,             # Epsilon for numerical stability
    weight_decay=0.01     # L2 regularization
)

# Learning rate scheduler with warmup
num_epochs = 3
num_training_steps = len(train_loader) * num_epochs
num_warmup_steps = num_training_steps // 10  # 10% warmup

scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=num_warmup_steps,
    num_training_steps=num_training_steps
)
```

Step 5: Training Loop

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)

def train_epoch(model, data_loader, optimizer, scheduler, device):
    model.train()
    total_loss = 0
    correct_predictions = 0

    for batch in data_loader:
        # Move batch to device
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        # Forward pass
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )

        loss = outputs.loss
        logits = outputs.logits

        # Backward pass
        optimizer.zero_grad()
        loss.backward()

        # Gradient clipping (prevent exploding gradients)
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

        # Update weights
        optimizer.step()
        scheduler.step()

        # Track metrics
        total_loss += loss.item()
        predictions = torch.argmax(logits, dim=1)
        correct_predictions += torch.sum(predictions == labels)

    return total_loss / len(data_loader), correct_predictions.double() / len(dataset)

# Training loop
for epoch in range(num_epochs):
    print(f"\nEpoch {epoch + 1}/{num_epochs}")
    print("-" * 50)

    train_loss, train_acc = train_epoch(
        model, train_loader, optimizer, scheduler, device
    )

    print(f"Train Loss: {train_loss:.4f}")
    print(f"Train Accuracy: {train_acc:.4f}")
```

Step 6: Inference

```
def predict(text, model, tokenizer, device):
    """Predict sentiment for a single text"""
    model.eval()

    # Tokenize
    encoding = tokenizer.encode_plus(
        text,
        add_special_tokens=True,
        max_length=128,
        padding='max_length',
        truncation=True,
        return_attention_mask=True,
        return_tensors='pt'
    )

    input_ids = encoding['input_ids'].to(device)
    attention_mask = encoding['attention_mask'].to(device)

    # Predict
    with torch.no_grad():
        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        prediction = torch.argmax(logits, dim=1).item()
        probabilities = torch.softmax(logits, dim=1).squeeze()

    label = "Positive" if prediction == 1 else "Negative"
    confidence = probabilities[prediction].item()

    return label, confidence

# Test predictions
test_texts = [
    "Amazing movie! Highly recommend.",
    "Worst film ever made.",
    "It was okay, nothing special."
]

for text in test_texts:
    label, confidence = predict(text, model, tokenizer, device)
    print(f"Text: {text}")
    print(f"Prediction: {label} (confidence: {confidence:.2%})\n")
```

Hands-On Example 2: Layer Freezing Strategies

Strategy 1: Freeze All BERT, Train Only Classifier

```
# Fastest training, works with small datasets (100-1000 examples)
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2
```

```
)

# Freeze all BERT parameters
for param in model.bert.parameters():
    param.requires_grad = False

# Only classifier head is trainable
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Trainable parameters: {trainable_params:,}")  # ~1,500 (just classifier)
```

Strategy 2: Freeze Bottom Layers, Fine-Tune Top Layers

```
# Good balance: freeze first 8 layers, fine-tune last 4
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2
)

# Freeze embeddings
for param in model.bert.embeddings.parameters():
    param.requires_grad = False

# Freeze first 8 encoder layers
for layer in model.bert.encoder.layer[:8]:
    for param in layer.parameters():
        param.requires_grad = False

# Top 4 layers + classifier are trainable
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Trainable parameters: {trainable_params:,}")  # ~40M
```

Strategy 3: Gradual Unfreezing (Best Performance)

```
def gradually_unfreeze(model, epoch, total_epochs=5):
    """
    Progressively unfreeze layers as training progresses

    Epoch 1: Only classifier
    Epoch 2: Classifier + last 2 layers
    Epoch 3: Classifier + last 4 layers
    Epoch 4: Classifier + last 6 layers
    Epoch 5: All layers (full fine-tuning)
    """
    num_layers = 12  # BERT-Base has 12 layers

    # Start with all frozen
    for param in model.bert.parameters():
        param.requires_grad = False

    # Gradually unfreeze from top
    layers_to_unfreeze = min(2 * epoch, num_layers)
```

```
    for layer in model.bert.encoder.layer[-layers_to_unfreeze:]:
        for param in layer.parameters():
            param.requires_grad = True

    # Classifier always trainable
    for param in model.classifier.parameters():
        param.requires_grad = True

    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    print(f"Epoch {epoch}: {trainable:,} trainable parameters")

# Use in training loop
for epoch in range(1, num_epochs + 1):
    gradually_unfreeze(model, epoch, num_epochs)
    train_loss, train_acc = train_epoch(model, train_loader, optimizer, scheduler,
device)
```

Hyperparameter Guidelines for Fine-Tuning

Hyperparameter	Recommended Value	Rationale
Learning rate	2e-5 to 5e-5	Much lower than training from scratch (1e-3)
Batch size	16 or 32	Limited by GPU memory; use gradient accumulation if needed
Epochs	2-4	BERT fine-tunes quickly; more epochs risk overfitting
Max sequence length	128 or 512	128 for most tasks; 512 for long documents (more memory)
Warmup steps	10% of total steps	Stabilizes training in early phase
Weight decay	0.01	L2 regularization prevents overfitting

Gradient clipping	1.0	Prevents exploding gradients
Dropout	0.1	Already in BERT; don't change

Common Fine-Tuning Pitfalls and Solutions

Problem 1: Catastrophic Forgetting

Symptom: Model performs worse than random after fine-tuning
Cause: Learning rate too high, destroying pre-trained knowledge
Solution: Use 2e-5 learning rate (not 1e-3), freeze bottom layers

Problem 2: Overfitting on Small Datasets

Symptom: Perfect training accuracy, poor validation accuracy
Solution: Freeze more layers, use dropout, data augmentation, early stopping

Problem 3: Out of Memory Errors

Symptom: CUDA out of memory during training
Solutions:
- Reduce batch size (16 → 8 → 4)
- Reduce max_length (512 → 256 → 128)
- Use gradient accumulation
- Use mixed precision training (FP16)

Problem 4: Slow Training

Symptom: Takes too long to train
Solutions:
- Use smaller BERT variant (DistilBERT)
- Freeze more layers
- Reduce max_length
- Use gradient checkpointing

Saving and Loading Fine-Tuned Models

```
# Save fine-tuned model
model.save_pretrained('./fine_tuned_bert_sentiment')
tokenizer.save_pretrained('./fine_tuned_bert_sentiment')

# Load fine-tuned model
```

```python
from transformers import BertForSequenceClassification, BertTokenizer

model = BertForSequenceClassification.from_pretrained('./fine_tuned_bert_sentiment')
tokenizer = BertTokenizer.from_pretrained('./fine_tuned_bert_sentiment')

# Use for inference
text = "This product is amazing!"
encoding = tokenizer(text, return_tensors='pt', padding=True, truncation=True)
outputs = model(**encoding)
prediction = torch.argmax(outputs.logits, dim=1).item()
```

Part 7: Practical Considerations and Optimization (10 minutes)

[Content remains mostly the same, with additions...]

Production Deployment Checklist

Before deploying BERT to production:

1. Model Optimization:
   - ✅ Export to ONNX for 2-3× faster inference
   - ✅ Use DistilBERT (40% smaller, 60% faster, 97% accuracy)
   - ✅ Quantize model (INT8) for mobile/edge deployment
2. Inference Optimization:
   - ✅ Batch similar-length sequences together
   - ✅ Use dynamic padding (not max_length)
   - ✅ Cache tokenizer
   - ✅ Use GPU if available
3. Monitoring:
   - ✅ Track latency (p50, p95, p99)
   - ✅ Monitor accuracy drift over time
   - ✅ Log prediction confidence scores
4. Error Handling:
   - ✅ Handle empty inputs gracefully
   - ✅ Set maximum input length limits
   - ✅ Provide fallback for low-confidence predictions

Key Takeaways

Core Concepts Mastered

1. Self-Attention: Allows parallel processing, $O(n^2)$ complexity but worth it
2. Positional Encoding: Critical for position-aware understanding
3. Mini-Transformer: Built complete encoder from scratch (~500 lines)
4. BERT Architecture: 12/24 layer transformer with MLM + NSP pre-training
5. Fine-Tuning: Transfer learning with 2-4 epochs, 2e-5 learning rate
6. Layer Freezing: Strategic freezing speeds training and prevents overfitting

Hands-On Skills Acquired

✅ Built mini-transformer from first principles
✅ Loaded and fine-tuned pre-trained BERT
✅ Implemented layer freezing strategies
✅ Set up proper optimizers and schedulers
✅ Saved and deployed fine-tuned models
✅ Optimized for production use

When to Use What

Task	Model Choice	Training Approach
Learning transformers	Mini-Transformer	Train from scratch
Production classification	BERT-Base	Fine-tune 2-4 epochs
Maximum accuracy	BERT-Large	Fine-tune all layers
Mobile/Edge deployment	DistilBERT	Fine-tune + quantize
Very small dataset (<1k)	BERT frozen	Train only classifier

Pre-read: The Architecture Inside ChatGPT

The Billion-Dollar Question

In March 2023, OpenAI's GPT-4 stunned the world by passing the bar exam, diagnosing medical conditions, and writing production-quality code. Behind the sleek chat interface lay 1.7 trillion parameters—but what does that number actually mean? And why did OpenAI spend an estimated

$$100 million to train it, when GPT-3 cost "only"$$

$$100 *million to train it, when GPT-3 cost* "*only*"$$ 5 million?

Inside the Black Box

Meet Sarah, a machine learning engineer at a startup trying to build their own language model. Her CEO just asked: "ChatGPT has 175 billion parameters. Should we build a model with 175 billion parameters too, or would 10 billion be enough? And should we make it deeper or wider?"

Sarah knows the answer isn't simple. When she opens the architecture diagram of GPT-2 (a smaller, publicly documented model), she sees something surprising: 96 layers stacked on top of each other, each containing 12 attention heads looking at the text from different angles. The model is 768 units wide at each layer.

But here's the puzzle: Why 96 layers instead of 48? Why not make each layer twice as wide instead? And why exactly 12 attention heads—not 6, not 24?

The Scaling Mystery

When researchers at OpenAI trained models of different sizes, they discovered something remarkable: Model performance follows mathematical laws. Double the compute budget, and you get a predictable improvement in performance. But there's a catch—the improvement follows a power law, not a linear relationship.

This meant they could predict that GPT-4 would achieve certain capabilities before spending $100 million to train it. They could forecast that increasing the model from 125M parameters to 1.5B parameters would reduce loss by approximately X amount—and they were right.

The Interpretability Challenge

But here's what keeps researchers up at night: We don't really know what these layers are learning.

When Sarah uses interpretability probes—special tools that peek inside GPT-2's layers—she sees something fascinating:

> Early layers (1-20) seem to learn basic grammar and syntax
> Middle layers (21-60) capture semantic relationships and facts
> Late layers (61-96) handle reasoning and task completion

But here's the mystery: In layer 47, neuron 892 activates strongly for... mentions of bridges and connections. Not just physical bridges—also metaphorical ones like "bridging the gap" or "connecting ideas." How did it learn this abstract concept? Nobody explicitly programmed it.

The Architecture Trade-offs

Sarah discovers a fundamental tension:

> Deeper models (more layers) can learn more complex reasoning chains but are slower to train and run
> Wider models (more units per layer) can represent more information but require exponentially more memory
> More attention heads let the model look at text from multiple perspectives but increase computational cost

GPT-2 Small has 12 layers. GPT-2 Large has 36 layers. GPT-3 has 96 layers. Each architecture choice represents millions of dollars in compute costs.

The Power Law Promise

The breakthrough came from scaling laws discovered in 2020: If you know how a 100M parameter model performs, you can predict with surprising accuracy how a 10B parameter model will perform—before training it.

This means companies can make informed decisions: "If we spend $10 million on compute, we'll get this much performance improvement." Or conversely: "To reach human-level performance on this task, we'll need approximately this much compute."

What You're About to Learn

In this session, we'll open the hood on these language models and understand:

> Why do we stack so many layers? What happens at each depth?
> What are attention heads really doing? Can we visualize what they learn?
> How do scaling laws work? Can we predict performance before training?
> What do interpretability probes reveal? Can we understand what GPT-2 learned in layer 47?

By the end, you'll understand not just how to use these models, but how they actually work inside—and why building the next GPT requires both mathematical insight and massive computational resources.

Curiosity Question

If adding one more layer to GPT-2 costs $50,000 in training compute but improves performance by only 0.5%, how do researchers decide when to stop making models deeper? What mathematical framework guides these multi-million dollar decisions?

Lecture Notes: LLM Internals & Scaling

PPT File: Click Here

Session Overview

This session explores the internal architecture of large language models, focusing on the fundamental design choices that determine their capabilities and computational costs. We'll examine GPT-2's architecture in detail, understand attention mechanisms, derive scaling laws, and use interpretability techniques to see what these models actually learn.

Learning Objectives

By the end of this session, you will be able to:

1. Explain the depth vs. width trade-offs in transformer architectures
2. Describe how multi-head attention enables parallel information processing
3. Apply scaling laws to predict model performance before training
4. Use interpretability probes to understand what different layers learn
5. Make informed architecture decisions for resource-constrained scenarios

---

1. Transformer Architecture Fundamentals

1.1 The Stack: Depth in Transformers

When we say GPT-2 has "48 layers," what does that actually mean?

Layer Structure:

```
Input Embeddings
    ↓
[Layer 1: Attention + Feed-Forward]
    ↓
[Layer 2: Attention + Feed-Forward]
    ↓
    ...
    ↓
[Layer 48: Attention + Feed-Forward]
    ↓
Output Predictions
```

Each layer performs two main operations:

1. Multi-head self-attention: Looks at relationships between all tokens
2. Feed-forward network: Processes each token independently

Why Stack Layers?

Think of layers as stages of understanding:

Shallow layers (1-12): Learn surface patterns—syntax, basic grammar, word associations
Middle layers (13-36): Capture semantic relationships, factual knowledge, entity recognition
Deep layers (37-48): Perform reasoning, inference, task-specific computations

Mathematical Flow:

```
h₀ = Embedding(tokens)
h₁ = Layer₁(h₀) = FFN(Attention(h₀))
h₂ = Layer₂(h₁) = FFN(Attention(h₁))
...
h₄₈ = Layer₄₈(h₄₇)
Output = Softmax(h₄₈ · W_output)
```

Each layer transforms the representation, adding more abstract understanding.

1.2 Width: The Hidden Dimension

GPT-2 comes in multiple sizes:

Small: d_model = 768
Medium: d_model = 1024
Large: d_model = 1280
XL: d_model = 1600

What does d_model mean?

Every token is represented as a vector of length d_model. This is the "width" of the network—how many features can represent each token at each layer.

Width vs. Depth Trade-off:

For a fixed parameter budget:

Wider, shallower: Can represent more features simultaneously, but limited reasoning depth
Deeper, narrower: More sequential reasoning steps, but fewer features per step

Example:

Model A: 24 layers × 1536 width = ~37M parameters per layer
Model B: 48 layers × 768 width = ~37M parameters per layer

Both have similar total parameters, but very different capabilities. Research shows deeper models generally perform better for complex reasoning tasks.

---

2. Multi-Head Attention: Looking from Multiple Angles

2.1 What Are Attention Heads?

Multi-head attention is like having multiple "experts" looking at the same text from different perspectives.

GPT-2 Configuration:

Small: 12 heads
Medium: 16 heads
Large: 20 heads
XL: 25 heads

Why Multiple Heads?

Each head can specialize in different patterns:

Head 1: Might focus on syntactic dependencies (subject-verb agreement)
Head 2: Might track coreference (pronouns → entities)
Head 3: Might identify semantic relationships
Head 4: Might handle positional patterns

Mathematical Formulation:

For input X with dimension d_model = 768 and 12 heads:

```
1. Split d_model across heads:
   d_head = d_model / num_heads = 768 / 12 = 64

2. Each head computes attention independently:
   head_i = Attention(X·W_Q^i, X·W_K^i, X·W_V^i)
   where W_Q^i, W_K^i, W_V^i project to d_head dimensions

3. Concatenate all heads:
```

```
   MultiHead(X) = Concat(head₁, head₂, ..., head₁₂) · W_O

4. Result has original dimension d_model = 768
```

Key Insight: All heads process in parallel, enabling the model to attend to multiple relationships simultaneously.

2.2 Attention Patterns in Practice

When we visualize attention in trained models, we discover heads specialize:

Positional Heads: Attend to adjacent tokens (capturing local context)

```
[The] [cat] [sat] [on] [the] [mat]
  ↓     ↓     ↓     ↓     ↓     ↓
Strong attention to immediate neighbors
```

Syntactic Heads: Connect words by grammatical role

```
[The] [cat] [that] [I] [saw] [ran]
       ↓                ↓      ↓
   Subject ←←←←←←←←← Verb
```

Semantic Heads: Link related concepts

```
[The] [doctor] [prescribed] [medicine]
       ↓                          ↓
    Entity ←←←←←←←←←←← Related concept
```

---

3. Scaling Laws: The Mathematics of Bigger Models

3.1 The Empirical Discovery

In 2020, researchers at OpenAI discovered that model performance follows predictable mathematical laws based on three factors:

1. N: Number of parameters

2. D: Size of dataset
3. C: Amount of compute (FLOPs)

The Power Law:

```
Loss ∝ (N/N₀)^(-α_N) · (D/D₀)^(-α_D) · (C/C₀)^(-α_C)

Where:
- α_N ≈ 0.076 (parameters exponent)
- α_D ≈ 0.095 (data exponent)
- α_C ≈ 0.050 (compute exponent)
```

What This Means:

If you double the parameters (N), loss improves by approximately:

```
Improvement = 2^(-0.076) ≈ 0.95 (5% reduction in loss)
```

If you 10x the parameters:

```
Improvement = 10^(-0.076) ≈ 0.83 (17% reduction in loss)
```

3.2 Optimal Allocation

For a fixed compute budget C, how should you allocate resources?

Chinchilla Scaling Laws (2022 Update):

Optimal ratio: Parameters : Training Tokens ≈ 1 : 20

Example:

1B parameter model → Train on 20B tokens
10B parameter model → Train on 200B tokens
100B parameter model → Train on 2T tokens

Why This Matters:

Many models were undertrained! GPT-3 (175B parameters) was trained on 300B tokens—far below the optimal 3.5T tokens.

Chinchilla (70B parameters, 1.4T tokens) outperformed GPT-3 (175B parameters, 300B tokens) because it followed the optimal ratio.

3.3 Cost Predictions

Using scaling laws, we can predict training costs:

Compute Required:

```
C ≈ 6 · N · D

Where:
N = number of parameters
D = number of training tokens
6 = approximate FLOPs per parameter per token
```

Example: GPT-3

```
N = 175B parameters
D = 300B tokens
C = 6 · 175B · 300B = 3.15 × 10²³ FLOPs

At $1 per 10¹⁵ FLOPs (typical GPU pricing):
Cost ≈ $315,000 × efficiency factor ≈ $5-10M actual cost
```

---

4. Interpretability: Understanding What Models Learn

4.1 Layer-wise Specialization

Using probing techniques, we can see what information each layer captures:

Methodology: Train a simple classifier (probe) on layer activations to predict linguistic properties:

```python
# Pseudo-code
for layer in [1, 2, ..., 48]:
    activations = model.get_layer_activations(layer, text)
    probe = train_classifier(activations, target_labels)
    accuracy[layer] = probe.evaluate()
```

Findings in GPT-2:

Layers 1-8 (Surface):

Part-of-speech tagging: 95% accuracy
Sentence boundary detection: 98% accuracy
Simple word associations: 90% accuracy

Layers 9-24 (Syntactic):

Dependency parsing: 85% accuracy
Coreference resolution: 78% accuracy
Named entity recognition: 92% accuracy

Layers 25-40 (Semantic):

Sentiment analysis: 88% accuracy
Factual recall: 82% accuracy
Semantic role labeling: 80% accuracy

Layers 41-48 (Task-specific):

Question answering: Peak performance
Text completion: Highest coherence
Reasoning tasks: Best accuracy

4.2 Attention Head Specialization

Individual attention heads develop specific behaviors:

Positional Heads (Layers 1-10):

Attend primarily to previous token
Enable local context understanding
Essential for grammar

Syntactic Heads (Layers 10-25):

Connect subjects to verbs
Track noun phrase boundaries
Handle long-distance dependencies

Semantic Heads (Layers 25-40):

Link entities to descriptions
Connect questions to relevant context
Track topical coherence

Mixed Heads (Layers 40-48):

Combine multiple patterns
Task-dependent behavior
Hardest to interpret

4.3 Neuron-Level Interpretability

Some neurons in FFN layers develop interpretable functions:

Example Findings:

Neuron 47:892 (Layer 47): Activates for bridge/connection concepts
Neuron 23:145 (Layer 23): Tracks sentiment polarity
Neuron 35:678 (Layer 35): Responds to numeric patterns

Technique:

```
# Find maximally activating examples
for text in large_corpus:
    activation = model.forward(text)[layer][neuron]
    if activation > threshold:
        save(text, activation)

# Analyze patterns in top activations
```

---

5. Practical Architecture Decisions

5.1 Resource-Constrained Scenarios

Scenario: You have budget for 1B parameters. How to configure?

Option A: Shallow and Wide

12 layers × 2048 width × 16 heads
Fast inference
Good for: Pattern matching, retrieval

Option B: Deep and Narrow

48 layers × 1024 width × 8 heads
Better reasoning
Good for: Complex tasks, multi-step reasoning

Option C: Balanced

> 24 layers × 1536 width × 12 heads
> Middle ground
> Good for: General purpose

Research Consensus: Slightly favor depth over width for most tasks.

5.2 Training vs. Inference Trade-offs

Training Cost ∝ N · D (parameters × tokens) Inference Cost ∝ N · L (parameters × sequence length)

For deployment:

> Depth increases latency (sequential processing)
> Width increases memory (larger activations)
> Heads increase compute (parallel attention)

Optimization Strategies:

1. Knowledge Distillation: Train large (teacher) → compress to small (student)
2. Quantization: Reduce precision (FP32 → INT8)
3. Pruning: Remove less important connections

---

6. Key Takeaways

Architecture Principles

1. Depth enables sequential reasoning: More layers = more computation steps
2. Width enables representation capacity: Larger d_model = more features
3. Multiple heads enable specialization: Different patterns learned in parallel

Scaling Laws

1. Performance is predictable: Power laws govern loss vs. resources
2. Optimal allocation exists: ~20 tokens per parameter is ideal
3. Compute-optimal ≠ Parameter-optimal: Chinchilla beats GPT-3 with fewer parameters

Interpretability

1. Layers specialize hierarchically: Surface → Syntax → Semantics → Reasoning
2. Heads develop specific functions: Positional, syntactic, semantic roles
3. Individual neurons can be meaningful: Some track interpretable concepts

Practical Implications

1. Bigger ≠ Always Better: Must match architecture to task and budget
2. Training regimen matters: Number of tokens as important as parameters
3. Interpretability is limited: We understand patterns, not all mechanisms

---

Real-World Application

When designing your own language model:

1. Estimate budget: $C compute available
2. Apply scaling laws: Determine optimal N (parameters) and D (tokens)
3. Choose architecture: Favor depth for reasoning, width for representation
4. Configure heads: ~8-16 heads is typical, more doesn't always help
5. Plan training: Ensure sufficient tokens per parameter (~20:1 ratio)

Further Reading

Foundational Papers:

Attention Is All You Need (Vaswani et al., 2017)
Language Models are Few-Shot Learners (Brown et al., 2020)
Scaling Laws for Neural Language Models (Kaplan et al., 2020)
Training Compute-Optimal Large Language Models (Hoffmann et al., 2022)

Interpretability:

A Mathematical Framework for Transformer Circuits (Elhage et al., 2021)
Transformer Feed-Forward Layers Are Key-Value Memories (Geva et al., 2021)

Summary

Large language models are:

Architecturally: Deep stacks of attention + FFN layers, with careful depth/width/head balance
Mathematically: Governed by scaling laws that predict performance from compute
Behaviorally: Develop hierarchical representations from surface to semantic to reasoning
Practically: Require massive compute but follow predictable cost-performance curves

Understanding these internals enables informed decisions about model design, training regimens, and deployment strategies—critical skills as language models become central to modern AI systems.

Pre-Read: Interpreting Transformers

The Black Box That Knew Too Much

Thursday, 2:34 PM - AI Research Lab, MIT

Dr. Sarah Chen stared at her screen, bewildered. Her transformer model had just correctly answered a question that should have been impossible.

Question: "The trophy didn't fit in the suitcase because it was too big. What was too big?"

Model's answer: "The trophy."

Sarah had asked the same question to dozens of people. Half said "trophy," half said "suitcase." The sentence was genuinely ambiguous. Yet her model chose "trophy" with 94% confidence.

"How does it *know*?" she muttered.

Her postdoc, Kevin, walked over with coffee. "That's the billion-dollar question, isn't it? We build these things, they work, but we have no idea why."

Sarah pulled up the model's attention patterns. "Look at this. Layer 8, head 3. The word 'it' is attending to 'trophy' with 0.87 weight. But *why*? What did the earlier layers learn that makes this pattern emerge?"

The Investigation Begins

Sarah had been debugging this model for three weeks. It was performing well on benchmarks—too well. Suspiciously well. Her advisor suspected data contamination, but Sarah had verified the training set. The model genuinely understood something.

But what?

"I need to see inside," Sarah said. "Not just the attention patterns. I need to know what information each layer is actually encoding."

Kevin nodded. "Probing classifiers?"

"Exactly. If layer 5 knows the subject of the sentence, there should be a simple linear classifier that can extract it from the activations."

She started coding:

```
# Take activations from layer 5
layer_5_activations = model.get_activations(layer=5, text=sentence)

# Train a simple classifier: Can it predict the subject?
probe = LinearClassifier()
probe.train(layer_5_activations, subject_labels)

accuracy = probe.test()
print(f"Layer 5 encodes subject with {accuracy:.1%} accuracy")
```

The results shocked her:

> Layer 1: Subject detection: 45% (barely above chance)
> Layer 5: Subject detection: 91%
> Layer 8: Subject detection: 97%

"It's learning a hierarchy," Sarah whispered. "Early layers learn syntax. Deep layers learn semantics."

The Attention Roll-Out Mystery

But the attention patterns were still puzzling. Looking at a single layer showed local patterns—word-to-word connections. But information flows through 12 layers. How do these patterns combine?

Sarah implemented attention roll-out, a technique to trace how attention compounds across layers:

```
# Start with identity matrix (each token attends to itself)
rolled_attention = np.eye(seq_length)

# Roll through each layer
for layer in range(12):
    attention_layer = model.get_attention(layer)
    # Multiply: How does token i affect token j through all layers?
    rolled_attention = attention_layer @ rolled_attention

# Now we can see end-to-end attention flow
```

The visualization was stunning. In the rolled-out attention, "it" → "trophy" had a path strength of 0.73, while "it" → "suitcase" was only 0.31.

"Look at this," Sarah showed Kevin. "Layer 2 identifies 'trophy' as the subject. Layer 5 recognizes 'too big' as a size constraint. Layer 8 connects them through the pronoun 'it.' It's not magic—it's a computational pipeline."

The Causal Smoking Gun

But correlation isn't causation. Just because layer 8 shows strong attention doesn't mean it's *responsible* for the answer.

Sarah needed causal tracing.

The idea was elegant: corrupt a specific layer's activation, and see if the model's answer changes. If corrupting layer 5 destroys the model's ability to resolve "it," then layer 5 is causally necessary.

```
def causal_trace(model, sentence, corrupt_layer):
    # Normal forward pass
    normal_output = model(sentence)
```

```
    # Corrupted forward pass: add noise to one layer
    def corrupt_hook(layer_idx):
        if layer_idx == corrupt_layer:
            return lambda x: x + noise
        return lambda x: x

    corrupted_output = model(sentence, hooks=corrupt_hook)

    # Measure change in output
    causal_effect = difference(normal_output, corrupted_output)
    return causal_effect
```

Sarah ran the experiment, corrupting each layer one at a time:

Results:

> Corrupt layer 1: Output changes by 3%
> Corrupt layer 2: Output changes by 12%
> Corrupt layer 5: Output changes by 67% ← Critical layer!
> Corrupt layer 8: Output changes by 41%
> Corrupt layer 11: Output changes by 8%

"Layer 5 is the lynchpin," Sarah said. "That's where it decides what 'it' refers to."

The Discovery

Over the next week, Sarah built a complete interpretability toolkit:

1. Probing Classifiers showed what information each layer encodes:

> Layer 1-3: Part-of-speech, syntax tree depth
> Layer 4-6: Named entities, coreference
> Layer 7-9: Semantic roles, world knowledge
> Layer 10-12: Task-specific reasoning

2. Attention Roll-Out revealed the information flow:

> Early layers: Local syntactic dependencies (subject-verb, article-noun)
> Middle layers: Long-range semantic connections (pronoun-antecedent)
> Late layers: Task-specific focus (question-answer, premise-hypothesis)

3. Causal Tracing identified critical components:

Layer 5, head 2: Responsible for coreference resolution
Layer 8, head 3: Responsible for semantic consistency checking
Layer 11, head 1: Responsible for final answer selection

"It's not a black box anymore," Sarah told her advisor during their weekly meeting. "It's more like a glass factory. We can see every stage of processing."

The Implications

Sarah's paper, "Peering Inside the Transformer: A Three-Method Approach to Interpretability," changed how the field thought about large language models.

Key Findings:

1. Hierarchical Processing: Transformers learn a clear hierarchy—syntax → semantics → reasoning. This mirrors human language processing.
2. Specialized Heads: Different attention heads specialize in different tasks. Head 2 in layer 5 consistently handles pronouns across thousands of examples.
3. Redundancy: Multiple paths can lead to the same answer. Corrupting layer 5 hurts performance, but doesn't destroy it. The model has backup strategies.
4. Emergence: The "understanding" isn't programmed. It emerges from the interaction of 144 attention heads (12 layers × 12 heads) learning to predict the next word.

Her advisor smiled. "So the model doesn't 'know' that trophies are objects and objects can be too big?"

"It does know that," Sarah replied. "But not the way we know it. It's learned a distributed representation where 'trophy,' 'object,' and 'size' cluster together in high-dimensional space. When it sees 'too big,' layer 5 activates the 'object size' neurons, which bias the coreference toward 'trophy.'"

"Can you predict what it will do on new examples?"

Sarah pulled up her causal trace results. "Sometimes. If layer 5 shows high activation for object properties, I can predict it'll resolve pronouns to physical objects. If layer 8 shows high activation for animate entities, it'll prefer people over things."

"That's not a black box," her advisor said. "That's a microscope."

What You'll Learn

In the upcoming session, you'll master the same techniques Sarah used:

Attention Roll-Out: Trace information flow through all layers to understand end-to-end processing
Probing Classifiers: Train simple models on layer activations to decode what information is encoded at each stage
Causal Tracing: Intervene on specific components to identify what's causally responsible for model behavior

By the end, you'll be able to:

1. Visualize and interpret attention patterns across layers
2. Decode the information hierarchy in transformer representations
3. Identify which components are critical for specific behaviors
4. Debug and improve model performance by understanding failure modes

Remember Sarah's insight: Transformers aren't magic. They're computational pipelines. And with the right tools, you can see exactly what they're computing at every step.

Lecture Notes: Interpreting Transformers

Session Overview

Transformers have revolutionized NLP, but understanding *how* they work remains challenging. This session covers three powerful interpretability techniques: attention roll-out (tracking information flow), probing classifiers (decoding layer representations), and causal tracing (identifying critical components).

Learning Objectives

By the end of this session, you will be able to:

1. Compute and visualize attention roll-out to trace information flow
2. Train probing classifiers to decode layer activations
3. Perform causal interventions to identify critical model components
4. Debug transformer behavior using interpretability tools
5. Understand the hierarchical processing in transformer models

---

1. The Interpretability Challenge

1.1 Why Interpretability Matters

The Black Box Problem:

```
# We can use transformers easily
from transformers import AutoModel, AutoTokenizer

model = AutoModel.from_pretrained("bert-base-uncased")
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

input_text = "The trophy didn't fit in the suitcase because it was too big."
inputs = tokenizer(input_text, return_tensors="pt")
outputs = model(**inputs)

# But what happened inside?
# - 12 layers × 12 heads = 144 attention mechanisms
# - 110 million parameters
# - Hidden states of dimension 768
# How do we understand what it learned?
```

Key Questions:

1. What linguistic features does each layer encode?
2. How does information flow through the network?
3. Which components are responsible for specific behaviors?
4. Why does the model fail on certain inputs?

---

2. Attention Roll-Out

2.1 Understanding Attention Composition

The Problem: Looking at attention in a single layer shows local patterns, but information flows through ALL layers. How do attention patterns compose?

Attention Roll-Out Formula:

```
A_rollout = A_L × A_{L-1} × ... × A_2 × A_1

Where:
- A_i is the attention matrix from layer i
- Matrix multiplication captures composition
- Result: end-to-end attention from input to output
```

2.2 Mathematical Foundation

```python
import numpy as np
import torch

def compute_attention_rollout(attention_matrices, head_fusion='mean'):
    """
    Compute attention roll-out across all layers.

    Args:
        attention_matrices: List of attention matrices [num_layers, num_heads,
seq_len, seq_len]
        head_fusion: How to combine multiple heads ('mean', 'max', 'min')

    Returns:
        Rolled-out attention matrix [seq_len, seq_len]
    """
    num_layers = len(attention_matrices)

    # Step 1: Fuse attention heads in each layer
    fused_attention = []
    for layer_attention in attention_matrices:
        # layer_attention shape: [num_heads, seq_len, seq_len]
        if head_fusion == 'mean':
            fused = layer_attention.mean(dim=0)  # Average across heads
        elif head_fusion == 'max':
            fused = layer_attention.max(dim=0)[0]
        else:
            fused = layer_attention.min(dim=0)[0]

        fused_attention.append(fused)

    # Step 2: Add residual connections
```

```
    # In transformers: output = attention(input) + input
    # So: A_effective = 0.5 * A + 0.5 * I (identity)
    seq_len = fused_attention[0].shape[0]
    identity = torch.eye(seq_len)

    for i in range(num_layers):
        fused_attention[i] = 0.5 * fused_attention[i] + 0.5 * identity

    # Step 3: Roll out attention across layers
    rollout = identity.clone()

    for layer_idx in range(num_layers):
        # Matrix multiplication: how token i affects token j through all layers so far
        rollout = fused_attention[layer_idx] @ rollout

    return rollout

# Example usage
num_layers, num_heads, seq_len = 12, 12, 10

# Simulate attention patterns (in practice, extract from model)
attention_patterns = [
    torch.softmax(torch.randn(num_heads, seq_len, seq_len), dim=-1)
    for _ in range(num_layers)
]

rollout = compute_attention_rollout(attention_patterns)
print("Rolled-out attention shape:", rollout.shape)
print("\nAttention from token 0 to all tokens:")
print(rollout[0])
```

2.3 Visualization and Interpretation

```
import matplotlib.pyplot as plt
import seaborn as sns

def visualize_attention_rollout(rollout, tokens):
    """
    Visualize rolled-out attention as a heatmap.

    Args:
        rollout: Attention matrix [seq_len, seq_len]
        tokens: List of token strings
    """
    plt.figure(figsize=(12, 10))

    # Create heatmap
    sns.heatmap(
        rollout.numpy(),
        xticklabels=tokens,
        yticklabels=tokens,
        cmap='YlOrRd',
```

```
        annot=True,
        fmt='.2f',
        square=True,
        cbar_kws={'label': 'Attention Weight'}
    )

    plt.title('Attention Roll-Out: End-to-End Information Flow',
              fontsize=14, fontweight='bold')
    plt.xlabel('Attended To (Source)', fontsize=12)
    plt.ylabel('Attending From (Target)', fontsize=12)
    plt.tight_layout()
    plt.show()

# Example: Pronoun resolution
tokens = ['The', 'trophy', 'did', "n't", 'fit', 'because', 'it', 'was', 'big']
visualize_attention_rollout(rollout[:9, :9], tokens)
```

2.4 Practical Example: Coreference Resolution

```
from transformers import BertModel, BertTokenizer

def analyze_pronoun_attention(sentence, pronoun_idx):
    """
    Analyze what a pronoun attends to across all layers.
    """
    tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
    model = BertModel.from_pretrained('bert-base-uncased', output_attentions=True)

    # Tokenize
    inputs = tokenizer(sentence, return_tensors='pt')
    tokens = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])

    # Forward pass
    outputs = model(**inputs)
    attentions = outputs.attentions  # Tuple of [1, num_heads, seq_len, seq_len]

    # Extract attention matrices (remove batch dimension)
    attention_matrices = [attn[0] for attn in attentions]

    # Compute roll-out
    rollout = compute_attention_rollout(attention_matrices)

    # What does the pronoun attend to?
    pronoun_attention = rollout[pronoun_idx]

    # Find top attended tokens
    top_k = 5
    top_indices = torch.argsort(pronoun_attention, descending=True)[:top_k]

    print(f"Pronoun '{tokens[pronoun_idx]}' attends to:")
    for idx in top_indices:
        print(f"  {tokens[idx]}: {pronoun_attention[idx]:.3f}")
```

```
    return rollout, tokens

# Test
sentence = "The trophy didn't fit in the suitcase because it was too big."
rollout, tokens = analyze_pronoun_attention(sentence, pronoun_idx=13)  # 'it'
```

---

3. Probing Classifiers

3.1 What Are Probing Classifiers?

Core Idea: If a layer's hidden states encode some linguistic property (e.g., part-of-speech), we should be able to train a simple classifier to extract it.

Probing Classifier Architecture:

```
Layer Activations → Linear Classifier → Prediction
     [768]          →    [768 → K]      →    [K]
                                               K classes
```

Key Principle: Use simple classifiers (linear or small MLP) to avoid learning the property from scratch. We want to test if the information is *already there*, not if we can compute it.

3.2 Implementing Probing Classifiers

```
import torch
import torch.nn as nn
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

class LinearProbe(nn.Module):
    """
    Simple linear classifier for probing layer activations.
    """
    def __init__(self, input_dim, num_classes):
        super().__init__()
        self.linear = nn.Linear(input_dim, num_classes)

    def forward(self, x):
        return self.linear(x)

def extract_layer_activations(model, tokenizer, sentences, layer_idx, token_idx=-1):
    """
```

```
    Extract activations from a specific layer for a set of sentences.

    Args:
        model: Transformer model
        tokenizer: Corresponding tokenizer
        sentences: List of sentences
        layer_idx: Which layer to extract (0-11 for BERT-base)
        token_idx: Which token position (-1 for [CLS], or specific position)

    Returns:
        activations: Tensor of shape [num_sentences, hidden_dim]
    """
    model.eval()
    activations = []

    with torch.no_grad():
        for sentence in sentences:
            inputs = tokenizer(sentence, return_tensors='pt', padding=True,
truncation=True)
            outputs = model(**inputs, output_hidden_states=True)

            # outputs.hidden_states: tuple of [batch, seq_len, hidden_dim]
            # Index: [layer_idx][batch_idx, token_idx, :]
            layer_activation = outputs.hidden_states[layer_idx][0, token_idx, :]
            activations.append(layer_activation)

    return torch.stack(activations)

def train_probe(activations, labels, probe_type='linear'):
    """
    Train a probing classifier.

    Args:
        activations: [num_samples, hidden_dim]
        labels: [num_samples]
        probe_type: 'linear' or 'mlp'

    Returns:
        Trained classifier and accuracy
    """
    # Convert to numpy for sklearn
    X = activations.numpy()
    y = labels.numpy()

    # Train-test split
    from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

    if probe_type == 'linear':
        # Logistic regression (linear probe)
        clf = LogisticRegression(max_iter=1000, random_state=42)
    else:
        # Could use sklearn MLPClassifier for non-linear probe
        from sklearn.neural_network import MLPClassifier
```

```
        clf = MLPClassifier(hidden_layer_sizes=(128,), max_iter=1000, random_state=42)

    # Train
    clf.fit(X_train, y_train)

    # Evaluate
    y_pred = clf.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)

    return clf, accuracy

# Example: Probing for part-of-speech
sentences = [
    "The cat sat on the mat.",
    "She runs every morning.",
    "They are happy.",
    # ... more sentences
]
pos_labels = torch.tensor([0, 1, 0, ...])  # 0=noun, 1=verb, etc.

model = BertModel.from_pretrained('bert-base-uncased', output_hidden_states=True)
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# Extract activations from layer 5
activations_layer5 = extract_layer_activations(model, tokenizer, sentences,
layer_idx=5)

# Train probe
probe, accuracy = train_probe(activations_layer5, pos_labels)
print(f"Layer 5 POS accuracy: {accuracy:.1%}")
```

3.3 Probing Across All Layers

```
def probe_all_layers(model, tokenizer, sentences, labels, task_name):
    """
    Probe all layers to see where information is encoded.
    """
    num_layers = model.config.num_hidden_layers + 1  # +1 for embedding layer

    results = []

    for layer_idx in range(num_layers):
        # Extract activations
        activations = extract_layer_activations(model, tokenizer, sentences,
layer_idx)

        # Train probe
        _, accuracy = train_probe(activations, labels)

        results.append({
            'layer': layer_idx,
            'accuracy': accuracy
```

```
        })

        print(f"Layer {layer_idx:2d}: {accuracy:.1%}")

    # Visualize
    import pandas as pd
    df = pd.DataFrame(results)

    plt.figure(figsize=(12, 6))
    plt.plot(df['layer'], df['accuracy'], marker='o', linewidth=2, markersize=8)
    plt.axhline(0.5, color='red', linestyle='--', label='Random baseline')
    plt.xlabel('Layer', fontsize=12)
    plt.ylabel('Probe Accuracy', fontsize=12)
    plt.title(f'Probing Accuracy for {task_name} Across Layers', fontsize=14,
fontweight='bold')
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()

    return df

# Run for different tasks
tasks = {
    'Part-of-Speech': pos_labels,
    'Named Entity': ner_labels,
    'Semantic Role': role_labels
}

for task_name, labels in tasks.items():
    probe_all_layers(model, tokenizer, sentences, labels, task_name)
```

3.4 Interpreting Probe Results

Typical Findings:

```
print("""
Probing Results Interpretation:

Layer 0-2 (Early):
  - POS tagging: 75-85% accuracy
  - Syntax tree depth: 70-80%
  - Character-level features: 90%+

  → Learn SURFACE-level features

Layer 3-6 (Middle):
  - Named entities: 80-90%
  - Coreference: 70-85%
  - Dependency relations: 75-85%

  → Learn STRUCTURAL features
```

```
Layer 7-12 (Deep):
  - Semantic roles: 75-90%
  - Sentiment: 80-90%
  - World knowledge: 60-80%

  → Learn SEMANTIC features

Pattern: Clear hierarchy from syntax to semantics
""")
```

---

4. Causal Tracing

4.1 From Correlation to Causation

The Problem: Probing shows what information is *present*, but not what's *used*. A layer might encode POS tags without actually using them for the task.

Causal Question: "If I corrupt layer L, does the model's behavior change?"

Intervention Logic:

```
Normal:     Input → Layer 1 → ... → Layer L → ... → Output
Corrupted:  Input → Layer 1 → ... → NOISE   → ... → Output?

If output changes significantly → Layer L is causally important
If output unchanged → Layer L is not used (despite encoding info)
```

4.2 Implementing Causal Interventions

```
def causal_intervention(model, tokenizer, sentence, corrupt_layer, noise_level=1.0):
    """
    Perform causal intervention by corrupting a specific layer.

    Args:
        model: Transformer model
        tokenizer: Tokenizer
        sentence: Input sentence
        corrupt_layer: Which layer to corrupt
        noise_level: How much noise to add

    Returns:
        normal_output, corrupted_output, causal_effect
    """
```

```
    inputs = tokenizer(sentence, return_tensors='pt')

    # Normal forward pass
    with torch.no_grad():
        normal_outputs = model(**inputs, output_hidden_states=True)
        normal_hidden = normal_outputs.last_hidden_state

    # Corrupted forward pass using hooks
    corrupted_hidden = None

    def corruption_hook(module, input, output):
        """Hook to corrupt layer activations."""
        nonlocal corrupted_hidden

        # Add Gaussian noise
        noise = torch.randn_like(output) * noise_level * output.std()
        corrupted = output + noise

        # Store for analysis
        corrupted_hidden = corrupted

        return corrupted

    # Register hook on the target layer
    layer = model.encoder.layer[corrupt_layer]  # For BERT
    handle = layer.register_forward_hook(corruption_hook)

    # Forward pass with corruption
    with torch.no_grad():
        corrupted_outputs = model(**inputs, output_hidden_states=True)
        corrupted_final = corrupted_outputs.last_hidden_state

    # Remove hook
    handle.remove()

    # Measure causal effect (L2 distance)
    causal_effect = torch.norm(normal_hidden - corrupted_final, p=2)

    return normal_hidden, corrupted_final, causal_effect.item()

# Test causal importance of each layer
def test_causal_importance(model, tokenizer, sentence):
    """
    Test causal importance of all layers.
    """
    num_layers = model.config.num_hidden_layers
    effects = []

    for layer_idx in range(num_layers):
        _, _, effect = causal_intervention(model, tokenizer, sentence, layer_idx)
        effects.append(effect)
        print(f"Layer {layer_idx:2d}: Causal effect = {effect:.3f}")

    # Visualize
    plt.figure(figsize=(12, 6))
```

```
    plt.bar(range(num_layers), effects, color='steelblue', alpha=0.7)
    plt.xlabel('Layer', fontsize=12)
    plt.ylabel('Causal Effect (L2 Distance)', fontsize=12)
    plt.title('Causal Importance of Each Layer', fontsize=14, fontweight='bold')
    plt.grid(True, alpha=0.3, axis='y')
    plt.tight_layout()
    plt.show()

    return effects

sentence = "The trophy didn't fit in the suitcase because it was too big."
effects = test_causal_importance(model, tokenizer, sentence)
```

4.3 Activation Patching

More Precise Intervention: Instead of adding noise, *replace* activations from a corrupted context.

```
def activation_patching(model, tokenizer, clean_sentence, corrupted_sentence,
                        patch_layer, patch_position):
    """
    Replace activation at a specific position with activation from corrupted run.

    This reveals what specific information is necessary.
    """
    inputs_clean = tokenizer(clean_sentence, return_tensors='pt')
    inputs_corrupt = tokenizer(corrupted_sentence, return_tensors='pt')

    # Get corrupted activation
    with torch.no_grad():
        corrupt_outputs = model(**inputs_corrupt, output_hidden_states=True)
        corrupt_activation = corrupt_outputs.hidden_states[patch_layer][0,
patch_position, :]

    # Patch into clean run
    patched_output = None

    def patching_hook(module, input, output):
        nonlocal patched_output
        # Replace specific position
        output = output.clone()
        output[0, patch_position, :] = corrupt_activation
        patched_output = output
        return output

    layer = model.encoder.layer[patch_layer]
    handle = layer.register_forward_hook(patching_hook)

    with torch.no_grad():
        final_outputs = model(**inputs_clean, output_hidden_states=True)
        final_hidden = final_outputs.last_hidden_state

    handle.remove()
```

```
    return final_hidden

# Example: Does corrupting subject information affect verb prediction?
clean = "The cat sits on the mat."
corrupt = "The dog sits on the mat."  # Different subject

patched = activation_patching(model, tokenizer, clean, corrupt,
                              patch_layer=3, patch_position=2)  # Position of "cat"
```

---

5. Integrated Analysis Workflow

5.1 Complete Interpretability Pipeline

```
class TransformerInterpreter:
    """
    Complete toolkit for transformer interpretability.
    """
    def __init__(self, model_name='bert-base-uncased'):
        self.model = BertModel.from_pretrained(model_name, output_attentions=True,
                                               output_hidden_states=True)
        self.tokenizer = BertTokenizer.from_pretrained(model_name)
        self.model.eval()

    def analyze_sentence(self, sentence):
        """
        Complete analysis: attention, probing, and causal tracing.
        """
        print("="*60)
        print(f"ANALYZING: {sentence}")
        print("="*60)

        # 1. Attention Roll-Out
        print("\n1. ATTENTION ROLL-OUT")
        rollout, tokens = self.attention_rollout(sentence)

        # 2. Layer-wise Probing
        print("\n2. LAYER-WISE INFORMATION")
        info = self.probe_layers(sentence)

        # 3. Causal Importance
        print("\n3. CAUSAL IMPORTANCE")
        effects = self.causal_trace(sentence)

        return {
            'attention': rollout,
            'tokens': tokens,
```

```
            'layer_info': info,
            'causal_effects': effects
        }

    def attention_rollout(self, sentence):
        inputs = self.tokenizer(sentence, return_tensors='pt')
        outputs = self.model(**inputs)

        attentions = [attn[0] for attn in outputs.attentions]
        rollout = compute_attention_rollout(attentions)
        tokens = self.tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])

        return rollout, tokens

    def probe_layers(self, sentence):
        # Implement probing for multiple properties
        pass

    def causal_trace(self, sentence):
        # Implement causal interventions
        pass

# Usage
interpreter = TransformerInterpreter()
results = interpreter.analyze_sentence(
    "The trophy didn't fit in the suitcase because it was too big."
)
```

---

6. Key Takeaways

Attention Roll-Out

✅ Shows end-to-end information flow through all layers ✅ Reveals which tokens influence final representations ✅ Helps understand coreference, long-range dependencies

Probing Classifiers

✅ Decode what information each layer encodes ✅ Reveals hierarchical processing (syntax → semantics) ✅ Simple classifiers = information is explicitly represented

Causal Tracing

✅ Identifies which components are actually *used* ✅ Distinguishes correlation from causation ✅ Reveals critical layers and positions

Integration

Attention Roll-Out: "How does information flow?"
Probing: "What information is present?"
Causal Tracing: "What information is used?"

Together, they provide a complete picture of transformer internals.

---

7. Further Reading

Papers:

"Attention is Not Explanation" (Jain & Wallace, 2019)
"What Does BERT Look At?" (Clark et al., 2019)
"Transformer Feed-Forward Layers Are Key-Value Memories" (Geva et al., 2021)
"Locating and Editing Factual Associations in GPT" (Meng et al., 2022)

Tools:

BertViz: Attention visualization
Captum: Interpretability for PyTorch models
AllenNLP Interpret: Suite of interpretation tools

Pre-Read: Fine-Tuning LLMs & AutoML

The Startup That Couldn't Afford GPT-4

Monday, 9:00 AM - TechStart AI Office, Mumbai

Maya stared at her laptop screen, frustrated. Her startup had just landed a major client: a customer support platform serving 50,000 users. They needed an AI chatbot that could answer product-specific FAQs accurately.

"We can't just use GPT-4 API," her CTO Raj said, walking over. "The costs would eat our entire budget. We need to fine-tune our own model."

Maya groaned. "But fine-tuning a 7-billion parameter model... that needs what, 80GB of GPU memory? We only have a single A100 with 40GB."

Raj smiled. "Have you heard of LoRA?"

The Memory Problem

Raj pulled up a diagram:

```
Traditional Fine-Tuning:
Model: 7B parameters
Full precision (FP32): 7B × 4 bytes = 28GB
Optimizer states: 28GB × 2 = 56GB
Gradients: 28GB
Total: ~112GB GPU memory

Our hardware: 40GB
Result: CRASH!
```

"That's the problem with full fine-tuning," Raj explained. "You need to update ALL parameters, store optimizer states, gradients... it's memory-intensive."

"So what's the solution?" Maya asked.

"We freeze 99.9% of the model. Only train a tiny fraction."

Enter LoRA: The Clever Trick

LoRA = Low-Rank Adaptation

Raj drew on the whiteboard:

```
Standard fine-tuning:
W (Original weights) → W' (Updated weights)
Updates all 7 billion parameters!

LoRA:
W (Frozen!) + ΔW (Trainable)
ΔW = A × B

Where:
A: (d × r) - Down-projection matrix
B: (r × d) - Up-projection matrix
r: Rank (typically 8, 16, 32, 64)

If d = 4096 and r = 16:
ΔW parameters = 4096 × 16 + 16 × 4096 = 131,072
Instead of 4096 × 4096 = 16,777,216

That's 128× fewer parameters!
```

"Wait," Maya interrupted. "You're saying instead of updating a 4096×4096 matrix, we create two small matrices that multiply together?"

"Exactly! And here's the magic: mathematically, you can approximate ANY weight update using low-rank decomposition. For most fine-tuning tasks, a rank of 16 is enough."

The Results:

```
Memory Usage:
Full fine-tuning: 112GB
LoRA fine-tuning: 14GB

Training Speed:
Full fine-tuning: 8 hours
LoRA fine-tuning: 2 hours

Model Quality:
Full fine-tuning: 94.2% accuracy
LoRA fine-tuning: 93.8% accuracy

Cost difference: 10× cheaper!
```

"We lose 0.4% accuracy but save 90% memory," Raj said. "Worth it?"

Maya nodded. "Definitely. But can we go even lower?"

QLoRA: Going Even Smaller

"That's where QLoRA comes in," Raj said, pulling up another diagram.

QLoRA = Quantized LoRA

```
The Quantization Trick:

Normal model:
- Weights stored in FP32 (32 bits per number)
- 7B parameters × 4 bytes = 28GB

4-bit quantization:
- Weights stored in 4 bits
- 7B parameters × 0.5 bytes = 3.5GB

Memory savings: 8× reduction!
```

"Wait," Maya said. "Won't 4-bit precision destroy the model quality?"

Raj grinned. "That's the genius of QLoRA. They use NF4 (NormalFloat4) - a special 4-bit format optimized for neural network weights. Plus double quantization to save even more."

QLoRA Memory Breakdown:

```
Base model (4-bit quantized): 3.5GB
LoRA adapters (FP16): 0.3GB
Optimizer states: 1.2GB
Gradients: 0.8GB
Activation memory: 4GB
Total: ~10GB

Fits on a consumer GPU!
```

"So QLoRA = LoRA + Quantization," Maya summarized. "We get the memory savings of both techniques combined."

The Hyperparameter Nightmare

Two weeks later, Maya had a new problem.

"Raj, I've run 50 training experiments. Different learning rates, LoRA ranks, batch sizes... I still can't beat 91% accuracy. There are too many hyperparameters!"

Raj pulled up a spreadsheet:

```
Hyperparameters to tune:
1. Learning rate: [1e-5, 1e-4, 1e-3]
2. LoRA rank (r): [4, 8, 16, 32, 64]
3. LoRA alpha: [8, 16, 32, 64]
4. Batch size: [4, 8, 16]
5. Number of epochs: [3, 5, 10]
6. Warmup ratio: [0.03, 0.06, 0.1]

Total combinations: 3 × 5 × 4 × 3 × 3 × 3 = 1,620

If each experiment takes 2 hours:
1,620 × 2 = 3,240 hours = 135 days!
```

"That's insane," Maya said. "There has to be a better way."

"There is," Raj said. "AutoML with Optuna."

Optuna: The Smart Search

The Traditional Approach: Grid Search

```
Try every combination:
[1e-5, 1e-4, 1e-3] × [4, 8, 16, 32, 64] × ...

Problems:
- Exhaustive (wastes compute)
- Treats all hyperparameters equally
- Doesn't learn from previous trials
```

The Optuna Approach: Bayesian Optimization

```
Smart search algorithm:

Trial 1: lr=1e-4, rank=16 → accuracy=89.5%
Trial 2: lr=1e-3, rank=8 → accuracy=87.2%
Trial 3: lr=5e-5, rank=32 → accuracy=91.3% ← Good!

Optuna learns: "Lower lr + higher rank = better"

Trial 4: lr=3e-5, rank=64 → accuracy=92.8% ← Even better!
```

```
Converges to optimal in 20-30 trials instead of 1,620!
```

Raj showed Maya the code:

```python
import optuna

def objective(trial):
    # Suggest hyperparameters
    lr = trial.suggest_float("lr", 1e-5, 1e-3, log=True)
    rank = trial.suggest_int("rank", 4, 64)
    alpha = trial.suggest_int("alpha", 8, 64)

    # Train model
    model = train_lora_model(lr=lr, rank=rank, alpha=alpha)
    accuracy = evaluate(model)

    return accuracy

# Run optimization
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=30)

print(f"Best accuracy: {study.best_value}")
print(f"Best hyperparameters: {study.best_params}")
```

"Optuna uses Tree-structured Parzen Estimator (TPE) to model the relationship between hyperparameters and performance," Raj explained. "It's like a smart assistant that learns which combinations work."

The FAQ Model: Putting It All Together

Three weeks later, Maya had results:

```
Dataset: 10,000 customer FAQ pairs
Model: LLaMA-2-7B
Method: QLoRA fine-tuning
Hyperparameter optimization: Optuna

Results after 25 Optuna trials:
Best configuration:
- Learning rate: 3.2e-5
- LoRA rank: 48
- LoRA alpha: 32
- Batch size: 8
- Epochs: 5
```

```
Final Metrics:
- FAQ accuracy: 94.7%
- Response quality: 9.2/10 (human eval)
- Inference speed: 23 tokens/sec
- Memory usage: 11GB

Cost breakdown:
- Total training time: 6 hours
- GPU cost (A100): $2.40/hour × 6 = $14.40
- Optuna trials: 25 × 15 min = 6.25 hours extra
- Total cost: ~$30

Compare to:
- GPT-4 API for 50,000 users: $15,000/month
- Full fine-tuning: Would need 8× A100s = $1,200
```

The client demo went perfectly. The chatbot answered product questions accurately, handled edge cases gracefully, and responded in under a second.

"How did you build this so fast?" the client asked.

Maya smiled. "Smart techniques. LoRA let us fine-tune efficiently. QLoRA made it fit on affordable hardware. Optuna found the best settings automatically."

The LoRA Advantage

Six months later, Maya's startup had 20 clients. Each needed custom models for their domains: legal, medical, e-commerce, HR.

The Old Way:

```
Each client:
- Full 7B model: 28GB per model
- 20 clients: 560GB storage
- Can't batch inference (memory limits)
```

The LoRA Way:

```
One base model: 28GB
20 LoRA adapters: 20 × 300MB = 6GB
Total: 34GB

Bonus: Switch adapters on-the-fly!
```

"We load one base model into GPU memory," Maya explained to investors. "Then we swap LoRA adapters in milliseconds. Client A gets legal expertise. Client B gets medical knowledge. Same base model, different adapters."

The Business Model:

```
Economics:
- Train one adapter: $30
- Deploy one adapter: 300MB storage
- Switch adapters: 50ms latency

Comparison:
- Full fine-tuning: $1,200 per client
- Separate models: 560GB storage
- Model switching: 30 seconds to reload

LoRA made multi-client AI economically viable!
```

The Key Lessons

A year later, Maya spoke at a local ML meetup:

"Three Lessons from Building Production LLMs"

1. Parameter Efficiency is Everything

```
Full fine-tuning: Update 7 billion parameters
LoRA: Train 1 million parameters
Result: 7,000× fewer parameters, 0.4% accuracy loss

ΔW = A × B (low-rank decomposition)
```

2. Quantization Democratizes AI

```
FP32: 28GB (data center only)
4-bit: 3.5GB (consumer hardware)
Quality: Minimal degradation with NF4

QLoRA = LoRA + 4-bit quantization
```

3. AutoML Beats Manual Tuning

```
Grid search: 1,620 experiments
Optuna: 25 experiments
```

```
Time saved: 98.5%

Bayesian optimization > random search > grid search
```

4. Adapters Enable Scale

```
One base model + multiple adapters
Switch in milliseconds
Multi-tenant AI made practical
```

What You'll Learn

In the upcoming session, you'll master Maya's toolkit:

> LoRA Fundamentals: How low-rank decomposition enables efficient fine-tuning
> QLoRA Mechanics: 4-bit quantization with NF4 format
> Optuna Optimization: Bayesian hyperparameter search
> FAQ Fine-Tuning: End-to-end practical implementation
> Production Deployment: Multi-adapter serving strategies

By the end, you'll be able to fine-tune a 7B model on a single consumer GPU in under $30—something that would have cost thousands just two years ago.

Remember Maya's principles:

1. "Don't update what you don't need to" (freeze the base model)
2. "Low-rank approximations are surprisingly good" (LoRA works!)
3. "4 bits are enough" (QLoRA maintains quality)
4. "Let algorithms search, not humans" (Optuna optimization)
5. "Adapters > Full models" (efficient multi-client serving)

: Fine-Tuning LLMs & AutoML

PPT File: Click Here

Session Overview

This session covers parameter-efficient fine-tuning techniques for Large Language Models (LLMs), focusing on LoRA, QLoRA, and automated hyperparameter optimization with Optuna. You'll learn to fine-tune billion-parameter models on consumer hardware while maintaining quality.

Duration: 3 hours
Prerequisites: Deep learning basics, transformers architecture, PyTorch fundamentals

Learning Objectives

By the end of this session, you will be able to:

1. Understand the memory challenges of full fine-tuning
2. Implement LoRA for parameter-efficient training
3. Apply QLoRA for memory-constrained scenarios
4. Use Optuna for hyperparameter optimization
5. Fine-tune models for FAQ and domain-specific tasks
6. Deploy LoRA adapters in production

Part 1: The Fine-Tuning Challenge

Why Fine-Tuning?

Pre-trained LLMs (GPT, LLaMA, BLOOM) are trained on general text. Fine-tuning adapts them to:

Domain-specific tasks (medical, legal, coding)
Instruction following (chatbots, assistants)
Custom knowledge bases (company documentation, FAQs)
Style adaptation (formal, casual, technical)

Traditional Fine-Tuning Memory Requirements

For a 7B parameter model:

```
Memory Components:

1. Model Weights (FP32):
   7B parameters × 4 bytes = 28GB

2. Optimizer States (Adam):
   - First moment: 28GB
   - Second moment: 28GB
   - Total: 56GB

3. Gradients:
   7B parameters × 4 bytes = 28GB

4. Activation Memory (forward pass):
   Varies by batch size
   Typical: 8-12GB

Total Memory: ~112-120GB
```

Problem: Most researchers don't have A100 80GB GPUs!

Comparison:

GPU	Memory	Can Full Fine-Tune 7B?
RTX 3090	24GB	❌ No
RTX 4090	24GB	❌ No
A100 (40GB)	40GB	❌ No
A100 (80GB)	80GB	✅ Yes (barely)
H100	80GB	✅ Yes

Alternative: Parameter-Efficient Fine-Tuning (PEFT)

Core Idea: Don't update all parameters—only train a small subset.

PEFT Methods:

1. Adapter Layers - Insert small trainable layers between frozen layers
2. Prefix Tuning - Optimize continuous task-specific vectors
3. LoRA - Low-rank decomposition of weight updates
4. QLoRA - LoRA + quantization

Part 2: LoRA (Low-Rank Adaptation)

Mathematical Foundation

Weight Update Decomposition:

In traditional fine-tuning, we update weights:

$$W_{new} = W_{pretrained} + \Delta W$$

where

ΔW is a full-rank matrix.

LoRA Hypothesis: Weight updates during fine-tuning have low "intrinsic rank"

Instead of learning full

$\Delta W \in R^{d \times d}$

$d \times d$

, learn:

$\Delta W = BA$

$\Delta W = BA$

where:

- $B \in \mathbb{R}^{d \times r}$
- $B \in \mathbb{R}^{d \times r}$ (down-projection)
- $A \in \mathbb{R}^{r \times d}$
- $A \in \mathbb{R}^{r \times d}$ (up-projection)
- $r \ll d$
- $r \ll d$ (rank is much smaller than dimension)

Parameter Count:

```
Full rank: d × d parameters
Low rank: d × r + r × d = 2dr parameters

Example (d=4096, r=16):
Full: 4096 × 4096 = 16,777,216
LoRA: 2 × 4096 × 16 = 131,072

Reduction: 128× fewer parameters!
```

LoRA Architecture

Transformer Layer with LoRA:

```
Standard Transformer:
x → Linear(W) → output

LoRA Transformer:
x → Linear(W_frozen) + LoRA(BA) → output
         ↑                ↑
      Frozen          Trainable

Forward Pass:
h = W_frozen × x + B × (A × x) × α/r
```

```
Where:
- W_frozen: Pre-trained weights (frozen)
- A, B: LoRA matrices (trainable)
- α: Scaling factor (typically = r)
- r: Rank
```

Which Layers to Apply LoRA?

```
Transformer Block:
┌─────────────────────────────┐
│ Multi-Head Attention        │
│   Q, K, V, O ← LoRA         │ ✓ Most important
├─────────────────────────────┤
│ Feed-Forward Network        │
│   W1, W2 ← LoRA             │ ✓ Optional
└─────────────────────────────┘

Common configurations:
1. Q, V only (minimal)
2. Q, K, V, O (standard)
3. All linear layers (maximum)
```

LoRA Hyperparameters

Key Parameters:

1. Rank (r)

```
# Low rank (faster, less expressive)
r = 4   # 4 dimensions

# Medium rank (balanced)
r = 16  # Default choice

# High rank (slower, more expressive)
r = 64  # For complex tasks
```

Trade-offs:

Rank	Parameters	Training Speed	Quality	Use Case
4	Smallest	Fastest	Good	Simple tasks

8-16	Small	Fast	Very good	Most tasks
32-64	Medium	Moderate	Excellent	Complex tasks
128+	Large	Slow	Marginal gains	Rarely needed

2. Alpha (α)

Scaling factor that controls update magnitude:

```
lora_alpha = 16  # Typically 1× to 2× the rank

# Effective learning rate scaling:
scaling = alpha / r

Example:
r = 8, alpha = 16 → scaling = 2.0
r = 16, alpha = 16 → scaling = 1.0
r = 32, alpha = 16 → scaling = 0.5
```

Rule of thumb: Start with `alpha = r`

3. Dropout

```
lora_dropout = 0.05  # Light regularization
lora_dropout = 0.1   # Standard
lora_dropout = 0.2   # Heavy (for small datasets)
```

LoRA Implementation (PyTorch)

```
import torch
import torch.nn as nn

class LoRALayer(nn.Module):
    def __init__(self, in_features, out_features, rank=16, alpha=16):
        super().__init__()
        self.rank = rank
        self.alpha = alpha

        # LoRA matrices
        self.lora_A = nn.Parameter(torch.randn(in_features, rank))
        self.lora_B = nn.Parameter(torch.zeros(rank, out_features))

        # Scaling
        self.scaling = alpha / rank
```

```
        # Initialize A with kaiming, B with zeros
        nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))

    def forward(self, x):
        # x shape: (batch, seq_len, in_features)
        # LoRA forward: x @ A @ B
        lora_output = (x @ self.lora_A @ self.lora_B) * self.scaling
        return lora_output

class LinearWithLoRA(nn.Module):
    def __init__(self, linear_layer, rank=16, alpha=16):
        super().__init__()

        # Freeze original weights
        self.linear = linear_layer
        for param in self.linear.parameters():
            param.requires_grad = False

        # Add LoRA
        in_features = linear_layer.in_features
        out_features = linear_layer.out_features
        self.lora = LoRALayer(in_features, out_features, rank, alpha)

    def forward(self, x):
        # Frozen linear + LoRA adaptation
        return self.linear(x) + self.lora(x)
```

LoRA with HuggingFace PEFT

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig, get_peft_model

# Load base model
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b-hf",
    torch_dtype=torch.float16,
    device_map="auto"
)

# Configure LoRA
lora_config = LoraConfig(
    r=16,                           # Rank
    lora_alpha=16,                  # Scaling
    target_modules=[                # Which layers
        "q_proj",
        "k_proj",
        "v_proj",
        "o_proj"
    ],
    lora_dropout=0.05,              # Dropout
    bias="none",                    # Don't train biases
```

```
    task_type="CAUSAL_LM"         # Task type
)

# Apply LoRA
model = get_peft_model(model, lora_config)

# Print trainable parameters
model.print_trainable_parameters()
# trainable params: 4,194,304 || all params: 6,742,609,920 || trainable%: 0.06%
```

Part 3: QLoRA (Quantized LoRA)

Quantization Basics

Precision Formats:

Format	Bits	Range	Precision
FP32	32	±3.4e38	~7 digits
FP16	16	±65,504	~3 digits
INT8	8	-128 to 127	Integer
INT4	4	-8 to 7	Integer
NF4	4	Normalized	Special

Memory Impact:

```
7B model in different formats:

FP32: 7B × 4 bytes = 28GB
FP16: 7B × 2 bytes = 14GB
INT8: 7B × 1 byte = 7GB
INT4: 7B × 0.5 bytes = 3.5GB
```

```
Savings: 8× from FP32 to INT4!
```

QLoRA Innovations

1. NormalFloat4 (NF4)

Standard INT4 is uniform: -8, -7, ..., 6, 7

NF4 is information-theoretically optimal for normally distributed weights:

```
# NF4 quantization levels
NF4_LEVELS = [
    -1.0, -0.6961928009986877, -0.5250730514526367,
    -0.39491748809814453, -0.28444138169288635,
    -0.18477343022823334, -0.09105003625154495, 0.0,
    0.07958029955625534, 0.16093020141124725,
    0.24611230194568634, 0.33791524171829224,
    0.44070982933044434, 0.5626170039176941,
    0.7229568362236023, 1.0
]

# These levels are optimized for weights following N(0, σ²)
```

Why NF4 works:

Neural network weights after training follow approximately normal distribution. NF4 allocates more quantization levels near zero (where most weights cluster) and fewer at extremes.

2. Double Quantization

Quantize the quantization constants themselves!

```
Standard quantization:
- Values: Quantized to 4-bit
- Scale factors: Stored in FP32
- Memory: N×0.5 + N/64×4 bytes

Double quantization:
- Values: Quantized to 4-bit
- Scale factors: Quantized to 8-bit
- Memory: N×0.5 + N/64×1 bytes

Savings: ~0.4 bytes per parameter
For 7B model: 2.8GB → 2.4GB
```

3. Paged Optimizers

Handle memory spikes during training:

```
Problem: CUDA out-of-memory during gradient updates

Solution: Page optimizer states to CPU RAM
- Keep on GPU: Active gradients
- Move to CPU: Inactive optimizer states
- Swap as needed
```

QLoRA Configuration

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

# 4-bit quantization config
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,                    # Use 4-bit
    bnb_4bit_quant_type="nf4",           # NormalFloat4
    bnb_4bit_compute_dtype=torch.bfloat16, # Compute in BF16
    bnb_4bit_use_double_quant=True       # Double quantization
)

# Load quantized model
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b-hf",
    quantization_config=bnb_config,
    device_map="auto",
    trust_remote_code=True
)

# Prepare for training
model = prepare_model_for_kbit_training(model)

# Add LoRA
lora_config = LoraConfig(
    r=16,
    lora_alpha=16,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)

model = get_peft_model(model, lora_config)
```

Memory Comparison

```
7B Model Fine-Tuning Memory:

Full Fine-Tuning (FP32):
- Model: 28GB
- Optimizer: 56GB
```

```
- Gradients: 28GB
- Activations: 12GB
Total: ~124GB

LoRA (FP16):
- Model: 14GB (frozen, FP16)
- LoRA params: 0.3GB (trainable)
- Optimizer: 0.6GB (only LoRA)
- Gradients: 0.3GB (only LoRA)
- Activations: 8GB
Total: ~23GB

QLoRA (4-bit):
- Model: 3.5GB (frozen, NF4)
- LoRA params: 0.3GB (trainable, FP16)
- Optimizer: 0.6GB (only LoRA)
- Gradients: 0.3GB (only LoRA)
- Activations: 4GB (quantized)
Total: ~9GB

Fits on RTX 3090! (24GB)
```

Part 4: Hyperparameter Optimization with Optuna

The Hyperparameter Search Problem

Hyperparameters in LLM Fine-Tuning:

```python
hyperparameters = {
    # Model architecture
    'lora_rank': [4, 8, 16, 32, 64],
    'lora_alpha': [8, 16, 32, 64],
    'lora_dropout': [0.0, 0.05, 0.1],

    # Training
    'learning_rate': [1e-5, 3e-5, 5e-5, 1e-4],
    'batch_size': [4, 8, 16],
    'num_epochs': [3, 5, 10],
    'warmup_ratio': [0.03, 0.06, 0.1],

    # Regularization
    'weight_decay': [0.0, 0.01, 0.1],
    'max_grad_norm': [0.3, 1.0, 2.0]
}
```

```
# Total combinations: 5×4×3×4×3×3×3×3×3 = 29,160
```

Search Strategies:

Method	Trials Needed	Intelligence	Parallelizable
Grid Search	All (29,160)	None	✓ Yes
Random Search	100-500	Low	✓ Yes
Bayesian (Optuna)	20-50	High	Partial

Optuna Architecture

Core Concepts:

1. Study - Optimization experiment

```
study = optuna.create_study(
    direction="maximize",  # or "minimize"
    study_name="llm-finetuning"
)
```

2. Trial - Single experiment

```
def objective(trial):
    lr = trial.suggest_float("lr", 1e-5, 1e-3, log=True)
    return train_and_eval(lr)
```

3. Sampler - Search algorithm

TPE (Tree-structured Parzen Estimator) - Default
CmaEs - Covariance Matrix Adaptation
Random - Baseline

4. Pruner - Early stopping

```
pruner = optuna.pruners.MedianPruner(
    n_startup_trials=5,
```

```
    n_warmup_steps=100
)
```

Tree-Structured Parzen Estimator (TPE)

How TPE Works:

```
1. Separate trials into "good" and "bad" based on threshold
   Good: Top 25% by performance
   Bad: Bottom 75%

2. Model hyperparameter distributions:
   P(x | good) - Distribution of params that worked well
   P(x | bad) - Distribution of params that worked poorly

3. Select next hyperparameters by maximizing:
   EI(x) = P(x | good) / P(x | bad)

   High EI → Likely to be good

4. Train model, add result, repeat
```

Example:

```
After 10 trials:

Learning rate distribution:
Good trials: Clustered around 3e-5
Bad trials: Spread across 1e-5 to 1e-3

TPE suggests: lr ≈ 3e-5 (high EI)

After 20 trials:
Good trials: lr ∈ [2e-5, 5e-5], rank ∈ [16, 32]
TPE suggests: lr=3.5e-5, rank=24
```

Optuna for LLM Fine-Tuning

```python
import optuna
from transformers import TrainingArguments, Trainer

def objective(trial):
    # Suggest hyperparameters
    lr = trial.suggest_float("learning_rate", 1e-5, 1e-3, log=True)
    lora_r = trial.suggest_int("lora_rank", 4, 64, step=4)
    lora_alpha = trial.suggest_int("lora_alpha", 8, 64, step=8)
    batch_size = trial.suggest_categorical("batch_size", [4, 8, 16])
    warmup_ratio = trial.suggest_float("warmup_ratio", 0.0, 0.1)
```

```
    # Configure LoRA
    lora_config = LoraConfig(
        r=lora_r,
        lora_alpha=lora_alpha,
        target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
        lora_dropout=0.05,
        task_type="CAUSAL_LM"
    )

    # Apply LoRA to model
    model = get_peft_model(base_model, lora_config)

    # Training arguments
    training_args = TrainingArguments(
        output_dir=f"./trial_{trial.number}",
        learning_rate=lr,
        per_device_train_batch_size=batch_size,
        num_train_epochs=3,
        warmup_ratio=warmup_ratio,
        logging_steps=10,
        evaluation_strategy="epoch",
        save_strategy="epoch",
        load_best_model_at_end=True,
        metric_for_best_model="eval_loss"
    )

    # Train
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_dataset,
        eval_dataset=eval_dataset
    )

    trainer.train()

    # Evaluate
    metrics = trainer.evaluate()
    eval_loss = metrics["eval_loss"]

    # Report intermediate values for pruning
    trial.report(eval_loss, step=0)

    # Check if trial should be pruned
    if trial.should_prune():
        raise optuna.TrialPruned()

    return eval_loss

# Create study
study = optuna.create_study(
    direction="minimize",
    sampler=optuna.samplers.TPESampler(),
    pruner=optuna.pruners.MedianPruner(n_startup_trials=5)
```

```
)

# Optimize
study.optimize(objective, n_trials=30, timeout=86400)  # 24 hours

# Results
print(f"Best trial: {study.best_trial.number}")
print(f"Best value: {study.best_value}")
print(f"Best params: {study.best_params}")
```

Optuna Visualization

```
import optuna.visualization as vis

# Plot optimization history
fig = vis.plot_optimization_history(study)
fig.show()

# Plot parameter importances
fig = vis.plot_param_importances(study)
fig.show()

# Plot parallel coordinate
fig = vis.plot_parallel_coordinate(study)
fig.show()

# Plot slice (per parameter)
fig = vis.plot_slice(study)
fig.show()
```

Part 5: FAQ Model Fine-Tuning

Dataset Preparation

FAQ Dataset Structure:

```
[
  {
    "question": "How do I reset my password?",
    "answer": "Click 'Forgot Password' on the login page...",
    "category": "account"
  },
  {
    "question": "What payment methods do you accept?",
```

```
    "answer": "We accept credit cards, PayPal, and wire transfer...",
    "category": "billing"
  }
]
```

Convert to Instruction Format:

```
def format_faq_instruction(example):
    """Convert FAQ to instruction-following format."""
    instruction = f"""### Question:
{example['question']}

### Answer:"""

    response = example['answer']

    # Combine for training
    text = f"{instruction}\n{response}"

    return {"text": text}

# Apply to dataset
dataset = dataset.map(format_faq_instruction)
```

Tokenization:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b-hf")
tokenizer.pad_token = tokenizer.eos_token

def tokenize_function(examples):
    # Tokenize
    outputs = tokenizer(
        examples["text"],
        truncation=True,
        max_length=512,
        padding="max_length"
    )

    # For causal LM, labels = input_ids
    outputs["labels"] = outputs["input_ids"].copy()

    return outputs

tokenized_dataset = dataset.map(
    tokenize_function,
    batched=True,
    remove_columns=dataset.column_names
)
```

Complete Fine-Tuning Pipeline

```
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    TrainingArguments,
    Trainer,
    DataCollatorForLanguageModeling
)
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from datasets import load_dataset
import torch

# 1. Load model with QLoRA
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True
)

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b-hf",
    quantization_config=bnb_config,
    device_map="auto"
)

model = prepare_model_for_kbit_training(model)

# 2. Configure LoRA
lora_config = LoraConfig(
    r=16,
    lora_alpha=16,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)

model = get_peft_model(model, lora_config)

# 3. Load and prepare dataset
dataset = load_dataset("json", data_files="faq_data.json")
dataset = dataset["train"].train_test_split(test_size=0.1)

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b-hf")
tokenizer.pad_token = tokenizer.eos_token

def tokenize(examples):
    return tokenizer(
        examples["text"],
        truncation=True,
        max_length=512,
        padding="max_length"
```

```
    )

tokenized = dataset.map(tokenize, batched=True)

# 4. Training arguments
training_args = TrainingArguments(
    output_dir="./faq-model",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    fp16=True,
    logging_steps=10,
    evaluation_strategy="steps",
    eval_steps=100,
    save_strategy="steps",
    save_steps=100,
    save_total_limit=3,
    load_best_model_at_end=True,
    warmup_ratio=0.03,
    lr_scheduler_type="cosine"
)

# 5. Create trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized["train"],
    eval_dataset=tokenized["test"],
    data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False)
)

# 6. Train
trainer.train()

# 7. Save adapter
model.save_pretrained("./faq-lora-adapter")
```

Inference with Fine-Tuned Model

```
from peft import PeftModel

# Load base model
base_model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b-hf",
    torch_dtype=torch.float16,
    device_map="auto"
)

# Load LoRA adapter
model = PeftModel.from_pretrained(
    base_model,
```

```
    "./faq-lora-adapter"
)

# Tokenizer
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b-hf")

# Generate answer
def answer_faq(question):
    prompt = f"""### Question:
{question}

### Answer:"""

    inputs = tokenizer(prompt, return_tensors="pt").to("cuda")

    outputs = model.generate(
        **inputs,
        max_new_tokens=256,
        temperature=0.7,
        top_p=0.9,
        do_sample=True
    )

    response = tokenizer.decode(outputs[0], skip_special_tokens=True)

    # Extract answer (after prompt)
    answer = response.split("### Answer:")[-1].strip()

    return answer

# Test
question = "How do I reset my password?"
answer = answer_faq(question)
print(answer)
```

Part 6: Advanced Techniques

Multi-Adapter Serving

```
from peft import PeftModel

# Load base model once
base_model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b-hf",
    torch_dtype=torch.float16,
    device_map="auto"
)
```

```
# Dictionary of adapters
adapters = {
    "legal": "./adapters/legal-lora",
    "medical": "./adapters/medical-lora",
    "customer_support": "./adapters/support-lora"
}

# Switch adapters dynamically
def generate_with_adapter(prompt, adapter_name):
    # Load adapter
    model = PeftModel.from_pretrained(base_model, adapters[adapter_name])

    # Generate
    inputs = tokenizer(prompt, return_tensors="pt").to("cuda")
    outputs = model.generate(**inputs, max_new_tokens=128)

    return tokenizer.decode(outputs[0], skip_special_tokens=True)

# Use different adapters
legal_response = generate_with_adapter("What is contract law?", "legal")
medical_response = generate_with_adapter("What causes diabetes?", "medical")
```

Merging LoRA Weights

```
from peft import PeftModel

# Load base + adapter
model = PeftModel.from_pretrained(base_model, "./lora-adapter")

# Merge adapter into base weights
merged_model = model.merge_and_unload()

# Save merged model (can use without PEFT)
merged_model.save_pretrained("./merged-model")
tokenizer.save_pretrained("./merged-model")

# Now can load as standard model
merged = AutoModelForCausalLM.from_pretrained("./merged-model")
```

Gradient Checkpointing

Save memory by recomputing activations:

```
model.gradient_checkpointing_enable()

# Trade: 20% slower training for 40% less memory
```

Part 7: Best Practices

Choosing LoRA Rank

```
Task Complexity → Rank

Simple (sentiment analysis): r=4-8
Medium (FAQ, summarization): r=16-32
Complex (code generation): r=32-64
Very complex (multi-task): r=64-128

Rule: Start with r=16, increase if underfitting
```

Learning Rate Guidelines

```
# LoRA typically needs higher LR than full fine-tuning
lr_guidelines = {
    "full_finetuning": 1e-5,
    "lora": 2e-4,
    "qlora": 3e-4  # Can be even higher
}

# Scheduler
"cosine"  # Smooth decay (recommended)
"linear"  # Linear decay
"constant_with_warmup"  # Flat after warmup
```

Data Requirements

```
Minimum samples per task:
- Simple classification: 100-500
- FAQ/QA: 500-2000
- General instruction: 2000-10000
- Domain adaptation: 10000+

Quality > Quantity
```

Evaluation Metrics

```
# For FAQ/QA
metrics = {
```

```
    "perplexity": lower is better,
    "exact_match": answer == ground_truth,
    "f1_score": token overlap,
    "bleu": n-gram overlap,
    "human_eval": quality rating
}
```

Part 8: Common Pitfalls

Overfitting

Symptoms:

> Train loss ↓↓, Eval loss ↑
> Perfect training accuracy, poor test

Solutions:

```
- Increase dropout: lora_dropout=0.1
- Add weight decay: weight_decay=0.01
- Reduce epochs
- Use more data
- Decrease rank
```

Catastrophic Forgetting

Problem: Model forgets general knowledge after fine-tuning

Solutions:

```
- Use lower learning rate
- Shorter training (3-5 epochs max)
- Mix general data with task data
- Use LoRA instead of full fine-tuning
```

CUDA Out of Memory

Solutions:

```
1. Reduce batch size
```

```
2. Enable gradient checkpointing
3. Use QLoRA instead of LoRA
4. Reduce sequence length
5. Use gradient accumulation
```

Key Takeaways

LoRA Principles:

1. Low-rank decomposition: ΔW = BA
2. Freeze base model, train adapters
3. 100-1000× parameter reduction
4. Minimal quality loss (<1%)

QLoRA Advantages:

1. NF4 quantization (4-bit)
2. Double quantization
3. Paged optimizers
4. 7B models on 24GB GPUs

Optuna Benefits:

1. Bayesian optimization (TPE)
2. 50-100× fewer trials than grid search
3. Early pruning
4. Parallel execution

FAQ Fine-Tuning:

1. Instruction format critical
2. 500-2000 examples sufficient
3. r=16, lr=2e-4 typical
4. 3-5 epochs optimal

Production Deployment:

1. Multi-adapter serving
2. Sub-second adapter switching
3. Merge for deployment
4. Quantize for inference

Practical Exercise

Build an FAQ chatbot:

1. Collect 1000 FAQ pairs from your domain
2. Format as instructions
3. Fine-tune LLaMA-2-7B with QLoRA
4. Optimize hyperparameters with Optuna
5. Deploy adapter
6. Evaluate quality

Expected results:

> Training: 4-6 hours on single A100
> Memory: <12GB
> Quality: 90%+ on domain FAQs
> Cost: <$50 total

Further Reading

> LoRA Paper: "LoRA: Low-Rank Adaptation of Large Language Models" (Hu et al., 2021)
> QLoRA Paper: "QLoRA: Efficient Finetuning of Quantized LLMs" (Dettmers et al., 2023)
> Optuna: https://optuna.org
> HuggingFace PEFT: https://github.com/huggingface/peft

Cost & Performance Trade-offs

The $50,000 Wake-Up Call

Monday, 9:00 AM - TechStart AI Office, Bangalore

Rohan stared at his laptop screen, his coffee forgotten. The AWS billing dashboard showed a number that made his stomach drop.

Month-to-date charges: $47,832.18

"That can't be right," he muttered. Their startup, ChatMed AI, had launched their medical Q&A chatbot just three weeks ago. The budget was $5,000 for the entire month.

He called Priya, their ML engineer.

"Priya, we have a problem. Our GPU costs are through the roof."

The Investigation Begins

10:00 AM - War Room

Priya pulled up their usage logs:

```
GPT-4 API Usage (Last 3 weeks):
- Total requests: 2.4 million
- Total tokens processed: 18.6 billion
- Average tokens per request: 7,750

Cost breakdown:
Input tokens:  12.4B × $0.03/1K = $372,000 ❌
Output tokens: 6.2B × $0.06/1K = $372,000 ❌
Total: $744,000/month projected!

Current spend (3 weeks): $47,832
Projected monthly: $64,000+
```

"Wait," Rohan said. "7,750 tokens per request? That's... huge."

Priya opened a random conversation log:

```
User: "What are the side effects of aspirin?"

System context sent to API:
- Entire medical knowledge base: 50,000 tokens
```

```
- Conversation history (all 20 exchanges): 3,000 tokens
- User query: 10 tokens
- System instructions: 200 tokens

Total INPUT: 53,210 tokens
For a 10-token question!

Response generated: 500 tokens

Total cost per query: $1.59 + $0.03 = $1.62
At 50,000 queries/day = $81,000/day!
```

Rohan's eyes widened. "We're sending the entire knowledge base with every single question?"

"I thought more context = better answers," Priya said quietly.

"We need help. Now."

The Optimization Expert

Tuesday, 2:00 PM

They hired Vikram, an LLM optimization consultant. He spent one hour analyzing their system and came back with a report.

"You're making four critical mistakes," he said, pulling up a presentation.

Mistake #1: Context Window Waste

```
Current approach:
Every query: 53,210 input tokens

Optimized approach:
1. Semantic search for relevant context
2. Send only top 5 most relevant documents
3. Limit conversation history to last 3 exchanges

Result:
- Relevant docs: 2,000 tokens
- Recent history: 500 tokens
- User query: 10 tokens
- System instructions: 200 tokens
```

```
Total INPUT: 2,710 tokens (19× reduction!)

Cost per query: $0.08 (was $1.62)
Savings: 95%
```

Vikram explained:

"Think of context like packing for a trip. You don't bring your entire wardrobe. You bring what you need."

"More context ≠ better answers. Relevant context = better answers."

Mistake #2: No Batching

```
Current approach:
Process queries one at a time
- Query 1: Wait for GPU, process, return
- Query 2: Wait for GPU, process, return
- Query 3: Wait for GPU, process, return

GPU utilization: 30%
Throughput: 10 queries/second

Optimized approach (batching):
Collect 32 queries
Process all together in one GPU pass
Return all results

GPU utilization: 85%
Throughput: 320 queries/second (32× faster!)
```

Real example:

```python
# Before (No batching)
for query in queries:  # 1000 queries
    response = model.generate(query)
    # Time: 100 seconds (10 QPS)

# After (Batching)
batch_size = 32
for i in range(0, len(queries), batch_size):
    batch = queries[i:i+batch_size]
    responses = model.generate_batch(batch)
    # Time: 3.125 seconds (320 QPS)

Speedup: 32×
```

```
Cost reduction: Same GPU time, 32× more queries
Cost per query: 97% reduction
```

Mistake #3: Wrong Model for the Job

```
Current setup:
- Every query uses GPT-4 (most expensive)
- Simple queries: "What is aspirin?" → GPT-4
- Complex queries: "Interaction between..." → GPT-4

Cost per 1K tokens:
GPT-4: $0.03 input, $0.06 output
GPT-3.5: $0.001 input, $0.002 output (30× cheaper!)

Optimized approach:
1. Classify query complexity
2. Simple queries → GPT-3.5 (95% of queries)
3. Complex queries → GPT-4 (5% of queries)

Result:
Average cost per query: $0.15 → $0.008 (95% reduction)
```

The Rule:

"Don't use a sledgehammer to crack a nut. Save GPT-4 for when you really need it."

Mistake #4: No Caching

```
Analysis of queries:
"What are the side effects of aspirin?" - Asked 2,847 times
"How much aspirin is safe?" - Asked 1,923 times
"Can I take aspirin with ibuprofen?" - Asked 1,456 times

Top 100 questions = 68% of all queries

Current: Generate fresh response every time
Cost: 2,847 × $0.08 = $227.76 for same question!

Optimized: Cache common responses
Cost: 1 × $0.08 = $0.08 (first time only)
Subsequent queries: $0 (cache hit)

For top 100 questions:
```

```
Without cache: $15,400/day
With cache: $8/day + cache storage
Savings: 99.95%
```

The Implementation

Wednesday - Friday: Three Days of Optimization

Priya and her team implemented all four fixes:

Day 1: Context Window Optimization

```python
# Before
context = entire_knowledge_base  # 50,000 tokens

# After
def get_relevant_context(query, max_tokens=2000):
    # Semantic search for relevant docs
    relevant_docs = vector_search(query, top_k=5)

    # Truncate if needed
    context = ""
    token_count = 0

    for doc in relevant_docs:
        doc_tokens = count_tokens(doc)
        if token_count + doc_tokens <= max_tokens:
            context += doc
            token_count += doc_tokens
        else:
            break

    return context

context = get_relevant_context(user_query)  # ~2,000 tokens
```

Day 2: Implement Batching

```python
# Before
def process_queries(queries):
    results = []
    for query in queries:
        result = model.generate(query)  # One at a time
        results.append(result)
    return results
```

```python
# After
def process_queries_batched(queries, batch_size=32):
    results = []
    for i in range(0, len(queries), batch_size):
        batch = queries[i:i+batch_size]
        batch_results = model.generate_batch(batch)  # All at once
        results.extend(batch_results)
    return results
```

Day 3: Model Routing + Caching

```python
import hashlib
from redis import Redis

cache = Redis()

def classify_complexity(query):
    # Simple heuristic: length + keywords
    if len(query.split()) < 10 and no_medical_terms(query):
        return "simple"
    return "complex"

def get_response(query):
    # Check cache first
    query_hash = hashlib.md5(query.encode()).hexdigest()
    cached = cache.get(query_hash)

    if cached:
        return cached  # Cache hit - $0 cost!

    # Route to appropriate model
    complexity = classify_complexity(query)

    if complexity == "simple":
        response = gpt35_model.generate(query)  # Cheap
    else:
        response = gpt4_model.generate(query)   # Expensive

    # Cache for future
    cache.setex(query_hash, 3600, response)  # 1 hour TTL

    return response
```

The Results

Monday, 9:00 AM - One Week Later

Rohan pulled up the new metrics:

```
Week 1 (Before Optimization):
- Queries processed: 350,000
- Total cost: $47,832
- Cost per query: $0.137
- GPU utilization: 30%
- Throughput: 10 QPS

Week 2 (After Optimization):
- Queries processed: 420,000 (20% more!)
- Total cost: $892
- Cost per query: $0.002 (98.5% reduction!)
- GPU utilization: 82%
- Throughput: 280 QPS (28× faster)

Projected monthly cost:
Before: $64,000
After: $950

Savings: $63,050/month (98.5%)
```

The Breakdown:

```
Optimization impact:

Context reduction (53K → 2.7K tokens):
Savings: 95% on token costs

Batching (1 → 32 queries/batch):
Savings: 97% on GPU time

Model routing (100% GPT-4 → 5% GPT-4, 95% GPT-3.5):
Savings: 95% on remaining costs

Caching (68% cache hit rate):
Savings: 68% on cached queries

Combined effect:
0.05 × 0.03 × 0.05 × 0.32 = 0.0000024
= 99.9998% theoretical max savings
= 98.5% actual savings (accounting for overhead)
```

The Lessons Learned

Rohan's Retrospective (Company All-Hands)

"We learned four critical lessons about LLM costs:"

1. Token Economics Matter

```
Every token costs money:
- Input tokens: $0.03 per 1K (GPT-4)
- Output tokens: $0.06 per 1K (GPT-4)

A 10,000-token conversation:
- 7,000 input: $0.21
- 3,000 output: $0.18
- Total: $0.39

At 100,000 conversations/day:
- Daily cost: $39,000
- Monthly cost: $1,170,000

Reduce tokens by 95%:
- Monthly cost: $58,500
- Savings: $1,111,500
```

2. Context Length ≠ Quality

```
Sending more context:
❌ Wastes tokens
❌ Increases latency
❌ May confuse model
❌ Costs more

Sending relevant context:
✓ Reduces tokens
✓ Faster responses
✓ Better focus
✓ Much cheaper
```

3. Batching is Magic

```
Processing queries individually:
- GPU sits idle between queries
- Low utilization (30%)
- High cost per query

Processing in batches:
- GPU always busy
- High utilization (85%)
- Low cost per query
- Much faster throughput

Trade-off: Slight latency increase
```

```
(Wait to form batch before processing)
```

4. Not All Queries Need GPT-4

```
Query complexity distribution:
- Simple: 70% (can use GPT-3.5)
- Medium: 25% (can use GPT-3.5)
- Complex: 5% (needs GPT-4)

Cost comparison (per 1K tokens):
GPT-3.5: $0.001
GPT-4: $0.03 (30× more expensive!)

Smart routing:
Use GPT-4 only when necessary
Save 95% on model costs
```

The Final Metrics

Three Months Later

```
ChatMed AI Statistics:

Users: 50,000 → 250,000 (5× growth)
Queries/day: 50,000 → 400,000 (8× growth)
Monthly LLM cost: $892 (down from $64,000)
Cost per query: $0.002 (down from $0.137)

ROI on optimization:
- Time invested: 3 days
- Consultant cost: $5,000
- Monthly savings: $63,050
- Payback period: 2.8 hours (yes, hours!)
- Annual savings: $756,600

Additional benefits:
- 28× faster response times
- Better user experience
- Scaled to 5× more users
- No infrastructure changes needed
```

The Four Pillars of LLM Cost Optimization

Rohan's Framework (Now Teaching Others)

Pillar 1: Token Management

```
Measure: Count every token
Optimize: Send only what's needed
Monitor: Track token usage trends
```

Pillar 2: Context Optimization

```
Principle: Relevance > Volume
Technique: Semantic search + truncation
Target: <3,000 tokens per query
```

Pillar 3: GPU Efficiency

```
Strategy: Batch processing
Goal: >80% GPU utilization
Method: Collect queries, process together
```

Pillar 4: Smart Routing

```
Philosophy: Right model for right task
Implementation: Complexity classifier
Result: 95% cost reduction possible
```

What You'll Learn

In the upcoming session, you'll master the techniques that saved ChatMed AI $756,600 annually:

Token Economics:

How to count tokens accurately
Where token costs come from

How to minimize token usage
Token vs context length tradeoffs

Context Length Management:

Context window limits
Chunking strategies
Semantic search for relevance
Cost per token calculations

GPU Hour Optimization:

Batch size selection
Throughput vs latency tradeoffs
GPU utilization monitoring
Cost per query optimization

Batching Strategies:

When to batch
Optimal batch sizes
Latency implications
Real-world implementation

Cost-Performance Tradeoffs:

Model selection (GPT-4 vs GPT-3.5)
Caching strategies
Quality vs cost balance
Monitoring and optimization

Remember Rohan's Rules:

1. "Measure before you optimize" - Know your baseline
2. "Context is expensive" - Send only what's relevant
3. "Batch everything you can" - GPU efficiency matters
4. "Cache aggressively" - Don't recompute the same thing
5. "Right model, right task" - Don't overpay for capability you don't need

By the end of this session, you'll be able to:

Calculate exact LLM costs for any workload
Optimize token usage by 90%+
Implement efficient batching
Choose the right model for each task
Build cost-effective LLM applications

Cost & Performance Trade-offs in LLM Applications

PPT File: Click Here

Colab File: Click Here

Session Overview

This session covers the critical economic and performance considerations when deploying Large Language Models in production. You'll learn how to optimize token usage, manage context windows, maximize GPU utilization, implement effective batching, and make informed cost-performance tradeoffs.

Duration: 3 hours
Prerequisites: Understanding of LLMs, basic Python, API usage

Learning Objectives

By the end of this session, you will be able to:

1. Calculate LLM costs accurately using token-based pricing
2. Optimize context windows to minimize token usage

3. Maximize GPU efficiency through batching strategies
4. Implement cost-saving techniques like caching and model routing
5. Make informed tradeoffs between cost, latency, and quality
6. Monitor and optimize production LLM applications

Part 1: Token Economics

Understanding Token-Based Pricing

Core Concept: LLM APIs charge per token, not per API call or word.

Token Definition:

```
Token ≠ Word

Tokens are subword units created by tokenization algorithms (BPE)

Examples:
"Hello" → ["Hello"] (1 token)
"Hello world" → ["Hello", " world"] (2 tokens)
"supercalifragilisticexpialidocious" →
  ["super", "cal", "if", "rag", "il", "istic", "exp", "ial", "id", "oci", "ous"]
  (11 tokens)

Average ratio: 1 word ≈ 1.3 tokens (English)
```

Pricing Models (OpenAI GPT-4 Example)

```
Model: GPT-4 (8K context)
Input pricing: $0.03 per 1,000 tokens
Output pricing: $0.06 per 1,000 tokens

Model: GPT-3.5-turbo
Input pricing: $0.001 per 1,000 tokens
Output pricing: $0.002 per 1,000 tokens

GPT-4 is 30× more expensive than GPT-3.5!
```

Calculating Request Costs

Formula:

```
Cost per request = (Input tokens × Input price per 1K / 1000)
                 + (Output tokens × Output price per 1K / 1000)
```

Example 1: Simple Query

```python
import tiktoken

# Initialize tokenizer for GPT-4
enc = tiktoken.encoding_for_model("gpt-4")

# Input
user_query = "What is the capital of France?"
system_prompt = "You are a helpful assistant."

input_text = system_prompt + "\n" + user_query
input_tokens = len(enc.encode(input_text))

# Output
response = "The capital of France is Paris."
output_tokens = len(enc.encode(response))

# Calculate cost
input_cost = (input_tokens / 1000) * 0.03
output_cost = (output_tokens / 1000) * 0.06
total_cost = input_cost + output_cost

print(f"Input tokens: {input_tokens}")
print(f"Output tokens: {output_tokens}")
print(f"Total cost: ${total_cost:.6f}")

# Output:
# Input tokens: 13
# Output tokens: 8
# Total cost: $0.000870
```

Example 2: Long Document Analysis

```python
# Long document
document = """[10,000 word article about AI]"""
system_prompt = "Summarize this article in 3 paragraphs."

input_text = system_prompt + "\n" + document
input_tokens = len(enc.encode(input_text))  # ~13,000 tokens

# Expected output
output_tokens = 200  # Estimated summary length

# Calculate cost
input_cost = (input_tokens / 1000) * 0.03   # $0.39
output_cost = (output_tokens / 1000) * 0.06  # $0.012
total_cost = input_cost + output_cost        # $0.402
```

```
print(f"Cost for this request: ${total_cost:.3f}")

# At 1,000 requests/day:
daily_cost = total_cost * 1000  # $402/day
monthly_cost = daily_cost * 30  # $12,060/month

print(f"Monthly cost at scale: ${monthly_cost:,.0f}")
```

Token Counting Tools

```
import tiktoken

def count_tokens(text, model="gpt-4"):
    """Count tokens in text for a specific model."""
    enc = tiktoken.encoding_for_model(model)
    return len(enc.encode(text))

def estimate_cost(input_text, output_tokens, model="gpt-4"):
    """Estimate cost for a request."""
    pricing = {
        "gpt-4": {"input": 0.03, "output": 0.06},
        "gpt-3.5-turbo": {"input": 0.001, "output": 0.002},
    }

    input_tokens = count_tokens(input_text, model)

    input_cost = (input_tokens / 1000) * pricing[model]["input"]
    output_cost = (output_tokens / 1000) * pricing[model]["output"]

    return {
        "input_tokens": input_tokens,
        "output_tokens": output_tokens,
        "input_cost": input_cost,
        "output_cost": output_cost,
        "total_cost": input_cost + output_cost
    }

# Example usage
text = "Your input text here..." * 1000
cost = estimate_cost(text, output_tokens=500)

print(f"Total cost: ${cost['total_cost']:.4f}")
```

Token Optimization Strategies

1. Truncate Unnecessary Context

```
def truncate_to_token_limit(text, max_tokens=3000, model="gpt-4"):
    """Truncate text to fit within token limit."""
    enc = tiktoken.encoding_for_model(model)
```

```
    tokens = enc.encode(text)

    if len(tokens) <= max_tokens:
        return text

    # Truncate and decode
    truncated_tokens = tokens[:max_tokens]
    truncated_text = enc.decode(truncated_tokens)

    return truncated_text

# Example
long_text = "..." * 100000
optimized = truncate_to_token_limit(long_text, max_tokens=2000)

print(f"Original: {count_tokens(long_text)} tokens")
print(f"Optimized: {count_tokens(optimized)} tokens")
print(f"Savings: {count_tokens(long_text) - count_tokens(optimized)} tokens")
```

2. Remove Redundant Information

```
def remove_duplicates(conversation_history):
    """Remove duplicate information from history."""
    seen = set()
    unique_messages = []

    for message in conversation_history:
        # Create hash of message content
        content_hash = hash(message["content"])

        if content_hash not in seen:
            seen.add(content_hash)
            unique_messages.append(message)

    return unique_messages

# Example
history = [
    {"role": "user", "content": "What is AI?"},
    {"role": "assistant", "content": "AI is..."},
    {"role": "user", "content": "What is AI?"},  # Duplicate!
    {"role": "user", "content": "Tell me more"}
]

optimized_history = remove_duplicates(history)
print(f"Reduced from {len(history)} to {len(optimized_history)} messages")
```

3. Compress System Prompts

```
# Inefficient system prompt (verbose)
verbose_prompt = """
You are a helpful AI assistant. You should always be polite and
```

```
courteous. You should provide accurate information. You should
ask clarifying questions when needed. You should format your
responses clearly. You should cite sources when possible.
"""

# Optimized system prompt (concise)
concise_prompt = """
You are a helpful AI assistant. Be polite, accurate, and clear.
Ask clarifying questions and cite sources.
"""

print(f"Verbose: {count_tokens(verbose_prompt)} tokens")
print(f"Concise: {count_tokens(concise_prompt)} tokens")
print(f"Savings: {count_tokens(verbose_prompt) - count_tokens(concise_prompt)}
tokens")

# At 1 million requests:
savings_per_million = (count_tokens(verbose_prompt) - count_tokens(concise_prompt)) *
1_000_000
cost_savings = (savings_per_million / 1000) * 0.03
print(f"Cost savings: ${cost_savings:.2f} per million requests")
```

Part 2: Context Length Management

Context Window Limits

Definition: Maximum number of tokens (input + output) a model can process in one request.

```
Model Context Windows:

GPT-3.5-turbo: 4,096 tokens
GPT-4: 8,192 tokens
GPT-4-32k: 32,768 tokens
GPT-4-turbo: 128,000 tokens
Claude-2: 100,000 tokens
Claude-3: 200,000 tokens

Context = Input tokens + Output tokens

Example (GPT-4):
Max context: 8,192 tokens
Input: 7,000 tokens
Available for output: 1,192 tokens
```

The Context Window Problem

Scenario: Chatbot with conversation history

```
class Chatbot:
    def __init__(self):
        self.history = []
        self.max_context = 4096  # GPT-3.5 limit

    def add_message(self, role, content):
        self.history.append({"role": role, "content": content})

    def get_total_tokens(self):
        full_history = "\n".join([
            f"{msg['role']}: {msg['content']}"
            for msg in self.history
        ])
        return count_tokens(full_history)

    def chat(self, user_message):
        self.add_message("user", user_message)

        # Check context limit
        total_tokens = self.get_total_tokens()

        if total_tokens > self.max_context - 500:  # Reserve 500 for response
            print(f"WARNING: Context full ({total_tokens} tokens)")
            return None

        # Make API call...
        return response

# Problem: After 20 exchanges, context fills up!
bot = Chatbot()
for i in range(20):
    bot.chat(f"Question {i+1}")
    # Eventually: WARNING: Context full
```

Context Management Strategies

1. Sliding Window (Keep Recent History)

```
class SlidingWindowChatbot:
    def __init__(self, max_messages=10):
        self.history = []
        self.max_messages = max_messages
        self.system_prompt = "You are a helpful assistant."

    def add_message(self, role, content):
        self.history.append({"role": role, "content": content})
```

```
        # Keep only last N messages
        if len(self.history) > self.max_messages:
            # Keep system prompt + recent messages
            self.history = self.history[-self.max_messages:]

    def get_context(self):
        messages = [{"role": "system", "content": self.system_prompt}]
        messages.extend(self.history)
        return messages

# Example
bot = SlidingWindowChatbot(max_messages=6)

for i in range(20):
    bot.add_message("user", f"Question {i+1}")
    bot.add_message("assistant", f"Answer {i+1}")
    print(f"History length: {len(bot.history)}")

# Output: History never exceeds 6 messages
```

2. Summarization (Compress Old History)

```
class SummarizingChatbot:
    def __init__(self):
        self.history = []
        self.summary = ""
        self.max_history_tokens = 2000

    def should_summarize(self):
        history_text = "\n".join([
            f"{msg['role']}: {msg['content']}"
            for msg in self.history
        ])
        return count_tokens(history_text) > self.max_history_tokens

    def summarize_history(self):
        """Compress old history into summary."""
        history_text = "\n".join([
            f"{msg['role']}: {msg['content']}"
            for msg in self.history[:-3]  # Keep last 3 exchanges
        ])

        # Call LLM to summarize
        summary_prompt = f"Summarize this conversation:\n{history_text}"
        summary = call_llm(summary_prompt, max_tokens=200)

        # Update state
        self.summary = summary
        self.history = self.history[-3:]  # Keep only recent

    def get_context(self):
        context = ""
        if self.summary:
```

```
            context += f"Previous conversation summary: {self.summary}\n\n"

        context += "Recent messages:\n"
        for msg in self.history:
            context += f"{msg['role']}: {msg['content']}\n"

        return context

# Example
bot = SummarizingChatbot()

for i in range(50):
    bot.add_message("user", f"Question about topic {i}")
    bot.add_message("assistant", f"Detailed answer about topic {i}")

    if bot.should_summarize():
        bot.summarize_history()
        print(f"Summarized! Summary tokens: {count_tokens(bot.summary)}")
```

3. Semantic Search (Retrieve Relevant Context)

```
from sentence_transformers import SentenceTransformer
import numpy as np

class SemanticChatbot:
    def __init__(self):
        self.encoder = SentenceTransformer('all-MiniLM-L6-v2')
        self.messages = []
        self.embeddings = []

    def add_message(self, role, content):
        self.messages.append({"role": role, "content": content})

        # Create embedding
        embedding = self.encoder.encode(content)
        self.embeddings.append(embedding)

    def get_relevant_context(self, query, top_k=5):
        """Retrieve most relevant past messages."""
        # Encode query
        query_embedding = self.encoder.encode(query)

        # Calculate similarities
        similarities = [
            np.dot(query_embedding, emb) /
            (np.linalg.norm(query_embedding) * np.linalg.norm(emb))
            for emb in self.embeddings
        ]

        # Get top k most relevant
        top_indices = np.argsort(similarities)[-top_k:][::-1]
        relevant = [self.messages[i] for i in top_indices]
```

```
        return relevant

    def chat(self, user_query):
        # Get relevant history instead of all history
        relevant_context = self.get_relevant_context(user_query, top_k=3)

        # Build prompt
        context = "Relevant conversation history:\n"
        for msg in relevant_context:
            context += f"{msg['role']}: {msg['content']}\n"

        context += f"\nCurrent query: {user_query}"

        # Make API call with only relevant context
        return call_llm(context)

# Benefit: Constant context size regardless of conversation length!
```

Cost Comparison: Context Strategies

```
def compare_context_strategies():
    """Compare costs of different context management approaches."""

    # Simulation parameters
    num_exchanges = 50
    avg_message_tokens = 50

    # Strategy 1: Full history (no management)
    full_history_tokens = num_exchanges * 2 * avg_message_tokens  # 5,000 tokens
    full_cost = (full_history_tokens / 1000) * 0.03

    # Strategy 2: Sliding window (keep last 10 messages)
    window_tokens = 10 * 2 * avg_message_tokens  # 1,000 tokens
    window_cost = (window_tokens / 1000) * 0.03

    # Strategy 3: Summarization (200 token summary + 6 recent)
    summary_tokens = 200 + (6 * avg_message_tokens)  # 500 tokens
    summary_cost = (summary_tokens / 1000) * 0.03
    summary_cost += 0.01  # One-time cost to generate summary

    # Strategy 4: Semantic search (top 5 relevant)
    semantic_tokens = 5 * avg_message_tokens  # 250 tokens
    semantic_cost = (semantic_tokens / 1000) * 0.03

    print("Cost per request after 50 exchanges:")
    print(f"Full history: ${full_cost:.4f}")
    print(f"Sliding window: ${window_cost:.4f} ({(1-window_cost/full_cost)*100:.1f}%
savings)")
    print(f"Summarization: ${summary_cost:.4f} ({(1-summary_cost/full_cost)*100:.1f}%
savings)")
    print(f"Semantic search: ${semantic_cost:.4f}
({(1-semantic_cost/full_cost)*100:.1f}% savings)")
```

```
    # At scale (1 million requests)
    print(f"\nCost at 1 million requests:")
    print(f"Full history: ${full_cost * 1_000_000:,.0f}")
    print(f"Semantic search: ${semantic_cost * 1_000_000:,.0f}")
    print(f"Savings: ${(full_cost - semantic_cost) * 1_000_000:,.0f}")

compare_context_strategies()

# Output:
# Full history: $0.1500
# Sliding window: $0.0300 (80.0% savings)
# Summarization: $0.0250 (83.3% savings)
# Semantic search: $0.0075 (95.0% savings)
#
# Cost at 1 million requests:
# Full history: $150,000
# Semantic search: $7,500
# Savings: $142,500
```

Part 3: GPU Hours and Utilization

Understanding GPU Costs

GPU Pricing (Example: AWS):

```
NVIDIA A100 (40GB):
- On-demand: $4.10 per hour
- 1-year reserved: $2.40 per hour
- 3-year reserved: $1.60 per hour

NVIDIA V100 (16GB):
- On-demand: $3.06 per hour
- Reserved: ~$1.20-1.80 per hour

Cost breakdown:
Idle GPU: Still costs full price!
50% utilized: Still costs full price!
100% utilized: Same price, but 2× throughput

Lesson: Utilization matters!
```

GPU Utilization Metrics

```python
import time
import torch

class GPUMonitor:
    def __init__(self):
        self.start_time = None
        self.active_time = 0
        self.idle_time = 0

    def start_task(self):
        if self.start_time is None:
            self.start_time = time.time()
        self.task_start = time.time()

    def end_task(self):
        task_duration = time.time() - self.task_start
        self.active_time += task_duration

    def wait(self, duration):
        time.sleep(duration)
        self.idle_time += duration

    def get_utilization(self):
        total_time = time.time() - self.start_time
        return (self.active_time / total_time) * 100 if total_time > 0 else 0

    def get_stats(self):
        total_time = time.time() - self.start_time
        return {
            "total_time": total_time,
            "active_time": self.active_time,
            "idle_time": self.idle_time,
            "utilization": self.get_utilization(),
            "wasted_cost": (self.idle_time / 3600) * 4.10  # A100 hourly rate
        }

# Example: Sequential processing
monitor = GPUMonitor()

for i in range(100):
    monitor.start_task()
    # Process single query (0.1 seconds)
    time.sleep(0.1)
    monitor.end_task()

    # Wait for next query (0.9 seconds)
    monitor.wait(0.9)

stats = monitor.get_stats()
print(f"GPU Utilization: {stats['utilization']:.1f}%")
print(f"Wasted cost: ${stats['wasted_cost']:.2f}")

# Output:
# GPU Utilization: 10.0%
# Wasted cost: $3.69
```

```
# (90% of GPU time wasted!)
```

Batch Processing for GPU Efficiency

Problem: Processing queries one at a time wastes GPU capacity

```
Sequential processing:
Query 1: ███░░░░░░░░░░░░░░░░░ (5% GPU used)
Query 2: ███░░░░░░░░░░░░░░░░░ (5% GPU used)
Query 3: ███░░░░░░░░░░░░░░░░░ (5% GPU used)
...

GPU utilization: 5%
Throughput: 10 queries/second
Cost per query: $0.041 (full GPU hour / 10 QPS)
```

Solution: Batch processing

```
Batch processing (32 queries):
Batch 1: ████████████████████ (90% GPU used)
Batch 2: ████████████████████ (90% GPU used)
Batch 3: ████████████████████ (90% GPU used)
...

GPU utilization: 90%
Throughput: 180 queries/second (18× faster!)
Cost per query: $0.0023 (full GPU hour / 180 QPS)

Savings: 95%
```

Implementing Batching

```python
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
import time

class BatchedInference:
    def __init__(self, model_name, batch_size=32):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForCausalLM.from_pretrained(model_name)
        self.batch_size = batch_size
        self.model.eval()

        # Move to GPU if available
        if torch.cuda.is_available():
            self.model = self.model.cuda()

    def generate_sequential(self, prompts):
```

```
        """Process prompts one at a time (inefficient)."""
        start = time.time()
        results = []

        for prompt in prompts:
            inputs = self.tokenizer(prompt, return_tensors="pt")
            if torch.cuda.is_available():
                inputs = {k: v.cuda() for k, v in inputs.items()}

            with torch.no_grad():
                outputs = self.model.generate(**inputs, max_length=50)

            result = self.tokenizer.decode(outputs[0])
            results.append(result)

        duration = time.time() - start
        return results, duration

    def generate_batched(self, prompts):
        """Process prompts in batches (efficient)."""
        start = time.time()
        results = []

        # Process in batches
        for i in range(0, len(prompts), self.batch_size):
            batch = prompts[i:i+self.batch_size]

            # Tokenize batch
            inputs = self.tokenizer(
                batch,
                return_tensors="pt",
                padding=True,
                truncation=True
            )

            if torch.cuda.is_available():
                inputs = {k: v.cuda() for k, v in inputs.items()}

            # Generate for entire batch at once
            with torch.no_grad():
                outputs = self.model.generate(**inputs, max_length=50)

            # Decode batch results
            batch_results = [
                self.tokenizer.decode(output)
                for output in outputs
            ]
            results.extend(batch_results)

        duration = time.time() - start
        return results, duration

# Comparison
model = BatchedInference("gpt2", batch_size=32)
```

```
prompts = ["Tell me about AI"] * 100

# Sequential
seq_results, seq_time = model.generate_sequential(prompts)
print(f"Sequential: {seq_time:.2f}s ({len(prompts)/seq_time:.1f} QPS)")

# Batched
batch_results, batch_time = model.generate_batched(prompts)
print(f"Batched: {batch_time:.2f}s ({len(prompts)/batch_time:.1f} QPS)")

print(f"Speedup: {seq_time/batch_time:.1f}×")

# Output:
# Sequential: 50.0s (2.0 QPS)
# Batched: 5.0s (20.0 QPS)
# Speedup: 10.0×
```

Optimal Batch Size Selection

```
def find_optimal_batch_size(model, sample_prompts, gpu_memory_gb=16):
    """Find optimal batch size for GPU."""
    batch_sizes = [1, 2, 4, 8, 16, 32, 64, 128]
    results = []

    for batch_size in batch_sizes:
        try:
            # Test this batch size
            batch = sample_prompts[:batch_size]

            start = time.time()
            model.generate_batched(batch)
            duration = time.time() - start

            throughput = batch_size / duration

            results.append({
                "batch_size": batch_size,
                "duration": duration,
                "throughput": throughput,
                "efficiency": throughput / batch_size  # QPS per query in batch
            })

            print(f"Batch size {batch_size:3d}: {throughput:6.1f} QPS")

        except RuntimeError as e:
            if "out of memory" in str(e):
                print(f"Batch size {batch_size:3d}: OOM (too large)")
                break
            raise

    # Find batch size with best throughput
    best = max(results, key=lambda x: x["throughput"])
```

```
    print(f"\nOptimal batch size: {best['batch_size']}")
    print(f"Throughput: {best['throughput']:.1f} QPS")

    return best["batch_size"]

# Example output:
# Batch size   1:   10.0 QPS
# Batch size   2:   18.0 QPS
# Batch size   4:   32.0 QPS
# Batch size   8:   55.0 QPS
# Batch size  16:   85.0 QPS
# Batch size  32:  120.0 QPS
# Batch size  64:  130.0 QPS  ← Diminishing returns
# Batch size 128: OOM (too large)
#
# Optimal batch size: 64
```

Part 4: Batching Strategies and Tradeoffs

Latency vs Throughput Tradeoff

Without Batching:

```
Request arrives → Process immediately → Return result

Latency per request: 100ms
Throughput: 10 QPS
User experience: Instant response ✓
```

With Batching:

```
Collect requests for 500ms → Process batch → Return results

Latency per request: 100ms (processing) + 500ms (waiting) = 600ms
Throughput: 64 QPS (if batch_size=32)
User experience: 500ms delay ✗
```

The Tradeoff:

```
                 Latency    Throughput    Cost/Query
No batching       Low (100ms)    Low (10 QPS)     High ($0.041)
Small batch (8)   Medium (200ms) Medium (40 QPS)  Medium ($0.010)
Large batch (64)  High (600ms)   High (100 QPS)   Low ($0.004)
```

```
Choose based on your requirements:
- Real-time chat: Low batching (or none)
- Bulk processing: Large batching
- API service: Medium batching with timeout
```

Dynamic Batching

Strategy: Collect queries until batch is full OR timeout occurs

```python
import asyncio
from collections import deque
import time

class DynamicBatcher:
    def __init__(self, model, batch_size=32, max_wait_ms=100):
        self.model = model
        self.batch_size = batch_size
        self.max_wait_ms = max_wait_ms
        self.queue = deque()
        self.processing = False

    async def add_request(self, prompt):
        """Add request to batch queue."""
        future = asyncio.Future()
        self.queue.append((prompt, future))

        # Start processing if not already running
        if not self.processing:
            asyncio.create_task(self.process_batch())

        # Wait for result
        return await future

    async def process_batch(self):
        """Process accumulated batch."""
        self.processing = True
        start_time = time.time()

        while True:
            # Check if should process now
            batch_full = len(self.queue) >= self.batch_size
            timeout_reached = (time.time() - start_time) * 1000 >= self.max_wait_ms

            if batch_full or (timeout_reached and len(self.queue) > 0):
                # Extract batch
                batch_size = min(len(self.queue), self.batch_size)
                batch = [self.queue.popleft() for _ in range(batch_size)]

                # Process
                prompts = [item[0] for item in batch]
                results = self.model.generate_batched(prompts)
```

```
                # Return results to waiting futures
                for (prompt, future), result in zip(batch, results):
                    future.set_result(result)

                # Reset timer
                start_time = time.time()

            # Check if done
            if len(self.queue) == 0:
                self.processing = False
                break

            # Small sleep to accumulate more requests
            await asyncio.sleep(0.01)

# Usage
async def main():
    batcher = DynamicBatcher(model, batch_size=32, max_wait_ms=100)

    # Simulate requests arriving
    tasks = []
    for i in range(100):
        task = asyncio.create_task(batcher.add_request(f"Query {i}"))
        tasks.append(task)

        # Random arrival times
        await asyncio.sleep(random.uniform(0, 0.02))

    # Wait for all results
    results = await asyncio.gather(*tasks)
    print(f"Processed {len(results)} requests")

asyncio.run(main())
```

Adaptive Batch Sizing

```
class AdaptiveBatcher:
    def __init__(self, model):
        self.model = model
        self.batch_size = 8  # Start small
        self.min_batch = 4
        self.max_batch = 128

        # Performance tracking
        self.latencies = deque(maxlen=100)
        self.throughputs = deque(maxlen=100)

    def adjust_batch_size(self):
        """Adapt batch size based on performance."""
        if len(self.latencies) < 10:
            return  # Not enough data
```

```
        avg_latency = sum(self.latencies) / len(self.latencies)
        avg_throughput = sum(self.throughputs) / len(self.throughputs)

        # If latency too high, reduce batch size
        if avg_latency > 500:  # ms
            self.batch_size = max(self.min_batch, self.batch_size // 2)
            print(f"Reducing batch size to {self.batch_size} (latency:
{avg_latency:.0f}ms)")

        # If latency acceptable and throughput can improve, increase
        elif avg_latency < 200 and self.batch_size < self.max_batch:
            self.batch_size = min(self.max_batch, self.batch_size * 2)
            print(f"Increasing batch size to {self.batch_size} (throughput:
{avg_throughput:.0f} QPS)")

    def process_batch(self, prompts):
        """Process batch and track metrics."""
        start = time.time()

        results = self.model.generate_batched(prompts[:self.batch_size])

        duration = time.time() - start
        latency_ms = duration * 1000
        throughput = len(prompts) / duration

        # Track metrics
        self.latencies.append(latency_ms)
        self.throughputs.append(throughput)

        # Periodically adjust
        if len(self.latencies) % 10 == 0:
            self.adjust_batch_size()

        return results
```

Part 5: Cost-Performance Tradeoffs

Model Selection Strategies

Cost vs Quality:

```
def route_to_model(query, complexity_threshold=0.7):
    """Route query to appropriate model based on complexity."""

    # Simple complexity heuristic
    complexity_score = calculate_complexity(query)
```

```
    if complexity_score < complexity_threshold:
        # Use cheaper model
        model = "gpt-3.5-turbo"
        cost_multiplier = 1.0
    else:
        # Use more capable model
        model = "gpt-4"
        cost_multiplier = 30.0

    return model, cost_multiplier

def calculate_complexity(query):
    """Estimate query complexity."""
    # Factors:
    # - Length
    # - Technical terms
    # - Multiple questions
    # - Ambiguity

    score = 0.0

    # Length factor
    words = len(query.split())
    if words > 50:
        score += 0.3
    elif words > 20:
        score += 0.1

    # Technical terms
    technical_terms = ["algorithm", "quantum", "derivative", "synthesis"]
    if any(term in query.lower() for term in technical_terms):
        score += 0.4

    # Multiple questions
    if query.count("?") > 1:
        score += 0.2

    # Requires reasoning
    reasoning_words = ["why", "how", "explain", "compare"]
    if any(word in query.lower() for word in reasoning_words):
        score += 0.3

    return min(score, 1.0)

# Example usage
queries = [
    "What is 2+2?",  # Simple
    "Explain the quantum entanglement paradox",  # Complex
    "How does gradient descent work in neural networks?",  # Complex
    "What time is it?"  # Simple
]

for query in queries:
    model, cost_mult = route_to_model(query)
    print(f"Query: {query[:50]}...")
```

```
    print(f"  → Route to {model} (cost: {cost_mult}×)")
    print()

# Output:
# Query: What is 2+2?
#   → Route to gpt-3.5-turbo (cost: 1.0×)
#
# Query: Explain the quantum entanglement paradox
#   → Route to gpt-4 (cost: 30.0×)
```

Caching Strategies

```
import hashlib
from redis import Redis
import json

class ResponseCache:
    def __init__(self, ttl=3600):
        self.cache = Redis()
        self.ttl = ttl  # Time to live in seconds

        # Statistics
        self.hits = 0
        self.misses = 0
        self.cost_saved = 0.0

    def get_cache_key(self, prompt):
        """Generate cache key from prompt."""
        # Normalize prompt
        normalized = prompt.lower().strip()

        # Create hash
        return hashlib.md5(normalized.encode()).hexdigest()

    def get(self, prompt):
        """Get cached response if exists."""
        key = self.get_cache_key(prompt)
        cached = self.cache.get(key)

        if cached:
            self.hits += 1
            return json.loads(cached)

        self.misses += 1
        return None

    def set(self, prompt, response, cost):
        """Cache response."""
        key = self.get_cache_key(prompt)
        value = json.dumps({
            "response": response,
            "cost": cost,
```

```
            "timestamp": time.time()
        })

        self.cache.setex(key, self.ttl, value)

    def get_with_generation(self, prompt, generate_fn):
        """Get from cache or generate and cache."""
        # Try cache first
        cached = self.get(prompt)
        if cached:
            self.cost_saved += cached["cost"]
            return cached["response"], 0.0  # No cost for cache hit

        # Generate
        response = generate_fn(prompt)
        cost = calculate_cost(prompt, response)

        # Cache for future
        self.set(prompt, response, cost)

        return response, cost

    def get_stats(self):
        """Get cache statistics."""
        total = self.hits + self.misses
        hit_rate = (self.hits / total * 100) if total > 0 else 0

        return {
            "hits": self.hits,
            "misses": self.misses,
            "hit_rate": hit_rate,
            "cost_saved": self.cost_saved
        }

# Usage example
cache = ResponseCache(ttl=3600)

common_queries = [
    "What is Python?",
    "How do I install NumPy?",
    "What is machine learning?"
]

# Simulate traffic with repeated queries
for _ in range(1000):
    query = random.choice(common_queries)
    response, cost = cache.get_with_generation(query, call_llm)

    # 70% of queries are from top 3 common queries
    # These will be cache hits after first time

stats = cache.get_stats()
print(f"Cache hit rate: {stats['hit_rate']:.1f}%")
print(f"Cost saved: ${stats['cost_saved']:.2f}")
```

```
# Output:
# Cache hit rate: 69.7%
# Cost saved: $27.88
# (Only 30.3% of queries hit the expensive LLM!)
```

Quality vs Cost Monitoring

```
class QualityMonitor:
    def __init__(self):
        self.responses = []

    def log_response(self, query, response, model, cost, quality_score):
        """Log response with quality metrics."""
        self.responses.append({
            "query": query,
            "response": response,
            "model": model,
            "cost": cost,
            "quality": quality_score
        })

    def analyze_tradeoffs(self):
        """Analyze cost-quality tradeoffs."""
        # Group by model
        by_model = {}
        for resp in self.responses:
            model = resp["model"]
            if model not in by_model:
                by_model[model] = {"costs": [], "qualities": []}

            by_model[model]["costs"].append(resp["cost"])
            by_model[model]["qualities"].append(resp["quality"])

        # Calculate averages
        print("Cost-Quality Analysis:")
        for model, data in by_model.items():
            avg_cost = sum(data["costs"]) / len(data["costs"])
            avg_quality = sum(data["qualities"]) / len(data["qualities"])

            print(f"\n{model}:")
            print(f"  Average cost: ${avg_cost:.4f}")
            print(f"  Average quality: {avg_quality:.2f}/5.0")
            print(f"  Cost per quality point: ${avg_cost/avg_quality:.4f}")

        # Find optimal model
        best_value = None
        best_ratio = float('inf')

        for model, data in by_model.items():
            avg_cost = sum(data["costs"]) / len(data["costs"])
            avg_quality = sum(data["qualities"]) / len(data["qualities"])
            ratio = avg_cost / avg_quality
```

```
            if ratio < best_ratio:
                best_ratio = ratio
                best_value = model

        print(f"\nBest value model: {best_value}")
        print(f"Cost per quality point: ${best_ratio:.4f}")

# Example output:
# Cost-Quality Analysis:
#
# gpt-3.5-turbo:
#   Average cost: $0.0015
#   Average quality: 3.80/5.0
#   Cost per quality point: $0.0004
#
# gpt-4:
#   Average cost: $0.0450
#   Average quality: 4.60/5.0
#   Cost per quality point: $0.0098
#
# Best value model: gpt-3.5-turbo
# Cost per quality point: $0.0004
```

Part 6: Best Practices and Monitoring

Comprehensive Cost Tracking

```
class LLMCostTracker:
    def __init__(self):
        self.requests = []
        self.daily_totals = {}

    def log_request(self, model, input_tokens, output_tokens, latency_ms):
        """Log a request with all relevant metrics."""
        # Calculate cost
        pricing = {
            "gpt-4": {"input": 0.03, "output": 0.06},
            "gpt-3.5-turbo": {"input": 0.001, "output": 0.002}
        }

        input_cost = (input_tokens / 1000) * pricing[model]["input"]
        output_cost = (output_tokens / 1000) * pricing[model]["output"]
        total_cost = input_cost + output_cost

        # Log
        timestamp = time.time()
```

```
        date = time.strftime("%Y-%m-%d", time.localtime(timestamp))

        request = {
            "timestamp": timestamp,
            "date": date,
            "model": model,
            "input_tokens": input_tokens,
            "output_tokens": output_tokens,
            "total_tokens": input_tokens + output_tokens,
            "cost": total_cost,
            "latency_ms": latency_ms
        }

        self.requests.append(request)

        # Update daily total
        if date not in self.daily_totals:
            self.daily_totals[date] = 0.0
        self.daily_totals[date] += total_cost

    def get_summary(self, last_n_days=7):
        """Get cost summary."""
        cutoff = time.time() - (last_n_days * 86400)
        recent = [r for r in self.requests if r["timestamp"] >= cutoff]

        if not recent:
            return "No data"

        total_cost = sum(r["cost"] for r in recent)
        total_requests = len(recent)
        total_tokens = sum(r["total_tokens"] for r in recent)
        avg_latency = sum(r["latency_ms"] for r in recent) / total_requests

        # By model
        by_model = {}
        for r in recent:
            model = r["model"]
            if model not in by_model:
                by_model[model] = {"count": 0, "cost": 0.0}
            by_model[model]["count"] += 1
            by_model[model]["cost"] += r["cost"]

        print(f"Summary (Last {last_n_days} days):")
        print(f"  Total requests: {total_requests:,}")
        print(f"  Total cost: ${total_cost:.2f}")
        print(f"  Total tokens: {total_tokens:,}")
        print(f"  Avg latency: {avg_latency:.0f}ms")
        print(f"  Cost per request: ${total_cost/total_requests:.4f}")
        print(f"\nBy model:")
        for model, stats in by_model.items():
            print(f"  {model}:")
            print(f"    Requests: {stats['count']:,}
({stats['count']/total_requests*100:.1f}%)")
            print(f"    Cost: ${stats['cost']:.2f}
({stats['cost']/total_cost*100:.1f}%)")
```

```
        return {
            "total_cost": total_cost,
            "total_requests": total_requests,
            "by_model": by_model
        }

# Usage
tracker = LLMCostTracker()

# Log some requests
tracker.log_request("gpt-4", 1000, 500, 1200)
tracker.log_request("gpt-3.5-turbo", 500, 200, 300)
# ... log thousands more ...

# Get summary
tracker.get_summary(last_n_days=7)
```

Alerts and Budget Management

```
class BudgetManager:
    def __init__(self, daily_budget=100, monthly_budget=2000):
        self.daily_budget = daily_budget
        self.monthly_budget = monthly_budget
        self.tracker = LLMCostTracker()

    def check_budget(self):
        """Check if budget exceeded."""
        today = time.strftime("%Y-%m-%d")
        month = time.strftime("%Y-%m")

        # Daily check
        daily_spend = self.tracker.daily_totals.get(today, 0.0)
        if daily_spend > self.daily_budget:
            self.alert(f"Daily budget exceeded: ${daily_spend:.2f} >
${self.daily_budget}")
        elif daily_spend > self.daily_budget * 0.8:
            self.alert(f"Warning: 80% of daily budget used (${daily_spend:.2f})")

        # Monthly check
        monthly_spend = sum(
            cost for date, cost in self.tracker.daily_totals.items()
            if date.startswith(month)
        )

        if monthly_spend > self.monthly_budget:
            self.alert(f"Monthly budget exceeded: ${monthly_spend:.2f} >
${self.monthly_budget}")
        elif monthly_spend > self.monthly_budget * 0.8:
            self.alert(f"Warning: 80% of monthly budget used (${monthly_spend:.2f})")

    def alert(self, message):
```

```
        """Send alert (implement your notification system)."""
        print(f"ALERT: {message}")
        # Send email, Slack message, etc.

    def project_monthly_cost(self):
        """Project end-of-month cost based on current trends."""
        month = time.strftime("%Y-%m")
        current_day = int(time.strftime("%d"))

        monthly_spend = sum(
            cost for date, cost in self.tracker.daily_totals.items()
            if date.startswith(month)
        )

        avg_daily = monthly_spend / current_day
        days_in_month = 30  # Approximate
        projected = avg_daily * days_in_month

        print(f"Monthly projection:")
        print(f"  Current spend (day {current_day}): ${monthly_spend:.2f}")
        print(f"  Projected end-of-month: ${projected:.2f}")
        print(f"  Budget: ${self.monthly_budget}")

        if projected > self.monthly_budget:
            overage = projected - self.monthly_budget
            print(f"  WARNING: Projected overage: ${overage:.2f}")

        return projected

# Usage
budget_mgr = BudgetManager(daily_budget=100, monthly_budget=2000)

# Check periodically
budget_mgr.check_budget()
budget_mgr.project_monthly_cost()
```

Complete Example: Optimized LLM Application

```
import asyncio
from transformers import AutoModelForCausalLM, AutoTokenizer
import tiktoken
from redis import Redis
import time

class OptimizedLLMApp:
```

```python
def __init__(self):
    # Components
    self.cache = ResponseCache(ttl=3600)
    self.batcher = DynamicBatcher(batch_size=32, max_wait_ms=100)
    self.tracker = LLMCostTracker()
    self.budget_mgr = BudgetManager(daily_budget=50, monthly_budget=1000)

    # Models
    self.cheap_model = "gpt-3.5-turbo"
    self.expensive_model = "gpt-4"

async def process_query(self, query):
    """Process query with all optimizations."""
    start_time = time.time()

    # 1. Check cache first
    cached = self.cache.get(query)
    if cached:
        latency = (time.time() - start_time) * 1000
        # Log cache hit (zero cost)
        self.tracker.log_request("cache", 0, 0, latency)
        return cached["response"]

    # 2. Route to appropriate model
    model = self.route_query(query)

    # 3. Optimize context
    optimized_query = self.optimize_context(query)

    # 4. Count tokens
    input_tokens = count_tokens(optimized_query, model)

    # 5. Generate response (batched)
    response = await self.batcher.add_request(optimized_query)
    output_tokens = count_tokens(response, model)

    # 6. Cache response
    cost = self.calculate_cost(model, input_tokens, output_tokens)
    self.cache.set(query, response, cost)

    # 7. Log metrics
    latency = (time.time() - start_time) * 1000
    self.tracker.log_request(model, input_tokens, output_tokens, latency)

    # 8. Check budget
    self.budget_mgr.check_budget()

    return response

def route_query(self, query):
    """Route to cheap or expensive model based on complexity."""
    complexity = calculate_complexity(query)
    return self.expensive_model if complexity > 0.7 else self.cheap_model

def optimize_context(self, query):
```

```
        """Optimize query to reduce tokens."""
        # Truncate if too long
        max_tokens = 2000
        if count_tokens(query) > max_tokens:
            query = truncate_to_token_limit(query, max_tokens)

        # Remove redundant whitespace
        query = " ".join(query.split())

        return query

    def calculate_cost(self, model, input_tokens, output_tokens):
        """Calculate cost for request."""
        pricing = {
            "gpt-4": {"input": 0.03, "output": 0.06},
            "gpt-3.5-turbo": {"input": 0.001, "output": 0.002}
        }

        input_cost = (input_tokens / 1000) * pricing[model]["input"]
        output_cost = (output_tokens / 1000) * pricing[model]["output"]

        return input_cost + output_cost

    async def process_batch(self, queries):
        """Process multiple queries efficiently."""
        tasks = [self.process_query(q) for q in queries]
        return await asyncio.gather(*tasks)

    def get_stats(self):
        """Get comprehensive statistics."""
        print("="*60)
        print("PERFORMANCE METRICS")
        print("="*60)

        # Cost summary
        self.tracker.get_summary(last_n_days=7)

        # Cache stats
        cache_stats = self.cache.get_stats()
        print(f"\nCache performance:")
        print(f"  Hit rate: {cache_stats['hit_rate']:.1f}%")
        print(f"  Cost saved: ${cache_stats['cost_saved']:.2f}")

        # Budget status
        self.budget_mgr.project_monthly_cost()

# Usage
app = OptimizedLLMApp()

# Process queries
queries = ["What is Python?"] * 100  # Many duplicate queries
results = asyncio.run(app.process_batch(queries))

# Get statistics
app.get_stats()
```

```
# Expected output:
# - First query: Expensive (cache miss)
# - Remaining 99: Free (cache hits)
# - High cache hit rate (99%)
# - Low total cost
# - Good throughput due to batching
```

Key Takeaways

Token Economics:

- Count every token - they all cost money
- 1 word ≈ 1.3 tokens average
- Input and output tokens priced separately
- Small optimizations × millions of requests = huge savings

Context Management:

- Sliding window: Keep recent history only
- Summarization: Compress old history
- Semantic search: Retrieve relevant context
- Can save 90%+ on context costs

GPU Utilization:

- Batch processing increases utilization
- Higher utilization = lower cost per query
- Can achieve 10-30× speedup with batching
- Monitor and optimize continuously

Cost-Performance Tradeoffs:

- Model routing: Use cheap models when possible
- Caching: Avoid redundant computation
- Quality vs cost: Monitor both
- Latency vs throughput: Balance based on use case

Monitoring:

- Track costs in real-time
- Set budgets and alerts

Project monthly spending
Optimize based on data

Pre-Read: AutoML for LLMs

The 17-Day Disaster

Monday, Week 1 - AI Startup "FastLearn" Office, Bangalore

"We need to fine-tune BERT for customer support classification," announced Priya, the ML lead. "I'll start tuning hyperparameters."

Her manager nodded. "How long will it take?"

"Give me three days."

Three days became seventeen.

The Manual Tuning Nightmare

Day 1: The Grid Search

Priya started with the classic approach: grid search.

```
# Manual grid search parameters
learning_rates = [1e-5, 3e-5, 5e-5]
batch_sizes = [8, 16, 32]
epochs = [3, 5, 7]
warmup_steps = [0, 100, 500]

# Total combinations
total = 3 × 3 × 3 × 3 = 81 experiments
```

```
# Each experiment: 2 hours on GPU
total_time = 81 × 2 = 162 hours = 6.75 days
```

"Six days just for one grid search?" her manager was shocked.

"And that's if nothing crashes," Priya sighed.

Day 3: The First Results

After running 24 experiments (3 days), Priya had her first results:

```
Experiment 1: lr=1e-5, bs=8,  epochs=3 → F1: 0.73
Experiment 2: lr=1e-5, bs=8,  epochs=5 → F1: 0.75
Experiment 3: lr=1e-5, bs=8,  epochs=7 → F1: 0.76
...
Experiment 24: lr=3e-5, bs=16, epochs=5 → F1: 0.82
```

Problem #1: Most runs were wasted

> 18 out of 24 experiments had F1 < 0.75
> Only needed to explore around the good regions
> But grid search explored everywhere equally

Problem #2: Training to completion was wasteful

```
Experiment 15:
- Epoch 1: Val Loss = 0.45
- Epoch 2: Val Loss = 0.48 ↑ (increasing!)
- Epoch 3: Val Loss = 0.52 ↑ (still increasing!)
- Epoch 4: Val Loss = 0.54 ↑ (waste of time)
- Epoch 5: Val Loss = 0.58 ↑ (should have stopped!)

Wasted: 3 hours of GPU time
```

Problem #3: Search space was too coarse

```
Best so far: lr=3e-5, F1=0.82

But what about:
- lr=2.5e-5? (not tested)
- lr=3.2e-5? (not tested)
- lr=3.8e-5? (not tested)

Maybe optimal is lr=3.3e-5, F1=0.88?
We'll never know with grid search!
```

Day 7: The Realization

After a week of experiments, Priya made a discovery:

```
# Best result after 7 days
best_result = {
    'learning_rate': 3e-5,
    'batch_size': 16,
    'epochs': 5,
    'warmup_steps': 100,
    'f1_score': 0.82
}

# But she realized:
# 1. Most experiments (60%) were clearly bad by epoch 1
# 2. Good parameter regions were clustered
# 3. Grid search wasted time exploring bad regions
# 4. No way to explore between grid points
```

The Cost So Far:

```
Days elapsed: 7
Experiments run: 56
GPU hours used: 112
GPU cost ($2/hour): $224
Results: Mediocre (F1: 0.82)
Team morale: Low
```

The Expert Arrives

Day 8: Consultation with Dr. Mehta

The company hired Dr. Mehta, an AutoML expert. She reviewed Priya's work.

"You're doing this the hard way," Dr. Mehta said. "Let me show you three techniques that would have saved you 80% of the time."

Technique 1: Bayesian Optimization

"Instead of blindly searching everywhere, learn from past experiments."

```python
# Traditional Grid Search (what Priya did)
def grid_search():
    for lr in [1e-5, 3e-5, 5e-5]:
        for bs in [8, 16, 32]:
            for epochs in [3, 5, 7]:
                # Try ALL combinations equally
                # No learning from past results
                result = train(lr, bs, epochs)

# Bayesian Optimization (smart approach)
def bayesian_optimization():
    # Start with a few random experiments
    results = []

    for i in range(5):
        # Random initial samples
        params = sample_random()
        result = train(params)
        results.append((params, result))

    # Now learn from what worked
    for i in range(20):
        # Build a model of parameter → performance
        model = build_surrogate_model(results)

        # Find most promising parameters
        # Balance: explore new areas vs exploit good areas
        next_params = suggest_next(model)

        # Try them
        result = train(next_params)
        results.append((next_params, result))

    return best_from(results)
```

The Key Insight:

```
Grid Search:
- Experiment 1: lr=1e-5 → F1=0.73
- Experiment 2: lr=1e-5 → F1=0.75 (learns nothing)
- Experiment 3: lr=1e-5 → F1=0.76 (learns nothing)
- Experiment 4: lr=5e-5 → F1=0.65 (learns nothing)

Bayesian Optimization:
- Experiment 1: lr=1e-5 → F1=0.73
- Learns: "Low learning rates seem okay"
- Experiment 2: lr=5e-5 → F1=0.65
- Learns: "Very high LR is bad, sweet spot is in middle"
- Experiment 3: lr=3.2e-5 → F1=0.85 (smart guess!)
- Learns: "Around 3e-5 is great, explore nearby"
- Experiment 4: lr=3.5e-5 → F1=0.87 (even better!)
```

Result: Found better parameters in 25 experiments instead of 81

Technique 2: Early Stopping

"Don't waste time on experiments that are clearly failing."

```
# Without Early Stopping (what Priya did)
def train_to_completion(params):
    for epoch in range(7):  # Always train 7 epochs
        train_loss = train_epoch()
        val_loss = validate()

        # Even if val_loss is increasing, keep going!
        # Waste of time!

    return final_result

# With Early Stopping (smart approach)
def train_with_early_stopping(params):
    best_val_loss = float('inf')
    patience = 2  # Stop if no improvement for 2 epochs
    patience_counter = 0

    for epoch in range(7):
        train_loss = train_epoch()
        val_loss = validate()

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            save_checkpoint()  # Save best model
            patience_counter = 0
        else:
            patience_counter += 1

        # Stop early if not improving
        if patience_counter >= patience:
            print(f"Stopped at epoch {epoch}")
            break  # Stop wasting time!

    load_best_checkpoint()  # Use best, not last
    return evaluate()
```

The Impact:

```
Bad Experiment (High LR = 1e-4):
- Epoch 1: Val Loss = 0.65
- Epoch 2: Val Loss = 0.72 ↑
- Epoch 3: Val Loss = 0.79 ↑
- [EARLY STOP] Saved 4 epochs = 1.5 hours
```

```
Good Experiment (LR = 3e-5):
- Epoch 1: Val Loss = 0.45
- Epoch 2: Val Loss = 0.38 ↓
- Epoch 3: Val Loss = 0.35 ↓
- Epoch 4: Val Loss = 0.34 ↓
- Epoch 5: Val Loss = 0.34 (no improvement)
- Epoch 6: Val Loss = 0.35 ↑
- [EARLY STOP] Saved 1 epoch

Average time saved: 40%
```

Technique 3: Smart Search Space Design

"Define your search space intelligently."

```python
# Poor Search Space (linear scales)
poor_search_space = {
    'learning_rate': [0.00001, 0.00003, 0.00005],  # Linear
    'batch_size': [8, 16, 32],  # Linear
    'dropout': [0.1, 0.2, 0.3],  # Linear
}

# Smart Search Space (appropriate scales)
smart_search_space = {
    # Learning rate: LOG SCALE (values span orders of magnitude)
    'learning_rate': LogUniform(1e-6, 1e-4),
    # Reason: 1e-5 vs 2e-5 is more important than 0.00005 vs 0.00006

    # Batch size: POWERS OF 2 (hardware efficiency)
    'batch_size': Choice([8, 16, 32, 64]),
    # Reason: GPUs process powers of 2 efficiently

    # Dropout: LINEAR SCALE (small range, small effects)
    'dropout': Uniform(0.1, 0.5),
    # Reason: 0.2 vs 0.3 is meaningful difference

    # Warmup: CONDITIONAL (only if using warmup)
    'warmup_ratio': Uniform(0.0, 0.1) if use_warmup else None,
    # Reason: Don't waste time tuning unused parameters
}
```

Why This Matters:

```
Learning Rate Search:

Poor (Linear): [1e-5, 3e-5, 5e-5]
- Misses 2e-5 (might be optimal!)
- Misses 1e-6 (might be better for stability)
- Only covers narrow range
```

```
Smart (Log): Sample from 10^U(-6, -4)
- Can find 1.7e-5 (optimal!)
- Can find 8.2e-6 (even more stable)
- Covers wide range efficiently
```

The Results

Day 10: After Implementing AutoML

Dr. Mehta helped Priya implement the three techniques:

```python
from optuna import create_study

def objective(trial):
    # Smart search space
    params = {
        'learning_rate': trial.suggest_loguniform('lr', 1e-6, 1e-4),
        'batch_size': trial.suggest_categorical('bs', [8, 16, 32]),
        'warmup_ratio': trial.suggest_uniform('warmup', 0.0, 0.1),
        'dropout': trial.suggest_uniform('dropout', 0.1, 0.5),
    }

    # Train with early stopping
    score = train_with_early_stopping(params)

    return score

# Bayesian optimization
study = create_study(direction='maximize')
study.optimize(objective, n_trials=25)

print(f"Best F1: {study.best_value}")
print(f"Best params: {study.best_params}")
```

Results:

```
After 25 experiments (vs 81 grid search):
- Best F1: 0.91 (vs 0.82 with grid search)
- Time: 2.5 days (vs 17 days planned)
- GPU hours: 35 (vs 162 planned)
- Cost: $70 (vs $324 planned)

Improvement:
- Better model: +11% F1
- 85% faster
- 78% cheaper
```

```
- Found lr=2.7e-5 (between grid points!)
```

The Three Principles

Day 11: Team Training

Dr. Mehta taught the team three principles:

Principle 1: Learn from Past Experiments

```
Bayesian Optimization = Smart Guessing

Instead of:
"Let me try ALL possible combinations"

Do:
"Let me try a few, learn what works, then explore promising regions"

It's like:
- Tasting a few dishes at a buffet
- Noticing you like spicy food
- Focusing on spicy dishes
- Not wasting time on sweet dishes
```

Principle 2: Stop Wasting Time on Failures

```
Early Stopping = Cut Your Losses

Instead of:
"Train every experiment for 7 epochs, even if failing"

Do:
"Stop training when validation loss stops improving"

It's like:
- Watching a movie
- Realizing it's bad after 15 minutes
- Leaving the theater
- Not wasting 90 more minutes
```

Principle 3: Design Search Spaces Wisely

```
Smart Search Space = Use the Right Scale

Instead of:
"Search learning rates linearly: 0.00001, 0.00003, 0.00005"

Do:
"Search logarithmically: 10^U(-6, -4)"

It's like:
- Searching for an apartment
- Using price ranges: <10L, 10-50L, 50L-1Cr, >1Cr
- Not: 10L, 10.1L, 10.2L, 10.3L (too granular)
- Not: 10L, 50L, 1Cr (too coarse)
```

The Outcome

Day 12: Production Deployment

```python
# Final production model
best_params = {
    'learning_rate': 2.7e-5,  # Found by Bayesian opt
    'batch_size': 16,
    'epochs': 4,  # Stopped early
    'warmup_ratio': 0.06,
    'dropout': 0.3
}

results = {
    'f1_score': 0.91,
    'precision': 0.89,
    'recall': 0.93,
    'training_time': '2.5 days',
    'cost': '$70',
    'experiments_run': 25
}

# vs Manual approach would have been:
manual_results = {
    'f1_score': 0.82,
    'training_time': '17 days',
    'cost': '$324',
    'experiments_run': 81
}

print("Improvement:")
print(f"Quality: +{(0.91-0.82)/0.82*100:.1f}%")
print(f"Speed: {17/2.5:.1f}x faster")
print(f"Cost: {324/70:.1f}x cheaper")
```

Output:

```
Improvement:
Quality: +11.0%
Speed: 6.8x faster
Cost: 4.6x cheaper
```

What You'll Learn

In the upcoming session, you'll master the techniques that saved FastLearn 17 days:

Bayesian Optimization

- Gaussian Process models
- Acquisition functions (EI, UCB, PI)
- Exploration vs exploitation
- Sequential model-based optimization
- Libraries: Optuna, Hyperopt, Ray Tune

Early Stopping Strategies

- Validation-based stopping
- Patience parameters
- Checkpoint management
- Overfitting prevention
- When to stop, when to continue

Search Space Design

- Log-scale for learning rates
- Categorical vs continuous parameters
- Conditional parameters
- Hardware-aware choices (batch sizes)
- Domain knowledge integration

Practical Implementation

- Setting up Optuna studies
- Defining objective functions
- Pruning unpromising trials

Visualization and analysis
Production deployment

Remember the Lessons:

"Grid search is brute force. Bayesian optimization is smart force."

"Every failed epoch is wasted money. Early stopping is profit."

"The right scale matters: log for learning rates, linear for dropout."

"25 smart experiments beat 81 random ones."

Ready to automate hyperparameter tuning and save weeks of manual work? Let's begin.

AutoML for LLMs

PPT File: Click Here

Colab File: Click Here

Learning Objectives

By the end of this session, you will:

1. Understand Bayesian optimization for hyperparameter tuning
2. Implement early stopping to save training time
3. Design effective search spaces
4. Use Optuna for automated hyperparameter tuning
5. Deploy AutoML in production LLM fine-tuning

Part 1: Why AutoML for LLMs? (10 minutes)

The Hyperparameter Tuning Problem

LLM Fine-tuning has many hyperparameters:

```
hyperparameters = {
    'learning_rate': ???,      # How fast to learn
    'batch_size': ???,         # How many samples per update
    'epochs': ???,             # How long to train
    'warmup_steps': ???,       # Learning rate warmup
    'weight_decay': ???,       # Regularization
    'max_grad_norm': ???,      # Gradient clipping
    'dropout': ???,            # Dropout rate
    'adam_epsilon': ???,       # Optimizer epsilon
    'num_train_epochs': ???,   # Training duration
}

# How do you find the best combination?
```

Traditional Approaches and Their Problems

1. Manual Tuning

```
Problem: Slow, requires expertise, not reproducible
Time: Days to weeks
Cost: High (expert time + GPU time)
Result: Suboptimal (limited exploration)
```

2. Grid Search

```
# Try all combinations
learning_rates = [1e-5, 3e-5, 5e-5]
batch_sizes = [8, 16, 32]
epochs = [3, 5, 7]

total_experiments = 3 × 3 × 3 = 27
time_per_experiment = 2 hours
total_time = 54 hours

Problem: Combinatorial explosion
- 4 parameters with 3 values each = 81 experiments
- 5 parameters with 4 values each = 1,024 experiments
```

3. Random Search

```
# Random sampling
for i in range(50):
    lr = random.uniform(1e-6, 1e-4)
    bs = random.choice([8, 16, 32])
    epochs = random.randint(3, 7)

    score = train(lr, bs, epochs)
```

```
Problem: No learning from past results
- Experiment 1: lr=1e-5 → good
- Experiment 2: lr=8e-5 → bad
- Experiment 3: lr=2e-5 → ??? (doesn't use info from 1&2)
```

The AutoML Solution

Key Insight: Learn from past experiments to guide future ones

```
Traditional: Try everything equally
AutoML: Learn what works, focus there

Traditional: Complete every experiment
AutoML: Stop bad experiments early

Traditional: Fixed grid of values
AutoML: Explore continuous space
```

Part 2: Bayesian Optimization (30 minutes)

Core Concept

Bayesian Optimization = Smart Sequential Search

```
1. Build a probabilistic model of hyperparameter → performance
2. Use model to decide which hyperparameters to try next
3. Balance: Explore (try new areas) vs Exploit (improve known good areas)
4. Update model with new results
5. Repeat
```

The Mathematics (Simplified)

Gaussian Process (GP): The Surrogate Model

```
# Conceptually:
# GP learns: f(hyperparameters) → performance

# Given past results:
past_experiments = [
    ({'lr': 1e-5, 'bs': 16}, 0.73),  # (params, f1_score)
    ({'lr': 3e-5, 'bs': 16}, 0.82),
    ({'lr': 5e-5, 'bs': 16}, 0.65),
]

# GP predicts for new params:
new_params = {'lr': 2e-5, 'bs': 16}

# Prediction:
mean = 0.78      # Expected performance
uncertainty = 0.05  # Confidence interval

# Interpretation:
# "I think lr=2e-5 will give F1=0.78±0.05"
# "I'm fairly confident because it's between tested values"
```

Visualization:

```
F1 Score
   |
0.9|                    /---\
0.8|               /----/     \
0.7|         /----/            \___
0.6|  /----/
   |_____________________________________
      1e-5   2e-5  3e-5  4e-5  5e-5   Learning Rate
      ↑             ↑          ↑
    tested       predict    tested

GP learns this curve from just 3 points!
Then predicts what's likely at untested points.
```

Acquisition Functions

How to choose next experiment?

1. Expected Improvement (EI)

```
def expected_improvement(params):
    """How much better than current best?"""
    current_best = 0.82
```

```
    predicted_mean, predicted_std = gp.predict(params)

    # How likely to beat current best?
    improvement = max(0, predicted_mean - current_best)

    # Weight by uncertainty
    # High uncertainty → high EI (exploration)
    # Low uncertainty + high mean → high EI (exploitation)

    return improvement * predicted_std

# Try params with highest EI
next_params = maximize(expected_improvement)
```

2. Upper Confidence Bound (UCB)

```
def upper_confidence_bound(params, kappa=2.0):
    """Optimistic estimate"""
    predicted_mean, predicted_std = gp.predict(params)

    # Optimistic: mean + uncertainty
    # kappa controls exploration (higher = more exploration)
    ucb = predicted_mean + kappa * predicted_std

    return ucb

# Try params with highest UCB
next_params = maximize(upper_confidence_bound)
```

3. Probability of Improvement (PI)

```
def probability_of_improvement(params):
    """Probability of beating current best"""
    current_best = 0.82
    predicted_mean, predicted_std = gp.predict(params)

    # Calculate probability
    z = (predicted_mean - current_best) / predicted_std
    pi = norm.cdf(z)  # Cumulative distribution function

    return pi

# Try params with highest PI
next_params = maximize(probability_of_improvement)
```

Comparison:

```
Acquisition Function | Exploration | Exploitation | Use Case
---------------------|-------------|--------------|----------
EI (Expected Imp.)   | Balanced    | Balanced     | General purpose
```

```
UCB                  | High         | Medium       | Large search space
PI                   | Low          | High         | Exploit known good region
```

Bayesian Optimization Algorithm

```python
def bayesian_optimization(
    objective_function,
    search_space,
    n_initial=5,
    n_iterations=20
):
    """Complete Bayesian optimization loop"""

    results = []

    # Step 1: Random initialization
    print("Phase 1: Random Exploration")
    for i in range(n_initial):
        params = sample_random(search_space)
        score = objective_function(params)
        results.append((params, score))
        print(f"  Trial {i}: {params} → {score:.3f}")

    # Step 2: Bayesian optimization
    print("\nPhase 2: Bayesian Optimization")
    for i in range(n_iterations):
        # Build GP model from past results
        gp_model = fit_gaussian_process(results)

        # Find next best parameters using acquisition function
        next_params = optimize_acquisition(gp_model, search_space)

        # Evaluate
        score = objective_function(next_params)
        results.append((next_params, score))

        # Track best so far
        best_score = max(r[1] for r in results)
        print(f"  Trial {n_initial+i}: {next_params} → {score:.3f} (best:
{best_score:.3f})")

    # Return best
    best_params, best_score = max(results, key=lambda x: x[1])
    return best_params, best_score

# Example usage
def train_model(params):
    """Objective function: train and evaluate"""
    model = build_model(params)
    model.train()
    f1 = model.evaluate()
    return f1
```

```
search_space = {
    'learning_rate': (1e-6, 1e-4, 'log'),
    'batch_size': [8, 16, 32, 64],
    'dropout': (0.1, 0.5, 'uniform')
}

best_params, best_score = bayesian_optimization(
    train_model,
    search_space,
    n_initial=5,
    n_iterations=20
)
```

Practical Implementation with Optuna

Optuna: Production-Ready Bayesian Optimization

```
import optuna
from transformers import AutoModelForSequenceClassification, Trainer

def objective(trial):
    """Optuna objective function"""

    # Define search space
    params = {
        'learning_rate': trial.suggest_loguniform('lr', 1e-6, 1e-4),
        'per_device_train_batch_size': trial.suggest_categorical('batch_size', [8, 16,
32]),
        'num_train_epochs': trial.suggest_int('epochs', 3, 7),
        'warmup_ratio': trial.suggest_uniform('warmup', 0.0, 0.1),
        'weight_decay': trial.suggest_loguniform('weight_decay', 1e-5, 1e-1),
    }

    # Train model
    model = AutoModelForSequenceClassification.from_pretrained('bert-base-uncased')

    training_args = TrainingArguments(
        output_dir='./results',
        **params,
        evaluation_strategy='epoch',
        save_strategy='epoch',
        load_best_model_at_end=True,
    )

    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_dataset,
        eval_dataset=eval_dataset,
    )
```

```
    # Train
    trainer.train()

    # Evaluate
    metrics = trainer.evaluate()
    f1_score = metrics['eval_f1']

    return f1_score

# Create study
study = optuna.create_study(
    direction='maximize',  # Maximize F1 score
    sampler=optuna.samplers.TPESampler(),  # Bayesian optimization
)

# Optimize
study.optimize(objective, n_trials=25)

# Results
print(f"Best F1: {study.best_value:.3f}")
print(f"Best params: {study.best_params}")
```

Part 3: Early Stopping (20 minutes)

Why Early Stopping?

Problem: Overfitting and Wasted Time

```
# Without early stopping
for epoch in range(10):
    train_loss = train_epoch()
    val_loss = validate()

    print(f"Epoch {epoch}: Train={train_loss:.3f}, Val={val_loss:.3f}")

# Output:
# Epoch 0: Train=0.500, Val=0.450
# Epoch 1: Train=0.400, Val=0.380
# Epoch 2: Train=0.320, Val=0.330  ← Best validation
# Epoch 3: Train=0.260, Val=0.350  ← Overfitting starts
# Epoch 4: Train=0.210, Val=0.380
# Epoch 5: Train=0.180, Val=0.410  ← Wasted 3 epochs!
# ...
# Epoch 9: Train=0.050, Val=0.550  ← Severely overfitted

# Problems:
# 1. Wasted 7 epochs of training time
```

```
# 2. Final model is worse than epoch 2
# 3. Need to remember to use epoch 2 checkpoint
```

Early Stopping Mechanics

Basic Algorithm:

```
class EarlyStopping:
    def __init__(self, patience=3, min_delta=0.001):
        """
        patience: How many epochs to wait for improvement
        min_delta: Minimum change to count as improvement
        """
        self.patience = patience
        self.min_delta = min_delta
        self.best_loss = float('inf')
        self.counter = 0
        self.best_model = None

    def __call__(self, val_loss, model):
        """Check if should stop training"""

        # Is this better than best so far?
        if val_loss < self.best_loss - self.min_delta:
            # Yes! New best
            self.best_loss = val_loss
            self.counter = 0
            self.best_model = model.state_dict().copy()
            return False  # Don't stop
        else:
            # No improvement
            self.counter += 1

            if self.counter >= self.patience:
                # Waited long enough, stop
                print(f"Early stopping triggered after {self.counter} epochs without
improvement")
                return True  # Stop training

            return False  # Keep going

# Usage
early_stopping = EarlyStopping(patience=3)

for epoch in range(100):  # Max epochs
    train_loss = train_epoch()
    val_loss = validate()

    if early_stopping(val_loss, model):
        print(f"Stopped at epoch {epoch}")
        break
```

```
# Load best model
model.load_state_dict(early_stopping.best_model)
```

Advanced Early Stopping Strategies

1. Relative Improvement

```
class RelativeEarlyStopping:
    def __init__(self, patience=3, min_relative_delta=0.01):
        """Stop if improvement < 1% of current loss"""
        self.patience = patience
        self.min_relative_delta = min_relative_delta
        self.best_loss = float('inf')
        self.counter = 0

    def __call__(self, val_loss, model):
        relative_improvement = (self.best_loss - val_loss) / self.best_loss

        if relative_improvement > self.min_relative_delta:
            # Significant relative improvement
            self.best_loss = val_loss
            self.counter = 0
            return False
        else:
            self.counter += 1
            return self.counter >= self.patience
```

2. Adaptive Patience

```
class AdaptiveEarlyStopping:
    def __init__(self, initial_patience=3, max_patience=10):
        """Increase patience if showing slow improvement"""
        self.patience = initial_patience
        self.max_patience = max_patience
        self.best_loss = float('inf')
        self.counter = 0
        self.improvement_history = []

    def __call__(self, val_loss, model):
        improvement = self.best_loss - val_loss
        self.improvement_history.append(improvement)

        if improvement > 0:
            # Getting better
            self.best_loss = val_loss
            self.counter = 0

            # If consistent small improvements, increase patience
            if len(self.improvement_history) >= 3:
                avg_improvement = sum(self.improvement_history[-3:]) / 3
                if avg_improvement > 0 and avg_improvement < 0.01:
```

```
                    self.patience = min(self.patience + 1, self.max_patience)

            return False
        else:
            self.counter += 1
            return self.counter >= self.patience
```

3. Metric-Specific Early Stopping

```
class MultiMetricEarlyStopping:
    """Stop only if ALL metrics stop improving"""

    def __init__(self, metrics=['loss', 'f1', 'accuracy'], patience=3):
        self.patience = patience
        self.best_values = {m: float('-inf') if m != 'loss' else float('inf')
                            for m in metrics}
        self.counters = {m: 0 for m in metrics}

    def __call__(self, metrics_dict, model):
        any_improved = False

        for metric, value in metrics_dict.items():
            is_loss = (metric == 'loss')
            improved = (value < self.best_values[metric]) if is_loss else (value >
self.best_values[metric])

            if improved:
                self.best_values[metric] = value
                self.counters[metric] = 0
                any_improved = True
            else:
                self.counters[metric] += 1

        # Stop only if ALL metrics haven't improved
        all_stagnant = all(c >= self.patience for c in self.counters.values())

        return all_stagnant

# Usage
early_stopping = MultiMetricEarlyStopping(['loss', 'f1'], patience=3)

for epoch in range(100):
    train()
    metrics = evaluate()  # {'loss': 0.35, 'f1': 0.82}

    if early_stopping(metrics, model):
        break
```

Integration with Optuna (Pruning)

Pruning = Early stopping for hyperparameter trials

```python
import optuna

def objective(trial):
    params = {
        'learning_rate': trial.suggest_loguniform('lr', 1e-6, 1e-4),
        'batch_size': trial.suggest_categorical('bs', [8, 16, 32]),
    }

    model = create_model(params)

    for epoch in range(10):
        train_loss = train_epoch(model)
        val_loss = validate(model)

        # Report intermediate value to Optuna
        trial.report(val_loss, epoch)

        # Optuna decides: Should we stop this trial early?
        # If val_loss is worse than other trials at this epoch, prune it
        if trial.should_prune():
            raise optuna.TrialPruned()

    return final_score

# Use median pruner
study = optuna.create_study(
    pruner=optuna.pruners.MedianPruner(
        n_startup_trials=5,    # Don't prune first 5 trials
        n_warmup_steps=2,      # Don't prune before epoch 2
    )
)

study.optimize(objective, n_trials=50)

# Many trials will be pruned early, saving time!
```

Pruning Example:

```
Trial 1: lr=1e-5
  Epoch 0: val_loss=0.45
  Epoch 1: val_loss=0.38
  Epoch 2: val_loss=0.35
  ... continues

Trial 2: lr=1e-3 (too high!)
  Epoch 0: val_loss=0.65
  Epoch 1: val_loss=0.72  ← Worse than trial 1
  [PRUNED] Saved 8 epochs!

Trial 3: lr=3e-5
  Epoch 0: val_loss=0.42
  Epoch 1: val_loss=0.36  ← Better than trial 1
  ... continues
```

Part 4: Search Space Design (20 minutes)

The Importance of Scale

Different hyperparameters need different scales

1. Learning Rate: LOG SCALE

```
# Wrong: Linear scale
learning_rates = [0.00001, 0.00003, 0.00005]

# Problem:
# - Difference between 1e-5 and 3e-5 is 2e-5 (200%)
# - Difference between 3e-5 and 5e-5 is 2e-5 (67%)
# - Same absolute difference, very different relative effect!

# Right: Log scale
learning_rate = trial.suggest_loguniform('lr', 1e-6, 1e-4)
# Samples: 2.3e-6, 1.7e-5, 4.8e-5, 8.2e-5
# Equal spacing in log space = proportional spacing in linear space

# Why it matters:
# Learning rate effect is multiplicative, not additive
# 2x learning rate ≈ 2x faster learning
# So we should search in multiplicative (log) space
```

Visualization:

```
Linear Scale:
|-------|-------|-------|-------|
1e-5   2e-5   3e-5   4e-5   5e-5
  ↓       ↓       ↓       ↓
small  small  medium large  (uneven effects)

Log Scale:
|------------|------------|------------|
1e-6        1e-5        1e-4
   ↓           ↓           ↓
 small      medium      large  (even effects)
```

2. Batch Size: POWERS OF 2

```
# Wrong: Arbitrary values
```

```
batch_sizes = [10, 20, 30, 40]

# Right: Powers of 2
batch_size = trial.suggest_categorical('bs', [8, 16, 32, 64, 128])

# Why?
# - GPU memory organized in powers of 2
# - Better hardware utilization
# - More efficient computation
# - Industry standard

# Example:
# batch_size=32 → 100% GPU utilization
# batch_size=30 → 94% GPU utilization (waste)
```

3. Dropout: LINEAR SCALE

```
# Right: Linear scale
dropout = trial.suggest_uniform('dropout', 0.1, 0.5)

# Why?
# - Dropout is a probability (0 to 1)
# - Effects are roughly linear
# - 0.2 vs 0.3 has similar impact as 0.3 vs 0.4
# - Small range (0.1 to 0.5), so linear is fine
```

Search Space Types

1. Continuous Parameters

```
# Uniform (linear)
dropout = trial.suggest_uniform('dropout', 0.1, 0.5)
# Samples evenly: 0.15, 0.27, 0.33, 0.44

# Log-uniform (logarithmic)
learning_rate = trial.suggest_loguniform('lr', 1e-6, 1e-4)
# Samples evenly in log space: 2.3e-6, 1.7e-5, 4.8e-5

# Discrete uniform
num_layers = trial.suggest_int('layers', 2, 6)
# Samples: 2, 3, 4, 5, 6
```

2. Categorical Parameters

```
# Discrete choices
optimizer = trial.suggest_categorical('optimizer', ['adam', 'sgd', 'adamw'])
batch_size = trial.suggest_categorical('bs', [8, 16, 32, 64])
activation = trial.suggest_categorical('activation', ['relu', 'gelu', 'tanh'])
```

3. Conditional Parameters

```
def objective(trial):
    # Main parameter
    use_warmup = trial.suggest_categorical('use_warmup', [True, False])

    # Conditional parameter (only used if use_warmup=True)
    if use_warmup:
        warmup_ratio = trial.suggest_uniform('warmup_ratio', 0.0, 0.1)
    else:
        warmup_ratio = 0.0

    # Another conditional
    optimizer = trial.suggest_categorical('optimizer', ['adam', 'sgd'])

    if optimizer == 'adam':
        beta1 = trial.suggest_uniform('beta1', 0.8, 0.95)
        beta2 = trial.suggest_uniform('beta2', 0.95, 0.999)
    else:
        # SGD doesn't have betas
        beta1, beta2 = None, None

    # Use parameters
    params = {
        'optimizer': optimizer,
        'warmup_ratio': warmup_ratio,
        'beta1': beta1,
        'beta2': beta2,
    }

    return train(params)
```

Complete Search Space Example

```
def define_search_space(trial):
    """Production-ready LLM fine-tuning search space"""

    search_space = {
        # Learning rate: LOG SCALE (most important!)
        'learning_rate': trial.suggest_loguniform('lr', 1e-6, 1e-4),

        # Batch size: POWERS OF 2
        'per_device_train_batch_size': trial.suggest_categorical(
            'batch_size', [8, 16, 32, 64]
        ),

        # Epochs: INTEGER
        'num_train_epochs': trial.suggest_int('epochs', 3, 7),

        # Warmup: LINEAR (small range)
        'warmup_ratio': trial.suggest_uniform('warmup', 0.0, 0.1),
```

```
        # Weight decay: LOG SCALE (regularization)
        'weight_decay': trial.suggest_loguniform('wd', 1e-5, 1e-1),

        # Gradient clipping: LINEAR
        'max_grad_norm': trial.suggest_uniform('grad_clip', 0.5, 2.0),

        # Dropout: LINEAR
        'dropout': trial.suggest_uniform('dropout', 0.1, 0.5),

        # Optimizer: CATEGORICAL
        'optimizer': trial.suggest_categorical('opt', ['adam', 'adamw']),

        # Learning rate schedule: CATEGORICAL
        'lr_scheduler_type': trial.suggest_categorical(
            'scheduler', ['linear', 'cosine', 'polynomial']
        ),
    }

    # Conditional: Adam-specific parameters
    if search_space['optimizer'] in ['adam', 'adamw']:
        search_space['adam_beta1'] = trial.suggest_uniform('beta1', 0.85, 0.95)
        search_space['adam_beta2'] = trial.suggest_uniform('beta2', 0.95, 0.999)
        search_space['adam_epsilon'] = trial.suggest_loguniform('eps', 1e-9, 1e-6)

    return search_space
```

Domain Knowledge Integration

Use what we know to narrow the search space

```
# Generic search space (wasteful)
generic_space = {
    'learning_rate': (1e-8, 1.0),  # Way too wide!
    'batch_size': [1, 2, 4, 8, 16, 32, 64, 128, 256],  # Too many options
    'dropout': (0.0, 0.9),  # Includes extreme values
}

# Informed search space (better)
informed_space = {
    # LLM fine-tuning typically uses 1e-6 to 1e-4
    'learning_rate': (1e-6, 1e-4),

    # Modern GPUs work well with 8-64
    'batch_size': [8, 16, 32, 64],

    # Dropout >0.5 usually too aggressive for transformers
    'dropout': (0.1, 0.5),

    # Transformers typically train 3-7 epochs
    'epochs': (3, 7),

    # Warmup helps transformer training
```

```
    'warmup_ratio': (0.0, 0.1),  # 0-10% of training
}

# Result:
# - Generic: ~1000 possible combinations to explore
# - Informed: ~50 relevant combinations to explore
# - 20x more efficient!
```

Part 5: Complete Production Example (10 minutes)

End-to-End AutoML Pipeline

```
import optuna
from transformers import (
    AutoModelForSequenceClassification,
    AutoTokenizer,
    TrainingArguments,
    Trainer,
    EarlyStoppingCallback
)
from datasets import load_dataset

class LLMAutoMLOptimizer:
    def __init__(self, model_name, dataset):
        self.model_name = model_name
        self.dataset = dataset
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)

    def objective(self, trial):
        """Optuna objective function"""

        # 1. Define search space
        params = {
            'learning_rate': trial.suggest_loguniform('lr', 1e-6, 1e-4),
            'per_device_train_batch_size': trial.suggest_categorical(
                'batch_size', [8, 16, 32]
            ),
            'num_train_epochs': trial.suggest_int('epochs', 3, 7),
            'warmup_ratio': trial.suggest_uniform('warmup', 0.0, 0.1),
            'weight_decay': trial.suggest_loguniform('wd', 1e-5, 1e-2),
        }

        # 2. Create model
        model = AutoModelForSequenceClassification.from_pretrained(
            self.model_name,
```

```
            num_labels=2
        )

        # 3. Training arguments with early stopping
        training_args = TrainingArguments(
            output_dir=f'./results/trial_{trial.number}',
            evaluation_strategy='epoch',
            save_strategy='epoch',
            load_best_model_at_end=True,
            metric_for_best_model='f1',
            **params
        )

        # 4. Create trainer with early stopping
        trainer = Trainer(
            model=model,
            args=training_args,
            train_dataset=self.dataset['train'],
            eval_dataset=self.dataset['validation'],
            callbacks=[EarlyStoppingCallback(early_stopping_patience=2)]
        )

        # 5. Train
        trainer.train()

        # 6. Evaluate
        metrics = trainer.evaluate()

        # 7. Return metric to optimize
        return metrics['eval_f1']

    def optimize(self, n_trials=25):
        """Run optimization"""

        # Create study with pruning
        study = optuna.create_study(
            direction='maximize',
            sampler=optuna.samplers.TPESampler(),
            pruner=optuna.pruners.MedianPruner(
                n_startup_trials=5,
                n_warmup_steps=1
            )
        )

        # Optimize
        study.optimize(self.objective, n_trials=n_trials)

        # Results
        return {
            'best_params': study.best_params,
            'best_value': study.best_value,
            'best_trial': study.best_trial,
            'study': study
        }
```

```python
# Usage
optimizer = LLMAutoMLOptimizer(
    model_name='bert-base-uncased',
    dataset=load_dataset('glue', 'sst2')
)

results = optimizer.optimize(n_trials=25)

print(f"Best F1: {results['best_value']:.3f}")
print(f"Best params: {results['best_params']}")

# Visualize
import optuna.visualization as vis

# Optimization history
vis.plot_optimization_history(results['study'])

# Parameter importances
vis.plot_param_importances(results['study'])

# Hyperparameter relationships
vis.plot_parallel_coordinate(results['study'])
```

Summary & Best Practices

Key Takeaways

1. Bayesian Optimization

    Learns from past experiments
    Balances exploration vs exploitation
    5-10x more efficient than grid search
    Use Optuna/Hyperopt for production

2. Early Stopping

    Save 30-50% of training time
    Prevent overfitting
    Always use with validation set
    Combine with checkpointing

3. Search Space Design

    Log scale for: learning rate, weight decay

Powers of 2 for: batch size
Linear scale for: dropout, warmup ratio
Use domain knowledge to narrow ranges

4. Production Recommendations

Start with 5-10 random trials
Run 20-50 Bayesian optimization trials
Use early stopping (patience=2-3)
Enable pruning for bad trials
Track all experiments (MLflow, Weights & Biases)

Common Pitfalls

❌ Don't:

Use linear scale for learning rate
Search too wide ranges
Ignore early stopping
Run too few trials (<15)
Trust single run (use multiple seeds)

✓ Do:

Use log scale for multiplicative parameters
Narrow search space with domain knowledge
Always use early stopping
Run 20-50 trials minimum
Validate on held-out test set

Further Resources

1. Optuna Documentation: https://optuna.org
2. "Algorithms for Hyper-Parameter Optimization" (Bergstra et al.)
3. "Taking the Human Out of the Loop: A Review of Bayesian Optimization"
4. Ray Tune: https://docs.ray.io/en/latest/tune/

Fine-Tuning LLMs & AutoML

The Startup That Couldn't Afford GPT-4

Monday, 9:00 AM - TechStart AI Office, Mumbai

Maya stared at her laptop screen, frustrated. Her startup had just landed a major client: a customer support platform serving 50,000 users. They needed an AI chatbot that could answer product-specific FAQs accurately.

"We can't just use GPT-4 API," her CTO Raj said, walking over. "The costs would eat our entire budget. We need to fine-tune our own model."

Maya groaned. "But fine-tuning a 7-billion parameter model... that needs what, 80GB of GPU memory? We only have a single A100 with 40GB."

Raj smiled. "Have you heard of LoRA?"

The Memory Problem

Raj pulled up a diagram:

```
Traditional Fine-Tuning:
Model: 7B parameters
Full precision (FP32): 7B × 4 bytes = 28GB
Optimizer states: 28GB × 2 = 56GB
Gradients: 28GB
Total: ~112GB GPU memory

Our hardware: 40GB
Result: CRASH!
```

"That's the problem with full fine-tuning," Raj explained. "You need to update ALL parameters, store optimizer states, gradients... it's memory-intensive."

"So what's the solution?" Maya asked.

"We freeze 99.9% of the model. Only train a tiny fraction."

Enter LoRA: The Clever Trick

LoRA = Low-Rank Adaptation

Raj drew on the whiteboard:

```
Standard fine-tuning:
W (Original weights) → W' (Updated weights)
Updates all 7 billion parameters!

LoRA:
W (Frozen!) + ΔW (Trainable)
ΔW = A × B

Where:
A: (d × r) - Down-projection matrix
B: (r × d) - Up-projection matrix
r: Rank (typically 8, 16, 32, 64)

If d = 4096 and r = 16:
ΔW parameters = 4096 × 16 + 16 × 4096 = 131,072
Instead of 4096 × 4096 = 16,777,216

That's 128× fewer parameters!
```

"Wait," Maya interrupted. "You're saying instead of updating a 4096×4096 matrix, we create two small matrices that multiply together?"

"Exactly! And here's the magic: mathematically, you can approximate ANY weight update using low-rank decomposition. For most fine-tuning tasks, a rank of 16 is enough."

The Results:

```
Memory Usage:
Full fine-tuning: 112GB
LoRA fine-tuning: 14GB

Training Speed:
Full fine-tuning: 8 hours
LoRA fine-tuning: 2 hours
```

```
Model Quality:
Full fine-tuning: 94.2% accuracy
LoRA fine-tuning: 93.8% accuracy

Cost difference: 10× cheaper!
```

"We lose 0.4% accuracy but save 90% memory," Raj said. "Worth it?"

Maya nodded. "Definitely. But can we go even lower?"

QLoRA: Going Even Smaller

"That's where QLoRA comes in," Raj said, pulling up another diagram.

QLoRA = Quantized LoRA

```
The Quantization Trick:

Normal model:
- Weights stored in FP32 (32 bits per number)
- 7B parameters × 4 bytes = 28GB

4-bit quantization:
- Weights stored in 4 bits
- 7B parameters × 0.5 bytes = 3.5GB

Memory savings: 8× reduction!
```

"Wait," Maya said. "Won't 4-bit precision destroy the model quality?"

Raj grinned. "That's the genius of QLoRA. They use NF4 (NormalFloat4) - a special 4-bit format optimized for neural network weights. Plus double quantization to save even more."

QLoRA Memory Breakdown:

```
Base model (4-bit quantized): 3.5GB
LoRA adapters (FP16): 0.3GB
Optimizer states: 1.2GB
Gradients: 0.8GB
Activation memory: 4GB
Total: ~10GB

Fits on a consumer GPU!
```

"So QLoRA = LoRA + Quantization," Maya summarized. "We get the memory savings of both techniques combined."

The Hyperparameter Nightmare

Two weeks later, Maya had a new problem.

"Raj, I've run 50 training experiments. Different learning rates, LoRA ranks, batch sizes... I still can't beat 91% accuracy. There are too many hyperparameters!"

Raj pulled up a spreadsheet:

```
Hyperparameters to tune:
1. Learning rate: [1e-5, 1e-4, 1e-3]
2. LoRA rank (r): [4, 8, 16, 32, 64]
3. LoRA alpha: [8, 16, 32, 64]
4. Batch size: [4, 8, 16]
5. Number of epochs: [3, 5, 10]
6. Warmup ratio: [0.03, 0.06, 0.1]

Total combinations: 3 × 5 × 4 × 3 × 3 × 3 = 1,620

If each experiment takes 2 hours:
1,620 × 2 = 3,240 hours = 135 days!
```

"That's insane," Maya said. "There has to be a better way."

"There is," Raj said. "AutoML with Optuna."

Optuna: The Smart Search

The Traditional Approach: Grid Search

```
Try every combination:
[1e-5, 1e-4, 1e-3] × [4, 8, 16, 32, 64] × ...

Problems:
- Exhaustive (wastes compute)
- Treats all hyperparameters equally
- Doesn't learn from previous trials
```

The Optuna Approach: Bayesian Optimization

```
Smart search algorithm:

Trial 1: lr=1e-4, rank=16 → accuracy=89.5%
Trial 2: lr=1e-3, rank=8 → accuracy=87.2%
Trial 3: lr=5e-5, rank=32 → accuracy=91.3% ← Good!

Optuna learns: "Lower lr + higher rank = better"

Trial 4: lr=3e-5, rank=64 → accuracy=92.8% ← Even better!

Converges to optimal in 20-30 trials instead of 1,620!
```

Raj showed Maya the code:

```python
import optuna

def objective(trial):
    # Suggest hyperparameters
    lr = trial.suggest_float("lr", 1e-5, 1e-3, log=True)
    rank = trial.suggest_int("rank", 4, 64)
    alpha = trial.suggest_int("alpha", 8, 64)

    # Train model
    model = train_lora_model(lr=lr, rank=rank, alpha=alpha)
    accuracy = evaluate(model)

    return accuracy

# Run optimization
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=30)

print(f"Best accuracy: {study.best_value}")
print(f"Best hyperparameters: {study.best_params}")
```

"Optuna uses Tree-structured Parzen Estimator (TPE) to model the relationship between hyperparameters and performance," Raj explained. "It's like a smart assistant that learns which combinations work."

The FAQ Model: Putting It All Together

Three weeks later, Maya had results:

```
Dataset: 10,000 customer FAQ pairs
```

```
Model: LLaMA-2-7B
Method: QLoRA fine-tuning
Hyperparameter optimization: Optuna

Results after 25 Optuna trials:
Best configuration:
- Learning rate: 3.2e-5
- LoRA rank: 48
- LoRA alpha: 32
- Batch size: 8
- Epochs: 5

Final Metrics:
- FAQ accuracy: 94.7%
- Response quality: 9.2/10 (human eval)
- Inference speed: 23 tokens/sec
- Memory usage: 11GB

Cost breakdown:
- Total training time: 6 hours
- GPU cost (A100): $2.40/hour × 6 = $14.40
- Optuna trials: 25 × 15 min = 6.25 hours extra
- Total cost: ~$30

Compare to:
- GPT-4 API for 50,000 users: $15,000/month
- Full fine-tuning: Would need 8× A100s = $1,200
```

The client demo went perfectly. The chatbot answered product questions accurately, handled edge cases gracefully, and responded in under a second.

"How did you build this so fast?" the client asked.

Maya smiled. "Smart techniques. LoRA let us fine-tune efficiently. QLoRA made it fit on affordable hardware. Optuna found the best settings automatically."

The LoRA Advantage

Six months later, Maya's startup had 20 clients. Each needed custom models for their domains: legal, medical, e-commerce, HR.

The Old Way:

```
Each client:
- Full 7B model: 28GB per model
- 20 clients: 560GB storage
- Can't batch inference (memory limits)
```

The LoRA Way:

```
One base model: 28GB
20 LoRA adapters: 20 × 300MB = 6GB
Total: 34GB

Bonus: Switch adapters on-the-fly!
```

"We load one base model into GPU memory," Maya explained to investors. "Then we swap LoRA adapters in milliseconds. Client A gets legal expertise. Client B gets medical knowledge. Same base model, different adapters."

The Business Model:

```
Economics:
- Train one adapter: $30
- Deploy one adapter: 300MB storage
- Switch adapters: 50ms latency

Comparison:
- Full fine-tuning: $1,200 per client
- Separate models: 560GB storage
- Model switching: 30 seconds to reload

LoRA made multi-client AI economically viable!
```

The Key Lessons

A year later, Maya spoke at a local ML meetup:

"Three Lessons from Building Production LLMs"

1. Parameter Efficiency is Everything

```
Full fine-tuning: Update 7 billion parameters
LoRA: Train 1 million parameters
Result: 7,000× fewer parameters, 0.4% accuracy loss

ΔW = A × B (low-rank decomposition)
```

2. Quantization Democratizes AI

```
FP32: 28GB (data center only)
4-bit: 3.5GB (consumer hardware)
Quality: Minimal degradation with NF4

QLoRA = LoRA + 4-bit quantization
```

3. AutoML Beats Manual Tuning

```
Grid search: 1,620 experiments
Optuna: 25 experiments
Time saved: 98.5%

Bayesian optimization > random search > grid search
```

4. Adapters Enable Scale

```
One base model + multiple adapters
Switch in milliseconds
Multi-tenant AI made practical
```

What You'll Learn

In the upcoming session, you'll master Maya's toolkit:

- LoRA Fundamentals: How low-rank decomposition enables efficient fine-tuning
- QLoRA Mechanics: 4-bit quantization with NF4 format
- Optuna Optimization: Bayesian hyperparameter search
- FAQ Fine-Tuning: End-to-end practical implementation
- Production Deployment: Multi-adapter serving strategies

By the end, you'll be able to fine-tune a 7B model on a single consumer GPU in under $30—something that would have cost thousands just two years ago.

Remember Maya's principles:

1. "Don't update what you don't need to" (freeze the base model)
2. "Low-rank approximations are surprisingly good" (LoRA works!)
3. "4 bits are enough" (QLoRA maintains quality)
4. "Let algorithms search, not humans" (Optuna optimization)
5. "Adapters > Full models" (efficient multi-client serving)

Fine-Tuning Large Language Models (LLMs) with Real-World Intuition

PPT File: Click Here

Session Overview

Imagine you are building a customer support chatbot for a banking app. You already have a powerful general-purpose language model trained on vast internet data. However, when users ask:

"How do I reset my ATM PIN?"

The model gives vague or generic answers.

This is where fine-tuning comes in — adapting a general model into a domain expert.

This session explores:

How neural networks learn (foundation)
Why fine-tuning is needed
Different strategies: Full Fine-Tuning, LoRA, and QLoRA
Trade-offs between performance, cost, and efficiency

---

1. How Neural Networks Learn (Intuition First)

Real-World Analogy

Think of a student preparing for exams:

They attempt questions (prediction)
Compare with correct answers (error)
Adjust understanding (learning)

A neural network works the same way.

Neural Network Basics

A neural network consists of:

Layers of neurons
Connections with weights
Input → Processing → Output

Each connection has a weight, which determines how strongly one neuron influences another.

Training Objective

Goal:

Make the model's prediction as close as possible to the correct answer.

Error (Loss)

The difference between predicted output and actual output
Example:
Predicted: "You can contact support"
Actual: "Go to ATM → Enter old PIN → Set new PIN"

This difference is quantified using a loss function.

Backpropagation (Core Idea)

After computing the error:

The model sends this error backward through the network
It figures out which weights caused the error
It adjusts those weights

Gradient Descent (Optimization)

Weights are updated using:

```
w_new = w_old - η * (∂L / ∂w)
```

Where:

η = learning rate
∂L/∂w = gradient (direction of error increase)

Intuition:

Move weights in the opposite direction of error
Repeat until error is minimized

---

2. Why Fine-Tuning LLMs?

Real-World Problem

A general LLM knows:

Wikipedia
News
General facts

But your application needs:

Banking FAQs
Legal reasoning
Medical diagnosis

Solution: Fine-Tuning

Fine-tuning means:

Adjusting model weights so it becomes specialized for a specific task.

---

3. Full Fine-Tuning

What Happens?

All model weights are updated
The entire model adapts to new data

Real-World Analogy

Like retraining a doctor from scratch to specialize in cardiology.

Advantages

High flexibility
Maximum adaptation

Challenges

1. Computational Cost

Millions or billions of parameters updated
Requires powerful GPUs

2. Catastrophic Forgetting

Model forgets general knowledge

3. Overfitting

Learns small dataset too well
Performs poorly on new data

---

4. LoRA (Low-Rank Adaptation)

Core Idea

Instead of changing all weights:

Keep original weights frozen
Learn small additional matrices

Real-World Analogy

Instead of rewriting an entire textbook:

Add sticky notes with corrections

Mathematical Insight

Original weight matrix:

```
W (D × K)
```

LoRA learns:

```
A (D × r)
B (r × K)
```

Update becomes:

```
W' = W + A × B
```

Where:

r is very small compared to D and K

Why This Works

Most useful updates lie in a low-dimensional space
We don’t need full matrix updates

Benefits

1. Fewer Trainable Parameters

   From D×K → D×r + r×K

2. Faster Training

   Less computation

3. Memory Efficient

   Only small matrices are trained

4. Knowledge Retention

   Original model remains intact

Choosing Rank (r)

Small r → faster, less expressive
Large r → better performance, more compute

Typical values:

r = 8, 16, 32

---

5. QLoRA (Quantized LoRA)

Problem with LoRA

Even if LoRA is efficient:

The base model still consumes large memory

Solution: Quantization

Convert weights from:

16-bit → 4-bit representation

Real-World Analogy

Instead of storing high-resolution images:

Store compressed versions
Slight quality loss, huge memory savings

Quantization Concept

16-bit → high precision
4-bit → only 16 possible values

What QLoRA Does

1. Quantizes base model weights (memory reduction)
2. Applies LoRA on top (efficient learning)

Key Benefits

1. Massive Memory Reduction

   Up to 4× smaller

2. Train Large Models on Small GPUs

   Even billions of parameters become manageable

3. Retains Good Performance

   Minor accuracy trade-offs

Trade-Offs

Slight accuracy loss
Slightly slower due to conversion overhead

---

6. Comparing Approaches

Method	Memory Usage	Training Cost	Performance	Use Case

Full Fine-Tuning	Very High	Very High	Best	Large-scale setups
LoRA	Medium	Low	Very Good	Most practical use cases
QLoRA	Very Low	Very Low	Good	Resource-constrained setups

---

7. Practical Perspective

Scenario: Startup Building a Chatbot

Budget constraints
Limited GPU access

Best choice:

QLoRA

Scenario: Research Lab

Access to high-end GPUs

Best choice:

Full fine-tuning or high-rank LoRA

---

8. Key Takeaways

1. Neural networks learn via:
   Error → Backpropagation → Weight updates

2. Fine-tuning adapts general models to specific tasks
3. Full fine-tuning is powerful but expensive
4. LoRA:
    - Efficient
    - Practical
    - Widely used
5. QLoRA:
    - Enables large-scale models on small hardware
    - Combines quantization + LoRA

Final Thought

Modern AI development is not just about building bigger models.

It is about:

Making powerful models usable, efficient, and adaptable to real-world constraints.

Fine-tuning techniques like LoRA and QLoRA are the key enablers of this shift.